Mathematical Modeling

Yifan Li

August 2026

Contents

Introduction **5**

1 Data Preprocessing and Statistical Foundations **8**
1.1 Data Representation and Preliminary Inspection 9
1.2 Missing Values and Duplicated Observations 9
1.3 Detection and Treatment of Outliers 11
1.4 Types and Directions of Evaluation Indicators 12
1.5 Normalization and Standardization 13
1.6 Categorical Variables and Data Transformations 14
1.7 Descriptive Statistics and Distribution Shape 15
1.8 Normality Testing 16
1.9 Correlation Coefficients 17
1.10 A Complete Preprocessing Workflow 18
1.11 MATLAB Implementation 19

2 Comprehensive Evaluation and Decision Models **22**
2.1 The Structure of a Comprehensive Evaluation Problem 23
2.2 Analytic Hierarchy Process 23
2.3 Fuzzy Comprehensive Evaluation 25
2.4 Entropy-Weight Method 27
2.5 TOPSIS 28
2.6 Grey Relational Analysis 29
2.7 Rank-Sum Ratio Method 30
2.8 VIKOR 31
2.9 Combined Subjective and Objective Weights 33
2.10 Model Selection and Sensitivity Analysis 33
2.11 MATLAB Implementation 34

3 Mathematical Programming and Optimization Models **38**
3.1 Formulation of an Optimization Model 39
3.2 Linear Programming 39
3.3 Duality and Sensitivity Analysis 40
3.4 Integer and Mixed-Integer Programming 42
3.5 Binary Programming 42
3.6 Multi-Objective and Goal Programming 43
3.7 Nonlinear Programming and Optimality Conditions 44
3.8 Dynamic Programming 45
3.9 Monte Carlo Simulation and Random Search 46
3.10 Model Verification and Sensitivity Analysis 47
3.11 MATLAB Implementation 47

4 Graph Theory and Network Optimization Models **52**
4.1 Graphs and Their Mathematical Representation 53
4.2 Shortest-Path Models 54
4.3 Dijkstra, Bellman–Ford, and Floyd–Warshall Algorithms 54
4.4 State-Expanded and Multimodal Networks 56
4.5 Minimum Spanning Trees 57
4.6 Maximum Flow and Minimum Cut 57

4.7 Minimum-Cost Flow and Transportation Networks . . . 59
4.8 The Traveling-Salesman Problem . . . 60
4.9 Selecting and Validating a Network Model . . . 61
4.10 MATLAB Implementation . . . 62

5 Interpolation, Curve Fitting, and Regression Models . . . 67
5.1 Interpolation and Approximation Problems . . . 68
5.2 Polynomial Interpolation . . . 68
5.3 Lagrange and Newton Interpolation . . . 69
5.4 Hermite and Piecewise Interpolation . . . 71
5.5 Cubic Spline Interpolation . . . 72
5.6 Two-Dimensional Interpolation . . . 73
5.7 Least-Squares Curve Fitting . . . 73
5.8 Multiple Linear Regression . . . 75
5.9 Residual Diagnostics and Multicollinearity . . . 76
5.10 Ridge and Lasso Regression . . . 77
5.11 Model Selection and Predictive Validation . . . 78
5.12 MATLAB Implementation . . . 79

6 Differential Equations and Dynamical-System Models . . . 83
6.1 From Dynamic Mechanisms to Differential Equations . . . 84
6.2 Equilibria and Linear Stability . . . 85
6.3 Numerical Solution of Differential Equations . . . 85
6.4 Malthus Exponential-Growth Model . . . 86
6.5 Logistic Growth Model . . . 87
6.6 Predator–Prey Model . . . 88
6.7 Competition Between Two Populations . . . 89
6.8 SI and SIS Epidemic Models . . . 89
6.9 SIR Epidemic Model . . . 91
6.10 SEIR, SIRS, and Extended Compartment Models . . . 92
6.11 Parameter Estimation and Model Validation . . . 93
6.12 MATLAB Implementation . . . 94

7 Time-Series Analysis and Forecasting Models . . . 98
7.1 Time-Series Components and Decomposition . . . 99
7.2 Moving-Average Models . . . 99
7.3 Simple Exponential Smoothing . . . 100
7.4 Holt Trend and Damped-Trend Models . . . 101
7.5 Holt–Winters Seasonal Models . . . 102
7.6 Stationarity, Differencing, and Autocorrelation . . . 103
7.7 Autoregressive Models . . . 104
7.8 Moving-Average and ARMA Models . . . 105
7.9 ARIMA and Seasonal ARIMA Models . . . 106
7.10 Estimation, Diagnostics, and Forecast Evaluation . . . 106
7.11 MATLAB Implementation . . . 107

8 Clustering and Classification Models . . . 113
8.1 Unsupervised and Supervised Learning . . . 114
8.2 Distances and Similarities . . . 114
8.3 Hierarchical Clustering . . . 116

8.4 K-Means Clustering . . . 117
8.5 K-Means++ and the Number of Clusters . . . 118
8.6 DBSCAN Density-Based Clustering . . . 119
8.7 K-Nearest-Neighbor Classification . . . 119
8.8 Logistic and Softmax Regression . . . 120
8.9 Decision Trees . . . 121
8.10 Support-Vector Machines . . . 121
8.11 Classification Performance Measures . . . 122
8.12 Validation and Model Selection . . . 123
8.13 MATLAB Implementation . . . 124

9 Dimension Reduction and Multivariate Analysis . . . 129
9.1 Multivariate Data and Covariance Structure . . . 130
9.2 Principal Component Analysis . . . 131
9.3 Selecting the Number of Principal Components . . . 132
9.4 Scores, Loadings, and Reconstruction . . . 133
9.5 Interpretation and Biplots . . . 133
9.6 Factor Analysis . . . 134
9.7 Factor Adequacy and Rotation . . . 135
9.8 PCA and Factor Analysis . . . 136
9.9 Principal Components in Evaluation and Clustering . . . 136
9.10 Principal Component Regression . . . 137
9.11 Validation and Common Pitfalls . . . 138
9.12 MATLAB Implementation . . . 139

10 Model Validation, Selection, and Mathematical Writing . . . 144
10.1 The Complete Modeling Process . . . 145
10.2 Assumptions and Model Boundaries . . . 145
10.3 Dimensional Consistency and Sanity Checks . . . 146
10.4 Parameter Calibration and Identifiability . . . 147
10.5 Residual Analysis and Goodness of Fit . . . 148
10.6 Training, Validation, and Generalization . . . 149
10.7 Sensitivity Analysis . . . 150
10.8 Uncertainty Quantification . . . 151
10.9 Model Comparison and Parsimony . . . 152
10.10Robust Decisions and Scenario Analysis . . . 152
10.11Structure of a Mathematical Modeling Paper . . . 153
10.12Figures, Tables, and Reproducibility . . . 154
10.13MATLAB Implementation . . . 155

Acknowledgments 161

Introduction

Mathematical modeling is the process of translating a real-world problem into a mathematical structure that can be analyzed, computed, tested, and interpreted. Unlike a conventional mathematics course organized around a single theoretical framework, mathematical modeling draws simultaneously from linear algebra, calculus, differential equations, probability, statistics, optimization, graph theory, and numerical computation. Its unity does not come from a common collection of theorems, but from a common way of thinking: identify the essential structure of a problem, represent that structure mathematically, and use the resulting model to obtain meaningful conclusions.

A real system is generally too complicated to reproduce in every detail. A model must therefore preserve the mechanisms that matter while deliberately ignoring features that are irrelevant to the purpose of the study. If the state of a system is denoted by x, its parameters by θ, its observed data by D, and the desired output by y, a model may be represented schematically as

$$y = \mathcal{M}(x, \theta; D).$$

The map $\mathcal{M}$ may be an algebraic formula, a differential equation, a probability distribution, an optimization problem, a graph, or a numerical algorithm. The appropriate form depends not only on the available data, but also on whether the objective is to describe, evaluate, predict, classify, control, or optimize the system.

Most modeling problems can be organized into the following stages:

$$\begin{aligned}\text{real problem} &\longrightarrow \text{assumptions and variables} \longrightarrow \text{mathematical formulation} \\ &\longrightarrow \text{parameter estimation and computation} \longrightarrow \text{validation} \longrightarrow \text{interpretation.}\end{aligned}$$

These stages are not strictly linear. An unreasonable prediction may reveal that an assumption is inappropriate; numerical instability may require a different formulation; and newly discovered data patterns may suggest a different family of models. Modeling is therefore an iterative process in which mathematical reasoning, computation, and interpretation continually inform one another.

The first major task is to determine what kind of question is being asked. When several alternatives must be compared using multiple indicators, evaluation models such as the analytic hierarchy process, entropy weighting, TOPSIS, and grey relational analysis may be appropriate. When limited resources must be allocated, the problem is naturally expressed through linear, integer, nonlinear, or multi-objective programming. When locations and connections are central, graph models lead to shortest paths, spanning trees, network flows, and traveling-salesman problems. When the goal is to describe relationships among variables, interpolation, curve fitting, and regression provide suitable tools. Dynamic systems are often represented by differential equations, while observations collected over time lead to time-series and forecasting models. Classification, clustering, and dimension-reduction methods are used when the main objective is to discover or predict structure in multivariate data.

Although these model families are presented separately, practical problems rarely belong to only one category. An evaluation problem may first require data normalization and objective weighting; a clustering model may be preceded by principal component analysis; an optimization problem may use parameters estimated by regression; and a time-series forecast may ultimately serve as an input to a resource-allocation model. A successful solution must therefore explain not only how a method works, but also why it is suitable for the problem and how it interacts with the other components of the analysis.

Computation plays an essential role in this process. Many models cannot be solved conveniently by hand, and even a theoretically simple method may require substantial data processing, parameter estimation, numerical optimization, or simulation. MATLAB code will therefore be included where it clarifies the implementation of a model. The purpose of the code is not to replace mathematical reasoning, but to make each step reproducible and to connect the mathematical formulation with its numerical realization. The quality of a model cannot be judged solely by whether a program produces an answer. A useful model should satisfy several requirements:

- its assumptions should be explicit and compatible with the problem;
- its variables and parameters should have clear practical meanings;
- its mathematical structure should correspond to the main mechanism of the system;
- its results should be validated using data, theoretical properties, or comparison with alternative models;
- its conclusions should remain reasonably stable under small changes in data or parameters;
- its limitations and range of applicability should be stated clearly.

There is consequently no universally best model. A more complicated model is not automatically more accurate, and a more accurate fitted model is not automatically more useful. The preferred model is usually the simplest one that captures the essential structure of the problem and produces conclusions reliable enough for the intended purpose. This balance between simplicity, accuracy, interpretability, and computational cost is one of the central judgments in mathematical modeling.

Mathematical models are the fundamental building blocks of mathematical modeling competitions and important tools for solving practical problems. A successful modeling solution requires more than identifying a familiar formula: it requires understanding the mathematical principles behind the model, recognizing the situations in which it is appropriate, explaining its assumptions and limitations, and implementing it correctly through numerical computation.

In mathematical modeling competitions, my primary responsibility has been the construction and analysis of models. I therefore need a broad and systematic understanding of the commonly used models in mathematical modeling. These notes are intended to record not only the formulas and computational steps of each method, but also its mathematical motivation, applicable scenarios, limitations, interpretation, and basic MATLAB implementation. The objective is to develop a practical reference that supports both model selection during competitions and a deeper understanding of why a model works.

These notes reorganize material collected from self-study lectures, mathematical modeling competitions, and previous modeling papers into a systematic reference. The original material contains many individual and occasionally disconnected techniques, so the methods are classified here according to the types of problems they solve. The chapters progress from data preprocessing and comprehensive evaluation to optimization, network models, regression, dynamical systems, time-series forecasting, clustering, classification, and multivariate analysis. The final chapter brings these methods together through model validation, sensitivity analysis, uncertainty quantification, model comparison, and mathematical writing. The aim is not merely to accumulate formulas or reusable code, but

to develop a coherent framework for choosing, constructing, solving, validating, and explaining mathematical models.

Chapter 1

Data Preprocessing and Statistical Foundations

1.1 Data Representation and Preliminary Inspection

Before selecting a mathematical model, the original data must be converted into a consistent mathematical form. Suppose that n observations are measured using m indicators. The resulting data matrix is

$$X = (x_{ij})_{n\times m} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1m} \\ x_{21} & x_{22} & \cdots & x_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nm} \end{pmatrix},$$

where x_{ij} is the value of the j-th indicator for the i-th observation.
The rows of X usually represent individuals, regions, years, products, or candidate plans, while the columns represent the corresponding attributes. Before performing any calculation, the practical meaning and unit of every column should be identified.
A preliminary inspection should determine whether the data contain

- missing observations;
- duplicated records;
- impossible or inconsistent values;
- abnormal observations;
- indicators measured on different scales;
- categorical variables that require numerical encoding;
- indicators whose preferred directions are inconsistent.

Example 1.1.1. Suppose that three cities are compared using annual income, commuting time, and air-quality grade:

$$X = \begin{pmatrix} 72 & 35 & \text{A} \\ 65 & 48 & \text{B} \\ 81 & 29 & \text{A} \end{pmatrix}.$$

Annual income and commuting time are numerical variables, whereas air-quality grade is categorical. Moreover, a larger income is preferred, while a smaller commuting time is preferred. Therefore, the three columns cannot be inserted directly into the same evaluation formula. The categorical variable must first be encoded, and the directions and scales of the numerical indicators must be made comparable.

Remark 1.1.2. Data preprocessing is not an independent goal. Every preprocessing operation should be chosen according to the assumptions and objectives of the model that will use the processed data.

1.2 Missing Values and Duplicated Observations

A missing value occurs when an observation that should have been recorded is unavailable. Missing values may result from measurement failure, incomplete surveys, data-entry errors, or the absence of a meaningful value.
Let $x_1, \ldots, x_n$ denote the observed values of an indicator. Common strategies for handling missing values include deletion, deterministic imputation, and model-based imputation.

Deletion. If only a very small proportion of observations are missing and the missingness is not systematically related to the variables under study, the corresponding rows or columns may be removed.

Mean imputation. For a numerical variable, a missing value may be replaced by the sample mean

$$\bar{x} = \frac{1}{n_{\text{obs}}} \sum_{i \in \mathcal{O}} x_i,$$

where $\mathcal{O}$ is the set of observed indices and $n_{\text{obs}} = |\mathcal{O}|$. Mean imputation is simple, but it reduces the observed variability and may distort correlations.

Median imputation. A missing value may instead be replaced by the sample median. This method is more robust when the data are skewed or contain extreme observations.

Grouped imputation. If observations belong to naturally different groups, the missing value may be replaced using the mean or median within the corresponding group.

Interpolation. For ordered or time-series data, a missing value between x_{t-1} and x_{t+1} may be estimated by linear interpolation:

$$\widehat{x}_t = \frac{x_{t-1} + x_{t+1}}{2}.$$

Example 1.2.1. Consider the observations

$$12, \quad 15, \quad \text{NaN}, \quad 18, \quad 20.$$

The mean of the observed values is

$$\bar{x} = \frac{12 + 15 + 18 + 20}{4} = 16.25.$$

Mean imputation replaces the missing value by 16.25. Because the missing value lies between 15 and 18, linear interpolation instead gives

$$\widehat{x}_3 = \frac{15 + 18}{2} = 16.5.$$

If these data are ordered in time, 16.5 is generally the more natural choice because it uses the local temporal structure.

Duplicated observations arise when the same record is entered more than once. Two records should be removed as duplicates only after comparing all variables that identify an observation. Repeated numerical values alone do not imply duplicated observations.

Example 1.2.2. Suppose that the records

$$(1024, \text{Philadelphia}, 18.6) \quad \text{and} \quad (1024, \text{Philadelphia}, 18.6)$$

have the same identification number, location, and measurement. If each identification number is supposed to occur only once, the second record is a duplicate. In contrast, two different identification numbers with the same location and measurement should not be deleted automatically.

1.3 Detection and Treatment of Outliers

An outlier is an observation that is unusually far from the main body of the data. An outlier may be caused by a recording error, but it may also represent a rare and important phenomenon. Therefore, detecting an outlier does not by itself justify deleting it.

Standardized-score method. For observations $x_1, \dots, x_n$, define

$$z_i = \frac{x_i - \bar{x}}{s}, \qquad s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2}.$$

A commonly used empirical rule flags x_i as a potential outlier when $|z_i| > 3$. This rule is most suitable when the data are approximately normally distributed.

Example 1.3.1. Suppose that an observation has value $x_i = 86$, while the sample mean and standard deviation are $\bar{x} = 50$ and $s = 10$. Then

$$z_i = \frac{86 - 50}{10} = 3.6.$$

Since $|z_i| > 3$, the observation should be inspected as a potential outlier. It should not be removed until its source and practical meaning have been checked.

Interquartile-range method. Let Q_1 and Q_3 be the first and third quartiles. The interquartile range is

$$\mathrm{IQR} = Q_3 - Q_1.$$

The lower and upper fences are

$$L = Q_1 - 1.5\,\mathrm{IQR}, \qquad U = Q_3 + 1.5\,\mathrm{IQR}\,.$$

An observation below L or above U is treated as a potential outlier. These fences form the basis of the standard boxplot.

Example 1.3.2. Consider the ordered data

$$1, \quad 2, \quad 2, \quad 3, \quad 3, \quad 4, \quad 4, \quad 20.$$

Using the medians of the lower and upper halves gives $Q_1 = 2$ and $Q_3 = 4$. Therefore,

$$\mathrm{IQR} = 4 - 2 = 2,$$

and the two fences are

$$L = 2 - 1.5(2) = -1, \qquad U = 4 + 1.5(2) = 7.$$

The value 20 lies above $U = 7$, so it is identified as a potential outlier.

After an outlier is detected, possible treatments include

- correcting it if it is caused by a verifiable recording error;
- retaining it if it is a valid and meaningful observation;
- applying a transformation such as $y = \log(1 + x)$;
- replacing it by a boundary value through winsorization;
- using robust statistics such as the median and IQR;
- fitting models both with and without it as a sensitivity analysis.

Remark 1.3.3. Deleting all observations outside the boxplot fences is generally inappropriate. The decision must depend on the origin of the observation and the purpose of the model.

1.4 Types and Directions of Evaluation Indicators

In a comprehensive evaluation problem, the indicators may have different preferred directions. Before weighting or aggregation, they should be transformed so that larger values consistently represent better performance. This process is called indicator orientation or indicator positivization.

Benefit-type indicator. A larger value is preferred. Examples include income, efficiency, reliability, and customer satisfaction. No directional transformation is required.

Cost-type indicator. A smaller value is preferred. Examples include cost, pollution, energy consumption, and waiting time. A simple transformation is

$$\widetilde{x}_i = \max_{1\le k\le n} x_k - x_i.$$

Alternatively, if all observations are positive, one may use $\widetilde{x}_i = 1/x_i$.

Example 1.4.1. The commuting times of three plans are

$$12, \quad 18, \quad 30.$$

Since commuting time is a cost-type indicator, the maximum-difference transformation gives

$$\widetilde{x} = (30-12,\ 30-18,\ 30-30) = (18,\ 12,\ 0).$$

The transformed values have the correct direction: a shorter commuting time corresponds to a larger score.

Target-type indicator. The value is preferred when it is close to a specified target x^*. One possible transformation is

$$\widetilde{x}_i = 1 - \frac{|x_i - x^*|}{\max_{1\le k\le n} |x_k - x^*|}.$$

Example 1.4.2. Suppose that the desired indoor temperature is $x^* = 22°\mathrm{C}$, and three measured temperatures are 20, 22, and 26. Their distances from the target are 2, 0, and 4, respectively. Thus,

$$\widetilde{x} = \left(1-\frac{2}{4}, 1-\frac{0}{4}, 1-\frac{4}{4}\right) = (0.5,\ 1,\ 0).$$

The observation exactly equal to the target receives the largest score.

Interval-type indicator. Suppose that values within $[a, b]$ are preferred. Let

$$M = \max\left\{a - \min_i x_i,\ \max_i x_i - b\right\}.$$

A commonly used transformation is

$$\widetilde{x}_i = \begin{cases} 1 - \dfrac{a - x_i}{M}, & x_i < a, \\ 1, & a \le x_i \le b, \\ 1 - \dfrac{x_i - b}{M}, & x_i > b. \end{cases}$$

Example 1.4.3. Suppose that an acceptable pH interval is $[6.5, 7.5]$, and the observed values are 6.0, 7.0, and 8.0. Here,

$$M = \max\{6.5 - 6.0,\ 8.0 - 7.5\} = 0.5.$$

Therefore,

$$\widetilde{x} = \left(1 - \frac{0.5}{0.5}, 1, 1 - \frac{0.5}{0.5}\right) = (0,\ 1,\ 0).$$

The value 7.0 lies in the preferred interval and receives score 1.

1.5 Normalization and Standardization

Even after all indicators have the same preferred direction, their units and scales may remain different. For example, income may be measured in dollars, distance in kilometers, and satisfaction on a five-point scale. Directly combining these quantities would allow indicators with larger numerical scales to dominate the result.

Min–max normalization. For a benefit-type indicator, define

$$z_{ij} = \frac{x_{ij} - \min_{1\le k\le n} x_{kj}}{\max_{1\le k\le n} x_{kj} - \min_{1\le k\le n} x_{kj}}.$$

Then $z_{ij} \in [0, 1]$, with larger values representing better performance.
For a cost-type indicator, the direction and scale can be adjusted simultaneously by

$$z_{ij} = \frac{\max_{1\le k\le n} x_{kj} - x_{ij}}{\max_{1\le k\le n} x_{kj} - \min_{1\le k\le n} x_{kj}}.$$

Example 1.5.1. Suppose that the values of a benefit-type indicator are

$$40, \quad 60, \quad 100.$$

Since the minimum is 40 and the maximum is 100, min–max normalization gives

$$z = \left(\frac{40-40}{100-40}, \frac{60-40}{100-40}, \frac{100-40}{100-40}\right) = \left(0, \frac{1}{3}, 1\right).$$

Vector normalization. For the j-th indicator, vector normalization is

$$z_{ij} = \frac{x_{ij}}{\sqrt{\sum_{k=1}^{n} x_{kj}^2}}.$$

It produces a column with Euclidean norm 1. This transformation appears naturally in TOPSIS and entropy-weight calculations.

Example 1.5.2. For the indicator vector $x = (3, 4)^{\mathsf{T}}$,

$$\|x\|_2 = \sqrt{3^2 + 4^2} = 5.$$

Hence,

$$z = \left(\frac{3}{5}, \frac{4}{5}\right)^{\mathsf{T}} = (0.6, 0.8)^{\mathsf{T}}.$$

z-score standardization. For indicator j, let

$$\bar{x}_j = \frac{1}{n}\sum_{i=1}^{n} x_{ij}, \qquad s_j = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_{ij} - \bar{x}_j)^2}.$$

The standardized observations are

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}.$$

The transformed indicator has sample mean 0 and sample standard deviation 1. Unlike min–max normalization, it is not restricted to $[0, 1]$.

Example 1.5.3. Consider the data $2, 4, 6$. The sample mean is $\bar{x} = 4$, and the sample standard deviation is

$$s = \sqrt{\frac{(2-4)^2 + (4-4)^2 + (6-4)^2}{2}} = 2.$$

Therefore, the standardized observations are

$$z = (-1, 0, 1).$$

Remark 1.5.4. Min–max normalization preserves the relative positions of observations within a fixed interval but is sensitive to extreme values. Standardization is often preferable for regression, principal component analysis, and distance-based methods. Vector normalization is especially common in comprehensive evaluation models.

If all observations in an indicator are equal, then min–max normalization and z-score standardization involve division by zero. Such a column has no ability to distinguish the observations and should usually be removed from an evaluation model.

1.6 Categorical Variables and Data Transformations

Some models require numerical input even when the original variable is categorical. A categorical variable may be nominal or ordinal.
A nominal variable has no natural ordering. Examples include city, color, and transportation type. It should usually be represented by dummy variables or one-hot encoding.

Example 1.6.1. Suppose that transportation type takes the values

$$\{\text{car}, \text{bus}, \text{bicycle}\}.$$

One-hot encoding gives

Type	Car	Bus	Bicycle
car	1	0	0
bus	0	1	0
bicycle	0	0	1

This representation avoids incorrectly imposing a numerical order on the three transportation types.

An ordinal variable has a meaningful order but does not necessarily have equal distances between adjacent categories. For example, the grades A, B, C, and D may be encoded as $4, 3, 2, 1$ when only their order is important. However, this coding implicitly treats the distance from A to B as equal to the distance from B to C, which should be justified before the encoded values are used quantitatively.
For a positive, strongly right-skewed variable, a logarithmic transformation may be useful:

$$y_i = \log(1 + x_i).$$

The addition of 1 allows the transformation to be applied when $x_i = 0$.

Example 1.6.2. Suppose that the original values are 0, 9, and 99. Their range is 99, while the transformed values are

$$0, \quad \log(10), \quad \log(100).$$

The logarithmic transformation compresses the large observation and reduces the influence of the right tail.

1.7 Descriptive Statistics and Distribution Shape

Descriptive statistics summarize the location, dispersion, and shape of a sample before a more complicated model is fitted.
For observations $x_1, \ldots, x_n$, the sample mean is

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i,$$

and the sample variance and standard deviation are

$$s^2 = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})^2, \qquad s = \sqrt{s^2}.$$

The coefficient of variation is

$$\mathrm{CV} = \frac{s}{|\bar{x}|}, \qquad \bar{x} \neq 0.$$

It measures relative dispersion and is useful when variables have different units or substantially different means.

Example 1.7.1. Two production processes have

$$\bar{x}_1 = 100, \quad s_1 = 5, \qquad \bar{x}_2 = 20, \quad s_2 = 2.$$

Their coefficients of variation are

$$\mathrm{CV}_1 = \frac{5}{100} = 0.05, \qquad \mathrm{CV}_2 = \frac{2}{20} = 0.10.$$

Although the first process has the larger standard deviation, the second has greater variability relative to its mean.

The skewness coefficient describes the asymmetry of a distribution. A moment-based sample skewness measure is

$$\text{Skewness} = \frac{n}{(n-1)(n-2)} \sum_{i=1}^{n} \left(\frac{x_i - \bar{x}}{s} \right)^3, \qquad n \geq 3.$$

A positive skewness indicates a longer right tail, a negative skewness indicates a longer left tail, and a skewness near 0 indicates approximate symmetry. Skewness equal to 0 does not by itself imply normality.

Example 1.7.2. Consider the data

$$1, \quad 2, \quad 2, \quad 3, \quad 7.$$

Most values are concentrated between 1 and 3, while the observation 7 extends the right tail. The distribution is therefore positively skewed. Its mean is $\bar{x} = 3$, which is larger than its median 2, as is often observed in right-skewed data.

1.8 Normality Testing

Many statistical procedures are derived under a normality assumption. Normality should be assessed using both graphical and numerical evidence. Histograms, quantile–quantile plots, skewness, and formal hypothesis tests provide complementary information.
The Shapiro–Wilk test uses the hypotheses

$$H_0 : \text{the data are sampled from a normal distribution,}$$

$$H_1 : \text{the data are not sampled from a normal distribution.}$$

Let $x_{(1)} \leq \cdots \leq x_{(n)}$ denote the ordered observations. The Shapiro–Wilk statistic has the form

$$W = \frac{\left(\sum_{i=1}^{n} a_i x_{(i)} \right)^2}{\sum_{i=1}^{n} (x_i - \bar{x})^2},$$

where the coefficients a_i are determined by the expected values and covariance matrix of normal order statistics.
For a significance level α, the usual decision rule is

- if $p \leq \alpha$, reject H_0;
- if $p > \alpha$, do not reject H_0.

Example 1.8.1. Suppose that a Shapiro–Wilk test applied to a sample gives

$$W = 0.974, \qquad p = 0.620.$$

At significance level $\alpha = 0.05$, we have $p > \alpha$. Therefore, there is insufficient evidence to reject the normality assumption. This does not prove that the population is exactly normal; it only means that the observed sample does not provide strong evidence against normality.

Remark 1.8.2. For a very large sample, a formal test may detect a small and practically unimportant departure from normality. For a very small sample, the same test may fail to detect a substantial departure. A Q–Q plot and the practical robustness of the intended model should also be considered.

1.9 Correlation Coefficients

Correlation coefficients measure the strength and direction of association between two variables. They do not by themselves establish a causal relationship.

Pearson correlation coefficient. For paired observations (x_i, y_i), the Pearson correlation coefficient is

$$r = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}}.$$

It measures linear association and satisfies $-1 \le r \le 1$.

Example 1.9.1. Let

$$x = (1, 2, 3, 4), \qquad y = (2, 1, 4, 3).$$

Both variables have mean 2.5. Direct calculation gives

$$\sum_{i=1}^{4}(x_i - \bar{x})(y_i - \bar{y}) = 3$$

and

$$\sum_{i=1}^{4}(x_i - \bar{x})^2 = \sum_{i=1}^{4}(y_i - \bar{y})^2 = 5.$$

Therefore,

$$r = \frac{3}{\sqrt{5}\sqrt{5}} = 0.6.$$

The two variables have a moderate positive linear association.

Spearman rank correlation coefficient. Spearman correlation is the Pearson correlation between the ranks of the observations. When there are no tied ranks, it can be calculated by

$$\rho_s = 1 - \frac{6\sum_{i=1}^{n} d_i^2}{n(n^2 - 1)},$$

where d_i is the difference between the two ranks for the i-th observation. Spearman correlation measures monotonic association and is less dependent on distributional assumptions than Pearson correlation.

Example 1.9.2. For

$$x = (1, 2, 3, 4), \qquad y = (2, 1, 4, 3),$$

the ranks of x are $(1, 2, 3, 4)$, while the ranks of y are $(2, 1, 4, 3)$. Hence,

$$d = (-1, 1, -1, 1), \qquad \sum_{i=1}^{4} d_i^2 = 4.$$

Therefore,

$$\rho_s = 1 - \frac{6(4)}{4(4^2 - 1)} = 0.6.$$

Kendall rank correlation coefficient. For two observations $i < j$, the pair is concordant if the order of x_i, x_j agrees with the order of y_i, y_j, and discordant if the orders disagree. When there are no ties, Kendall's coefficient is

$$\tau = \frac{C - D}{\binom{n}{2}},$$

where C and D are the numbers of concordant and discordant pairs.

Example 1.9.3. For the same data

$$x = (1, 2, 3, 4), \qquad y = (2, 1, 4, 3),$$

there are $\binom{4}{2} = 6$ pairs. Four are concordant and two are discordant. Hence,

$$\tau = \frac{4 - 2}{6} = \frac{1}{3}.$$

The positive value again indicates a positive monotonic association.

The three coefficients have different interpretations:

- Pearson correlation measures linear association;
- Spearman correlation measures monotonic rank association;
- Kendall correlation measures agreement between pairwise orderings.

A common descriptive classification uses $|r| > 0.7$ for strong correlation, $0.3 \leq |r| \leq 0.7$ for moderate correlation, and $|r| < 0.3$ for weak correlation. These thresholds are only conventions and should not replace subject-specific interpretation.

Remark 1.9.4. A correlation coefficient close to 0 indicates the absence of the type of association measured by that coefficient. In particular, $r = 0$ does not exclude a strong nonlinear relationship.

Example 1.9.5. Let X be uniformly distributed on $[-1, 1]$, and let $Y = X^2$. Then

$$\mathrm{Cov}(X, Y) = \mathbb{E}[X^3] - \mathbb{E}[X]\mathbb{E}[X^2] = 0.$$

Thus, X and Y have zero Pearson correlation, even though Y is completely determined by X.

1.10 A Complete Preprocessing Workflow

A practical preprocessing procedure can be summarized as follows:

1. identify the meaning, type, and unit of every variable;
2. remove verifiable duplicates and correct impossible values;
3. inspect the pattern and proportion of missing values;
4. choose an appropriate missing-value treatment;
5. detect potential outliers and investigate their origins;
6. encode categorical variables when numerical input is required;
7. transform cost-type, target-type, and interval-type indicators;
8. normalize or standardize numerical indicators;
9. calculate descriptive statistics and inspect distributions;
10. examine correlations and possible redundant indicators;
11. record every preprocessing operation so that the analysis is reproducible.

Example 1.10.1. Suppose that four candidate cities are evaluated using annual income, commuting time, and public-service satisfaction:

City	Income	Commuting Time	Satisfaction
A	60	40	4.2
B	75	55	4.6
C	68	35	4.1
D	82	50	NaN

Income and satisfaction are benefit-type indicators, whereas commuting time is a cost-type indicator. First, the missing satisfaction value may be imputed using the median:

$$\text{median}(4.2, 4.6, 4.1) = 4.2.$$

Next, apply min–max normalization. The normalized income values are

$$z^{(1)} = \left(0,\ \frac{15}{22},\ \frac{8}{22},\ 1\right),$$

and the normalized commuting-time scores are

$$z^{(2)} = \left(\frac{15}{20},\ 0,\ 1,\ \frac{5}{20}\right).$$

The normalized satisfaction values are

$$z^{(3)} = \left(\frac{1}{5},\ 1,\ 0,\ \frac{1}{5}\right).$$

The resulting matrix has consistent direction and scale:

$$Z = \begin{pmatrix} 0 & 0.75 & 0.20 \\ 0.6818 & 0 & 1 \\ 0.3636 & 1 & 0 \\ 1 & 0.25 & 0.20 \end{pmatrix}.$$

This matrix can now be used in a weighting, TOPSIS, clustering, or other multivariate model. However, the effect of imputing the missing value should still be examined in a sensitivity analysis.

1.11 MATLAB Implementation

The following MATLAB template performs several common preprocessing operations. The precise procedure should be adjusted according to the meaning of the variables and the assumptions of the subsequent model.

```
% Read data from a spreadsheet
T = readtable('data.xlsx');

% Display variable types and summary information
summary(T);

% Remove exact duplicated rows
T = unique(T, 'rows', 'stable');

```

```
% Fill missing numerical values by the column median
numericMask = varfun(@isnumeric, T, ...
    'OutputFormat', 'uniform');
numericNames = T.Properties.VariableNames(numericMask);

for j = 1:numel(numericNames)
    name = numericNames{j};
    T.(name) = fillmissing(T.(name), 'constant', ...
        median(T.(name), 'omitnan'));
end

% Extract the numerical data matrix
X = T{:, numericNames};

% Detect possible outliers by the IQR rule
outlierMask = isoutlier(X, 'quartiles');

% Min-max normalization to [0,1]
XMinMax = normalize(X, 'range');

% Z-score standardization
XZScore = normalize(X, 'zscore');

% Pearson, Spearman, and Kendall correlation matrices
RPearson = corr(X, 'Type', 'Pearson', ...
    'Rows', 'pairwise');
RSpearman = corr(X, 'Type', 'Spearman', ...
    'Rows', 'pairwise');
RKendall = corr(X, 'Type', 'Kendall', ...
    'Rows', 'pairwise');

% Descriptive plots
figure;
boxplot(X);
title('Boxplots of Numerical Indicators');

figure;
corrplot(X);
title('Correlation Matrix');
```

Listing 1.1: A basic MATLAB data-preprocessing template

For a vector x, the Shapiro–Wilk test is not included as a standard built-in test in every MATLAB version. If a reliable implementation is available, it may be applied directly. Otherwise, normality can be examined using a normal probability plot together with an available formal test.

```
x = X(:,1);

% Normal probability plot
figure;
qqplot(x);
title('Normal Q-Q Plot');
```

```
7
8  % Lilliefors normality test
9  % h = 0 means that normality is not rejected
10 % h = 1 means that normality is rejected
11 [h, pValue] = lillietest(x);
12
13 fprintf('Test decision h = %d\n', h);
14 fprintf('p-value = %.4f\n', pValue);
```

Listing 1.2: Graphical and numerical normality assessment

Remark 1.11.1. The result returned by a function should always be interpreted together with the mathematical definition of the method. Calling `normalize`, `corr`, or `isoutlier` does not determine whether the operation is appropriate for a particular modeling problem.

Chapter 2

Comprehensive Evaluation and Decision Models

2.1 The Structure of a Comprehensive Evaluation Problem

A comprehensive evaluation problem compares several alternatives using multiple indicators. Suppose that there are n alternatives and m evaluation indicators. After indicator orientation and preprocessing, the data matrix is

$$X = (x_{ij})_{n\times m},$$

where x_{ij} is the performance of alternative i under indicator j.
A general comprehensive evaluation model assigns each alternative a score

$$S_i = F(x_{i1}, x_{i2}, \ldots, x_{im}; w_1, w_2, \ldots, w_m),$$

where $w_j \geq 0$ is the weight of indicator j, with

$$\sum_{j=1}^{m} w_j = 1.$$

The alternatives are then ranked according to their scores.
The main components of an evaluation problem are

- the set of alternatives;
- the evaluation-indicator system;
- the orientation and normalization of the indicators;
- the method used to determine indicator weights;
- the aggregation or ranking model;
- the robustness and sensitivity of the final ranking.

Weighting methods can be divided into three broad categories:

- subjective methods, such as the analytic hierarchy process;
- objective methods, such as the entropy-weight method;
- combined methods that integrate subjective and objective weights.

Example 2.1.1. Suppose that three locations are evaluated for a new distribution center using construction cost, transportation convenience, and market potential. Construction cost is a cost-type indicator, while transportation convenience and market potential are benefit-type indicators. A complete evaluation must first transform construction cost into a benefit-type score, standardize all three indicators, determine their weights, and then apply an appropriate ranking model such as TOPSIS or VIKOR.

2.2 Analytic Hierarchy Process

The analytic hierarchy process, abbreviated as AHP, is a subjective weighting method for problems with a hierarchical decision structure. It is especially useful when expert judgment is important and the relative importance of the criteria cannot be determined directly from data.
An AHP model usually contains three levels:

- the goal level, representing the overall decision objective;

- the criterion level, containing the evaluation criteria;
- the alternative level, containing the candidate plans.

Pairwise comparison matrix. Suppose that m criteria are compared pairwise. Their relative importance is represented by the positive reciprocal matrix

$$A = (a_{ij})_{m \times m},$$

where a_{ij} measures the importance of criterion i relative to criterion j. The matrix satisfies

$$a_{ij} > 0, \qquad a_{ii} = 1, \qquad a_{ji} = \frac{1}{a_{ij}}.$$

A frequently used comparison scale is

a_{ij}	Interpretation
1	Equal importance
3	Moderate importance
5	Strong importance
7	Very strong importance
9	Extreme importance
$2, 4, 6, 8$	Intermediate judgments

Weight calculation by the eigenvector method. The weight vector w is obtained from the principal eigenvector:

$$Aw = \lambda_{\max} w, \qquad \sum_{j=1}^{m} w_j = 1.$$

An approximate alternative is the arithmetic-mean method. First normalize each column:

$$\widetilde{a}_{ij} = \frac{a_{ij}}{\sum_{k=1}^{m} a_{kj}},$$

and then average the rows:

$$w_i = \frac{1}{m} \sum_{j=1}^{m} \widetilde{a}_{ij}.$$

Consistency test. A perfectly consistent judgment matrix satisfies $a_{ik} a_{kj} = a_{ij}$. Human judgments are rarely perfectly consistent, so AHP measures the degree of inconsistency using

$$\mathrm{CI} = \frac{\lambda_{\max} - m}{m - 1}.$$

The consistency ratio is

$$\mathrm{CR} = \frac{\mathrm{CI}}{\mathrm{RI}},$$

where RI is the average random consistency index.
Common values of RI are

m	1	2	3	4	5	6	7	8	9
RI	0	0	0.58	0.90	1.12	1.24	1.32	1.41	1.45

The judgment matrix is generally regarded as having acceptable consistency when $\mathrm{CR} < 0.1$. Otherwise, the pairwise judgments should be reconsidered.

Example 2.2.1. Suppose that quality, price, and service are compared using

$$A = \begin{pmatrix} 1 & 2 & 5 \\ 1/2 & 1 & 2 \\ 1/5 & 1/2 & 1 \end{pmatrix}.$$

The principal eigenvalue and normalized principal eigenvector are approximately

$$\lambda_{\max} = 3.0055, \qquad w = \begin{pmatrix} 0.5954 \\ 0.2764 \\ 0.1283 \end{pmatrix}.$$

Thus, quality receives the greatest weight, followed by price and service.
The consistency index is

$$\mathrm{CI} = \frac{3.0055 - 3}{3 - 1} = 0.0028.$$

Since $\mathrm{RI} = 0.58$ for a matrix of order 3,

$$\mathrm{CR} = \frac{0.0028}{0.58} \approx 0.0048 < 0.1.$$

Therefore, the consistency of the judgment matrix is acceptable.

Remark 2.2.2. AHP does not eliminate subjectivity. It organizes subjective judgments into a transparent structure and tests whether those judgments are approximately consistent.

2.3 Fuzzy Comprehensive Evaluation

Fuzzy comprehensive evaluation is designed for objects whose boundaries or evaluation grades are not sharply defined. Examples include comfort, safety, environmental quality, customer satisfaction, and overall risk.
Fuzzy set and membership function. Let U be a universe of discourse. A fuzzy set A on U is described by a membership function

$$\mu_A : U \longrightarrow [0, 1].$$

The value $\mu_A(x)$ represents the degree to which x belongs to the fuzzy set A.
For example, the statement that a temperature is "comfortable" does not usually have a sharp boundary. A temperature may belong to the comfortable set with membership degree 0.8 and to the warm set with membership degree 0.2.
A triangular membership function can be written as

$$\mu_A(x) = \begin{cases} 0, & x \le a, \\ \dfrac{x - a}{b - a}, & a < x \le b, \\ \dfrac{c - x}{c - b}, & b < x < c, \\ 0, & x \ge c. \end{cases}$$

Factor and evaluation sets. Let

$$U = \{u_1, u_2, \ldots, u_m\}$$

be the factor set, and let

$$V = \{v_1, v_2, \ldots, v_s\}$$

be the evaluation-grade set.
For each factor u_i, its membership degrees in the s evaluation grades form the vector

$$r_i = (r_{i1}, r_{i2}, \ldots, r_{is}).$$

The fuzzy evaluation matrix is

$$R = \begin{pmatrix} r_{11} & r_{12} & \cdots & r_{1s} \\ r_{21} & r_{22} & \cdots & r_{2s} \\ \vdots & \vdots & \ddots & \vdots \\ r_{m1} & r_{m2} & \cdots & r_{ms} \end{pmatrix},$$

where each row usually satisfies

$$\sum_{j=1}^{s} r_{ij} = 1.$$

If the factor-weight vector is

$$w = (w_1, w_2, \ldots, w_m)^{\mathsf{T}},$$

then the fuzzy comprehensive evaluation vector is

$$B = w^{\mathsf{T}} R.$$

The final grade may be determined by the maximum-membership principle. If the grades are assigned numerical scores

$$c = (c_1, c_2, \ldots, c_s)^{\mathsf{T}},$$

a numerical comprehensive score is

$$S = Bc.$$

Example 2.3.1. A restaurant is evaluated using service quality and price reasonableness. The evaluation grades are

$$V = \{\text{excellent}, \text{good}, \text{average}\}.$$

Suppose that the factor weights and fuzzy evaluation matrix are

$$w = \begin{pmatrix} 0.6 \\ 0.4 \end{pmatrix}, \qquad R = \begin{pmatrix} 0.7 & 0.2 & 0.1 \\ 0.2 & 0.5 & 0.3 \end{pmatrix}.$$

Then

$$B = w^{\mathsf{T}} R = (0.6, 0.4) \begin{pmatrix} 0.7 & 0.2 & 0.1 \\ 0.2 & 0.5 & 0.3 \end{pmatrix} = (0.50, 0.32, 0.18).$$

The largest membership degree is 0.50, corresponding to "excellent."
If the three grades are assigned the scores 100, 80, and 60, then

$$S = (0.50, 0.32, 0.18) \begin{pmatrix} 100 \\ 80 \\ 60 \end{pmatrix} = 86.4.$$

Remark 2.3.2. The result of a fuzzy evaluation depends on the membership functions, factor weights, and grade scores. These components should be justified rather than selected only to produce a desired conclusion.

2.4 Entropy-Weight Method

The entropy-weight method determines indicator weights from the dispersion of the observed data. It is an objective weighting method in the sense that the weights are calculated from the data rather than assigned directly by experts.

If an indicator takes nearly identical values for all alternatives, it provides little information for distinguishing the alternatives and receives a small weight. Greater dispersion corresponds to lower information entropy and a larger weight.

Suppose that the positively oriented and normalized data matrix is

$$Z = (z_{ij})_{n\times m}, \qquad z_{ij} \geq 0.$$

Define

$$p_{ij} = \frac{z_{ij}}{\sum_{i=1}^{n} z_{ij}}.$$

The entropy of indicator j is

$$e_j = -\frac{1}{\ln n} \sum_{i=1}^{n} p_{ij} \ln p_{ij},$$

where $p_{ij} \ln p_{ij}$ is defined to be 0 when $p_{ij} = 0$.

The information utility is

$$d_j = 1 - e_j,$$

and the entropy weight is

$$w_j = \frac{d_j}{\sum_{k=1}^{m} d_k}.$$

Example 2.4.1. Suppose that the normalized evaluation matrix is

$$Z = \begin{pmatrix} 0.8 & 0.6 & 0.7 \\ 0.6 & 0.9 & 0.8 \\ 0.9 & 0.5 & 0.6 \end{pmatrix}.$$

The column proportions are approximately

$$P = \begin{pmatrix} 0.3478 & 0.3000 & 0.3333 \\ 0.2609 & 0.4500 & 0.3810 \\ 0.3913 & 0.2500 & 0.2857 \end{pmatrix}.$$

The corresponding entropy values are

$$e = \begin{pmatrix} 0.9876 \\ 0.9713 \\ 0.9938 \end{pmatrix},$$

so the information utilities are

$$d = \begin{pmatrix} 0.0124 \\ 0.0287 \\ 0.0062 \end{pmatrix}.$$

After normalization, the entropy weights are

$$w = \begin{pmatrix} 0.2618 \\ 0.6068 \\ 0.1314 \end{pmatrix}.$$

The second indicator has the largest weight because its values differ most substantially among the alternatives.

Remark 2.4.2. A large entropy weight means that an indicator distinguishes the observed alternatives strongly. It does not necessarily mean that the indicator is more important in the practical problem.

2.5 TOPSIS

The technique for order preference by similarity to an ideal solution, abbreviated as TOPSIS, ranks alternatives according to their distances from an ideal best solution and an ideal worst solution.
TOPSIS is suitable when

- several alternatives are evaluated using multiple indicators;
- the indicators can be oriented and normalized;
- a weight can be assigned to each indicator;
- closeness to an ideal point has a meaningful interpretation.

Suppose that $Z = (z_{ij})_{n\times m}$ is the positively oriented normalized matrix. For benefit-type normalized indicators, define the positive and negative ideal solutions by

$$z_j^+ = \max_{1\le i\le n} z_{ij}, \qquad z_j^- = \min_{1\le i\le n} z_{ij}.$$

Thus,

$$z^+ = (z_1^+, \dots, z_m^+), \qquad z^- = (z_1^-, \dots, z_m^-).$$

With indicator weights w_j, the weighted distances are

$$D_i^+ = \sqrt{\sum_{j=1}^{m} w_j (z_{ij} - z_j^+)^2}$$

and

$$D_i^- = \sqrt{\sum_{j=1}^{m} w_j (z_{ij} - z_j^-)^2}.$$

The relative closeness to the ideal solution is

$$C_i = \frac{D_i^-}{D_i^+ + D_i^-}.$$

Since $0 \le C_i \le 1$, a larger C_i indicates a better alternative.

Example 2.5.1. Consider the positively oriented data

$$X = \begin{pmatrix} 0.8 & 0.6 & 0.7 \\ 0.6 & 0.9 & 0.8 \\ 0.9 & 0.5 & 0.6 \end{pmatrix}.$$

Vector normalization gives

$$Z \approx \begin{pmatrix} 0.5946 & 0.5035 & 0.5735 \\ 0.4460 & 0.7553 & 0.6554 \\ 0.6690 & 0.4196 & 0.4915 \end{pmatrix}.$$

Using equal weights $w_1 = w_2 = w_3 = 1/3$, the positive and negative ideal solutions are

$$z^+ = (0.6690, 0.7553, 0.6554)$$

and

$$z^- = (0.4460, 0.4196, 0.4915).$$

The distances are approximately

$$D^+ = \begin{pmatrix} 0.1588 \\ 0.1287 \\ 0.2157 \end{pmatrix}, \qquad D^- = \begin{pmatrix} 0.1093 \\ 0.2157 \\ 0.1287 \end{pmatrix}.$$

Therefore,

$$C = \begin{pmatrix} 0.4078 \\ 0.6262 \\ 0.3738 \end{pmatrix}.$$

The ranking is

$$A_2 \succ A_1 \succ A_3.$$

Remark 2.5.2. Normalization and indicator orientation are different operations. Orientation ensures that larger values have the same meaning, while normalization removes the effect of scale. Both must be completed before TOPSIS is applied.

2.6 Grey Relational Analysis

Grey relational analysis measures the similarity between the geometric shapes of several data sequences. It is useful when the sample is small, the information is incomplete, or no strong probability-distribution assumption is available.
Let the reference sequence be

$$x_0 = (x_0(1), x_0(2), \ldots, x_0(m)),$$

and let the comparison sequences be

$$x_i = (x_i(1), x_i(2), \ldots, x_i(m)), \qquad i = 1, \ldots, n.$$

The sequences should first be made dimensionless and positively oriented.
Define the absolute difference

$$\Delta_i(k) = |x_0(k) - x_i(k)|.$$

Let

$$\Delta_{\min} = \min_{i,k} \Delta_i(k), \qquad \Delta_{\max} = \max_{i,k} \Delta_i(k).$$

The grey relational coefficient is

$$\xi_i(k) = \frac{\Delta_{\min} + \rho\Delta_{\max}}{\Delta_i(k) + \rho\Delta_{\max}},$$

where $\rho \in (0, 1)$ is the distinguishing coefficient. A common choice is $\rho = 0.5$.
With indicator weights w_k, the grey relational grade is

$$\gamma_i = \sum_{k=1}^{m} w_k \xi_i(k).$$

A larger γ_i indicates that the comparison sequence is more similar to the reference sequence.

Example 2.6.1. Suppose that the reference sequence and three alternatives are

$$x_0 = (9, 8, 10),$$

$$x_1 = (8, 7, 9), \qquad x_2 = (6, 8, 7), \qquad x_3 = (9, 5, 8).$$

The absolute-difference matrix is

$$\Delta = \begin{pmatrix} 1 & 1 & 1 \\ 3 & 0 & 3 \\ 0 & 3 & 2 \end{pmatrix}.$$

Thus, $\Delta_{\min} = 0$ and $\Delta_{\max} = 3$. Taking $\rho = 0.5$, the grey relational coefficients are

$$\Xi = \begin{pmatrix} 0.6000 & 0.6000 & 0.6000 \\ 0.3333 & 1.0000 & 0.3333 \\ 1.0000 & 0.3333 & 0.4286 \end{pmatrix}.$$

Using equal weights, the relational grades are

$$\gamma_1 = 0.6000, \qquad \gamma_2 = 0.5556, \qquad \gamma_3 = 0.5873.$$

Therefore,

$$A_1 \succ A_3 \succ A_2.$$

Remark 2.6.2. Grey relational analysis measures similarity of variation patterns. Its ranking should not automatically be interpreted as a direct measurement of absolute performance.

2.7 Rank-Sum Ratio Method

The rank-sum ratio method, abbreviated as RSR, converts multiple indicator values into ranks and then combines the ranks into a dimensionless score. It is useful when the indicators have different units or when the original data do not satisfy strong distributional assumptions.

For each indicator j, assign a rank R_{ij} to alternative i. Larger ranks should always represent better performance. Therefore,

- a benefit-type indicator is ranked from small to large;
- a cost-type indicator is ranked from large to small;
- tied observations receive average ranks.

The unweighted rank-sum ratio is

$$\mathrm{RSR}_i = \frac{1}{nm} \sum_{j=1}^{m} R_{ij}.$$

The weighted rank-sum ratio is

$$\mathrm{WRSR}_i = \frac{1}{n} \sum_{j=1}^{m} w_j R_{ij}, \qquad \sum_{j=1}^{m} w_j = 1.$$

A larger RSR or WRSR value indicates better overall performance.

Example 2.7.1. Suppose that three alternatives are ranked under three benefit-type indicators:

Alternative	R_1	R_2	R_3
A	3	2	3
B	2	3	2
C	1	1	1

The unweighted RSR values are

$$\mathrm{RSR}_A = \frac{3+2+3}{3(3)} = \frac{8}{9} \approx 0.8889,$$

$$\mathrm{RSR}_B = \frac{2+3+2}{9} = \frac{7}{9} \approx 0.7778,$$

and

$$\mathrm{RSR}_C = \frac{1+1+1}{9} = \frac{1}{3}.$$

Thus,

$$A \succ B \succ C.$$

For a more refined classification, the RSR values may be converted into cumulative frequencies and Probit values. If a cumulative frequency is p, the corresponding Probit value is

$$\mathrm{Probit} = 5 + \Phi^{-1}(p),$$

where Φ^{-1} is the inverse standard normal distribution function. A regression of RSR on Probit can then be used to construct evaluation-grade boundaries.

Remark 2.7.2. RSR is robust to measurement scale because it uses ranks, but ranking also discards information about the magnitudes of the original differences.

2.8 VIKOR

VIKOR is a multi-criteria compromise-ranking method. It seeks an alternative that balances overall performance against the largest individual regret.

Suppose that f_{ij} is the performance of alternative i under criterion j. For a benefit-type criterion, define

$$f_j^* = \max_i f_{ij}, \qquad f_j^- = \min_i f_{ij}.$$

For a cost-type criterion, the definitions are reversed.

The normalized regret under criterion j is

$$d_{ij} = \frac{f_j^* - f_{ij}}{f_j^* - f_j^-}.$$

For cost-type criteria, the same formula applies after the appropriate best and worst values have been defined.

The group-utility measure is

$$S_i = \sum_{j=1}^{m} w_j d_{ij},$$

and the individual-regret measure is

$$R_i = \max_{1 \le j \le m} \{w_j d_{ij}\}.$$

Smaller values of S_i and R_i are preferred.
Let

$$S^* = \min_i S_i, \qquad S^- = \max_i S_i,$$
$$R^* = \min_i R_i, \qquad R^- = \max_i R_i.$$

The VIKOR index is

$$Q_i = v\frac{S_i - S^*}{S^- - S^*} + (1-v)\frac{R_i - R^*}{R^- - R^*},$$

where $v \in [0,1]$ represents the emphasis placed on group utility. A common choice is $v = 0.5$. The alternatives are ranked from the smallest Q_i to the largest.

Example 2.8.1. Consider three alternatives with equal criterion weights:

$$X = \begin{pmatrix} 0.8 & 0.6 & 0.7 \\ 0.6 & 0.9 & 0.8 \\ 0.9 & 0.5 & 0.6 \end{pmatrix}, \qquad w_1 = w_2 = w_3 = \frac{1}{3}.$$

All criteria are benefit-type. The normalized-regret matrix is

$$D = \begin{pmatrix} 1/3 & 3/4 & 1/2 \\ 1 & 0 & 0 \\ 0 & 1 & 1 \end{pmatrix}.$$

Therefore,

$$S = \begin{pmatrix} 0.5278 \\ 0.3333 \\ 0.6667 \end{pmatrix}, \qquad R = \begin{pmatrix} 0.2500 \\ 0.3333 \\ 0.3333 \end{pmatrix}.$$

With $v = 0.5$, the VIKOR indices are

$$Q = \begin{pmatrix} 0.2917 \\ 0.5000 \\ 1.0000 \end{pmatrix}.$$

Thus, the VIKOR ranking is

$$A_1 \succ A_2 \succ A_3.$$

Alternative A_2 has the best average performance measured by S_i, but alternative A_1 has the smallest maximum regret. VIKOR balances these two considerations and ranks A_1 first.

If $A^{(1)}$ and $A^{(2)}$ are the first and second alternatives in the Q-ranking, a unique compromise solution is commonly accepted when

$$Q(A^{(2)}) - Q(A^{(1)}) \geq \frac{1}{n-1}$$

and $A^{(1)}$ is also ranked first by either S or R. If these conditions fail, a set of compromise alternatives should be reported.

Remark 2.8.2. TOPSIS rewards geometric closeness to an ideal point, whereas VIKOR explicitly balances total regret and worst-criterion regret. The two models may therefore produce different rankings from the same data.

2.9 Combined Subjective and Objective Weights

Subjective weights reflect expert judgment, while objective weights reflect the observed dispersion of the data. A simple linear combination is

$$w_j = \alpha w_j^{(s)} + (1-\alpha) w_j^{(o)}, \qquad 0 \le \alpha \le 1,$$

where $w_j^{(s)}$ is the subjective weight and $w_j^{(o)}$ is the objective weight.

Example 2.9.1. Suppose that AHP produces the subjective weights

$$w^{(s)} = (0.5, 0.3, 0.2),$$

while the entropy-weight method produces

$$w^{(o)} = (0.2, 0.5, 0.3).$$

If subjective judgment is assigned proportion $\alpha = 0.6$, then

$$w = 0.6w^{(s)} + 0.4w^{(o)} = (0.38, 0.38, 0.24).$$

The combined weighting reduces the dominance of the first subjective weight while preserving expert judgment.

The value of α should be justified through expert confidence, cross-validation, sensitivity analysis, or an optimization criterion. It should not be chosen only because it produces a preferred ranking.

2.10 Model Selection and Sensitivity Analysis

The evaluation models in this chapter answer different questions:

Model	**Primary purpose**
AHP	Determine subjective weights from pairwise comparisons.
Entropy weighting	Determine objective weights from data dispersion.
Fuzzy evaluation	Aggregate vague or linguistically defined evaluation grades.
TOPSIS	Rank alternatives by closeness to ideal and anti-ideal points.
Grey relation	Measure similarity between comparison sequences and a reference.
RSR	Combine indicators through ranks with weak distributional assumptions.
VIKOR	Identify a compromise between total performance and worst regret.

A ranking should not be reported without examining its stability. If $S_i(w)$ is the score of alternative i, a local sensitivity analysis changes the weights slightly:

$$w_j^{(\delta)} = \frac{w_j(1+\delta_j)}{\sum_{k=1}^{m} w_k(1+\delta_k)}.$$

The evaluation model is then recalculated under several perturbations δ_j. If small changes in the weights cause large changes in the ranking, the decision should be reported as unstable.

Example 2.10.1. Suppose that two alternatives have TOPSIS scores 0.601 and 0.598. Although the first is technically ranked higher, their difference is only 0.003. If a 5% change in one indicator weight reverses the ranking, the two alternatives should be described as practically indistinguishable rather than as having a decisive ordering.

2.11 MATLAB Implementation

The following MATLAB code implements AHP, entropy weighting, TOPSIS, grey relational analysis, and VIKOR. It assumes that all indicators have already been positively oriented.

```
% Pairwise comparison matrix
A = [1,   2,   5;
     1/2, 1,   2;
     1/5, 1/2, 1];

% Principal eigenvector
[V, D] = eig(A);
[eigenvalues, index] = max(real(diag(D)));
lambdaMax = eigenvalues;

wAHP = abs(real(V(:,index)));
wAHP = wAHP / sum(wAHP);

% Consistency test
n = size(A,1);
CI = (lambdaMax - n) / (n - 1);

RIValues = [0, 0, 0.58, 0.90, 1.12, ...
            1.24, 1.32, 1.41, 1.45];
RI = RIValues(n);

if RI == 0
    CR = 0;
else
    CR = CI / RI;
end

disp('AHP weights:');
disp(wAHP);
fprintf('lambda_max = %.6f\n', lambdaMax);
fprintf('CI = %.6f, CR = %.6f\n', CI, CR);
```

Listing 2.1: AHP weights and consistency test

```
% Positively oriented and nonnegative data
X = [0.8, 0.6, 0.7;
     0.6, 0.9, 0.8;
     0.9, 0.5, 0.6];

[n, m] = size(X);

% Column proportions
```

```
P = X ./ sum(X,1);

% Use the convention 0*log(0) = 0
logP = zeros(size(P));
positiveMask = P > 0;
logP(positiveMask) = log(P(positiveMask));

k = 1 / log(n);
entropyValue = -k * sum(P .* logP, 1);
informationUtility = 1 - entropyValue;

wEntropy = informationUtility ...
    / sum(informationUtility);

disp('Entropy weights:');
disp(wEntropy);
```

Listing 2.2: Entropy-weight calculation

```
% X must be positively oriented
X = [0.8, 0.6, 0.7;
     0.6, 0.9, 0.8;
     0.9, 0.5, 0.6];

w = [1/3, 1/3, 1/3];

% Vector normalization
Z = X ./ sqrt(sum(X.^2,1));

positiveIdeal = max(Z,[],1);
negativeIdeal = min(Z,[],1);

DPositive = sqrt(sum( ...
    w .* (Z - positiveIdeal).^2, 2));
DNegative = sqrt(sum( ...
    w .* (Z - negativeIdeal).^2, 2));

closeness = DNegative ...
    ./ (DPositive + DNegative);

[sortedScore, ranking] = sort( ...
    closeness, 'descend');

disp('TOPSIS closeness scores:');
disp(closeness);

disp('TOPSIS ranking:');
disp(ranking);
```

Listing 2.3: Weighted TOPSIS

```
reference = [9, 8, 10];
```

```
2
3  comparison = [8, 7, 9;
4                6, 8, 7;
5                9, 5, 8];
6
7  rho = 0.5;
8  w = [1/3, 1/3, 1/3];
9
10 difference = abs(comparison - reference);
11
12 deltaMin = min(difference,[],'all');
13 deltaMax = max(difference,[],'all');
14
15 coefficient = (deltaMin + rho*deltaMax) ...
16     ./ (difference + rho*deltaMax);
17
18 grade = coefficient * w';
19
20 [sortedGrade, ranking] = sort( ...
21     grade, 'descend');
22
23 disp('Grey relational grades:');
24 disp(grade);
25
26 disp('Ranking:');
27 disp(ranking);
```

Listing 2.4: Grey relational analysis

```
1  % All criteria are benefit-type
2  X = [0.8, 0.6, 0.7;
3       0.6, 0.9, 0.8;
4       0.9, 0.5, 0.6];
5
6  w = [1/3, 1/3, 1/3];
7  v = 0.5;
8
9  bestValue = max(X,[],1);
10 worstValue = min(X,[],1);
11
12 regret = (bestValue - X) ...
13     ./ (bestValue - worstValue);
14
15 S = sum(w .* regret, 2);
16 R = max(w .* regret, [], 2);
17
18 SBest = min(S);
19 SWorst = max(S);
20 RBest = min(R);
21 RWorst = max(R);
22
23 Q = v * (S - SBest) / (SWorst - SBest) ...
```

```
    + (1-v) * (R - RBest) ...
    / (RWorst - RBest);

[sortedQ, ranking] = sort(Q, 'ascend');

disp('S values:');
disp(S);

disp('R values:');
disp(R);

disp('Q values:');
disp(Q);

disp('VIKOR ranking:');
disp(ranking);
```

Listing 2.5: VIKOR compromise ranking

Remark 2.11.1. The code implements the numerical steps of the models, but it does not choose the indicators, justify the weights, or interpret the resulting ranking. These decisions remain part of the mathematical modeling process.

Chapter 3

Mathematical Programming and Optimization Models

3.1 Formulation of an Optimization Model

An optimization model determines the best feasible decision according to a specified objective. Its three fundamental components are

- decision variables, representing the quantities to be determined;
- an objective function, measuring the quality of a decision;
- constraints, describing the limitations that every feasible decision must satisfy.

A general constrained optimization problem has the form

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & f(x) \\ \text{subject to} \quad & g_i(x) \le 0, \qquad i = 1, \dots, p, \\ & h_j(x) = 0, \qquad j = 1, \dots, q, \\ & \ell_k \le x_k \le u_k, \quad k = 1, \dots, n. \end{aligned}$$

The set of all points satisfying the constraints is called the feasible set. A point x^* is a global minimizer if

$$f(x^*) \le f(x)$$

for every feasible x. It is a local minimizer if the same inequality holds for every feasible x in a sufficiently small neighborhood of x^*.
An optimization problem may be

- feasible and bounded, in which case an optimal solution may exist;
- infeasible, meaning that no point satisfies all constraints;
- unbounded, meaning that the objective can improve indefinitely;
- feasible with multiple optimal solutions.

Example 3.1.1. A factory produces two products. Let x_1 and x_2 denote their daily production quantities. The profits per unit are 40 and 30, each product requires one unit of labor capacity, and their machine-time requirements are 240 and 120. If at most 6 units of labor and 1200 units of machine time are available, the model is

$$\begin{aligned} \max_{x_1, x_2} \quad & 40x_1 + 30x_2 \\ \text{subject to} \quad & x_1 + x_2 \le 6, \\ & 240x_1 + 120x_2 \le 1200, \\ & x_1 \ge 1, \qquad x_2 \ge 1. \end{aligned}$$

Here, x_1, x_2 are the decision variables, $40x_1 + 30x_2$ is the objective function, and the remaining inequalities form the feasible set.

3.2 Linear Programming

A linear programming problem, abbreviated as LP, has a linear objective function and linear constraints. A general bounded-variable formulation is

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & c^{\mathsf{T}} x \\ \text{subject to} \quad & Ax \le b, \\ & A_{\text{eq}} x = b_{\text{eq}}, \\ & \ell \le x \le u. \end{aligned}$$

Linear programming is commonly used in

- production planning;
- transportation and distribution;
- diet and blending problems;
- workforce scheduling;
- portfolio allocation;
- resource-allocation problems.

Theorem 3.2.1. If a linear programming problem has an optimal solution and its feasible set contains an extreme point, then at least one optimal solution occurs at an extreme point of the feasible set.

This theorem explains why linear programming algorithms search among boundary structures rather than all points in the feasible region.

Example 3.2.2. Consider

$$\begin{aligned} \max_{x_1,x_2} \quad & 40x_1 + 30x_2 \\ \text{subject to} \quad & x_1 + x_2 \leq 6, \\ & 2x_1 + x_2 \leq 10, \\ & x_1 \geq 1, \qquad x_2 \geq 1. \end{aligned}$$

The relevant extreme points are

$$(1,1), \qquad (1,5), \qquad \left(\frac{9}{2}, 1\right), \qquad (4,2).$$

Their objective values are

(x_1, x_2)	$40x_1 + 30x_2$
$(1,1)$	70
$(1,5)$	190
$(9/2,1)$	210
$(4,2)$	220

Therefore, the optimal production plan is

$$x_1^* = 4, \qquad x_2^* = 2,$$

and the maximum profit is 220.

In MATLAB, `linprog` solves minimization problems. Therefore, a maximization problem with objective $c^\mathsf{T}x$ must be entered using the coefficient vector $-c$.

3.3 Duality and Sensitivity Analysis

Every linear programming problem has an associated dual problem. Consider the primal problem

$$\begin{aligned} \max_{x} \quad & c^\mathsf{T}x \\ \text{subject to} \quad & Ax \leq b, \\ & x \geq 0. \end{aligned}$$

Its dual is

$$\begin{aligned} \min_{y} \quad & b^{\mathsf{T}} y \\ \text{subject to} \quad & A^{\mathsf{T}} y \geq c, \\ & y \geq 0. \end{aligned}$$

Theorem 3.3.1 (Weak Duality)**.** For every primal feasible point x and dual feasible point y,

$$c^{\mathsf{T}} x \leq b^{\mathsf{T}} y.$$

Proof. Since $A^{\mathsf{T}} y \geq c$ and $x \geq 0$,

$$c^{\mathsf{T}} x \leq y^{\mathsf{T}} A x.$$

Since $Ax \leq b$ and $y \geq 0$,

$$y^{\mathsf{T}} A x \leq y^{\mathsf{T}} b = b^{\mathsf{T}} y.$$

Combining the two inequalities gives the result. □

Theorem 3.3.2 (Strong Duality)**.** If either the primal or dual linear program has a finite optimal solution, then the other also has a finite optimal solution, and their optimal objective values are equal.

The optimal dual variable y_i^* is often interpreted as the shadow price of the i-th resource. It approximately measures the change in the optimal objective value caused by increasing b_i by one unit, provided that the perturbation remains within an allowable range.

Example 3.3.3. Consider

$$\begin{aligned} \max_{x_1, x_2} \quad & 3x_1 + 2x_2 \\ \text{subject to} \quad & x_1 + x_2 \leq 4, \\ & x_1 \leq 2, \\ & x_1, x_2 \geq 0. \end{aligned}$$

The dual problem is

$$\begin{aligned} \min_{y_1, y_2} \quad & 4y_1 + 2y_2 \\ \text{subject to} \quad & y_1 + y_2 \geq 3, \\ & y_1 \geq 2, \\ & y_1, y_2 \geq 0. \end{aligned}$$

The primal optimal solution is $x^* = (2, 2)$, with objective value 10. The dual optimal solution is $y^* = (2, 1)$, also with objective value 10.
The shadow price $y_1^* = 2$ means that one additional unit of the first resource increases the optimal objective by approximately 2, while the current optimal basis remains unchanged. Similarly, $y_2^* = 1$ is the marginal value of relaxing the second constraint.

Remark 3.3.4. A shadow price is a local sensitivity measure. It should not be extrapolated to arbitrarily large changes in the resource level.

3.4 Integer and Mixed-Integer Programming

An integer programming problem requires some or all decision variables to take integer values. A mixed-integer linear program has the form

$$\begin{aligned} \min_{x} \quad & c^{\mathsf{T}}x \\ \text{subject to} \quad & Ax \le b, \\ & A_{\text{eq}}x = b_{\text{eq}}, \\ & \ell \le x \le u, \\ & x_j \in \mathbb{Z}, \qquad j \in \mathcal{I}. \end{aligned}$$

Integer restrictions are required when a variable represents an indivisible quantity, such as the number of vehicles, workers, machines, or facilities. Rounding a continuous solution usually does not produce the optimal integer solution and may even violate the constraints. A common solution method is branch and bound. The algorithm first solves the continuous relaxation obtained by removing the integer restrictions. If the solution is not integral, the feasible region is divided into smaller subproblems by branching on a fractional variable. Bounds from the relaxed problems are then used to eliminate subproblems that cannot contain a better integer solution.

Example 3.4.1. A delivery company can purchase small and large trucks. A small truck carries 3 tons and costs 4 units, while a large truck carries 5 tons and costs 6 units. At least 17 tons of capacity are required. The model is

$$\begin{aligned} \min_{x_1,x_2} \quad & 4x_1 + 6x_2 \\ \text{subject to} \quad & 3x_1 + 5x_2 \ge 17, \\ & x_1, x_2 \in \mathbb{Z}_{\ge 0}. \end{aligned}$$

Some feasible plans are

(x_1, x_2)	Capacity	Cost
$(6, 0)$	18	24
$(4, 1)$	17	22
$(3, 2)$	19	24
$(1, 3)$	18	22
$(0, 4)$	20	24

Thus, both $(4, 1)$ and $(1, 3)$ are optimal, with minimum cost 22. This example also shows that an integer program may have multiple optimal solutions.

3.5 Binary Programming

A binary variable satisfies $x_i \in \{0, 1\}$. It represents a yes-or-no decision such as whether to select a location, activate a project, assign a worker, or use a transportation route. Binary programming is widely used in

- set-covering problems;
- facility-location problems;
- assignment problems;
- project-selection problems;

- logical constraints;
- scheduling problems.

Set-covering formulation. Suppose that candidate facility j covers the objects in the set S_j. Let

$$x_j = \begin{cases} 1, & \text{if facility } j \text{ is selected,} \\ 0, & \text{otherwise.} \end{cases}$$

If $a_{ij} = 1$ when facility j covers object i, a minimum set-covering model is

$$\begin{aligned} \min_{x} \quad & \sum_{j=1}^{m} c_j x_j \\ \text{subject to} \quad & \sum_{j=1}^{m} a_{ij} x_j \geq 1, \qquad i = 1, \ldots, n, \\ & x_j \in \{0, 1\}, \qquad j = 1, \ldots, m. \end{aligned}$$

Example 3.5.1. Three candidate service centers cover three communities:

$$S_1 = \{1, 2\}, \qquad S_2 = \{2, 3\}, \qquad S_3 = \{1, 3\}.$$

If every selected center has unit cost, the model is

$$\begin{aligned} \min \quad & x_1 + x_2 + x_3 \\ \text{subject to} \quad & x_1 + x_3 \geq 1, \\ & x_1 + x_2 \geq 1, \\ & x_2 + x_3 \geq 1, \\ & x_1, x_2, x_3 \in \{0, 1\}. \end{aligned}$$

Selecting any two centers covers all three communities. Therefore, the optimal objective value is 2, and the model has three optimal solutions:

$$(1, 1, 0), \qquad (1, 0, 1), \qquad (0, 1, 1).$$

Logical statements can also be represented using binary variables. For example,

- "at most one of A and B" becomes $x_A + x_B \leq 1$;
- "at least one of A and B" becomes $x_A + x_B \geq 1$;
- "if A, then B" becomes $x_A \leq x_B$;
- "select exactly k projects" becomes $\sum_j x_j = k$.

3.6 Multi-Objective and Goal Programming

Many practical problems have several conflicting objectives. For example, a company may wish to maximize profit while minimizing cost, pollution, and risk. A multi-objective optimization problem has the form

$$\min_{x \in \mathcal{F}} \quad \Big(f_1(x), f_2(x), \ldots, f_r(x) \Big),$$

where $\mathcal{F}$ is the feasible set.

A feasible point x^* is Pareto optimal if there is no feasible x that improves at least one objective without worsening another.

Weighted-sum method. After making the objectives dimensionless, one may solve

$$\min_{x\in\mathcal{F}} \sum_{k=1}^{r} \alpha_k f_k(x), \qquad \alpha_k \geq 0, \qquad \sum_{k=1}^{r} \alpha_k = 1.$$

Different weights generally produce different Pareto-optimal solutions.

Goal programming. Suppose that objective $f_k(x)$ has target value g_k. Introduce nonnegative deviation variables d_k^+ and d_k^-, where d_k^+ represents exceeding the target and d_k^- represents falling below it:

$$f_k(x) + d_k^- - d_k^+ = g_k.$$

The objective minimizes the undesirable deviations.

Example 3.6.1. A factory produces two products. The profit is

$$P = 5x_1 + 4x_2,$$

and pollution is

$$E = 4x_1 + x_2.$$

Suppose that $x_1 + x_2 \leq 10$, the profit target is at least 40, and the pollution target is at most 20. A goal-programming model is

$$\begin{aligned} \min \quad & d_1^- + d_2^+ \\ \text{subject to} \quad & 5x_1 + 4x_2 + d_1^- - d_1^+ = 40, \\ & 4x_1 + x_2 + d_2^- - d_2^+ = 20, \\ & x_1 + x_2 \leq 10, \\ & x_1, x_2, d_1^{\pm}, d_2^{\pm} \geq 0. \end{aligned}$$

The plan $x_1 = 0$, $x_2 = 10$ gives

$$P = 40, \qquad E = 10.$$

It satisfies both targets, so $d_1^- = d_2^+ = 0$, and the minimum goal deviation is 0.

Remark 3.6.2. A weighted sum should not combine objectives with very different numerical scales without normalization. Otherwise, the largest-scale objective may dominate regardless of its assigned weight.

3.7 Nonlinear Programming and Optimality Conditions

A nonlinear programming problem contains a nonlinear objective function or at least one nonlinear constraint. Such problems may have several local optima, and their numerical solutions may depend on the initial point.

For an equality-constrained problem

$$\begin{aligned} \min_{x} \quad & f(x) \\ \text{subject to} \quad & h_j(x) = 0, \qquad j = 1, \ldots, q, \end{aligned}$$

the Lagrangian is

$$\mathcal{L}(x, \lambda) = f(x) + \sum_{j=1}^{q} \lambda_j h_j(x).$$

Under suitable regularity conditions, a local optimizer satisfies

$$\nabla_x \mathcal{L}(x^*, \lambda^*) = 0$$

together with the original equality constraints.

Example 3.7.1. Find the rectangle with maximum area among all rectangles with perimeter 20. If the side lengths are x and y, then $x + y = 10$, and the problem is

$$\begin{aligned} \max_{x,y} \quad & xy \\ \text{subject to} \quad & x + y = 10. \end{aligned}$$

The Lagrangian is

$$\mathcal{L}(x, y, \lambda) = xy + \lambda(10 - x - y).$$

The first-order conditions are

$$y - \lambda = 0, \qquad x - \lambda = 0, \qquad 10 - x - y = 0.$$

Therefore, $x = y = \lambda = 5$, and the maximum area is 25. Thus, among rectangles with a fixed perimeter, the square has the largest area.

For inequality constraints $g_i(x) \le 0$, the appropriate extension is the Karush–Kuhn–Tucker system. For the minimization problem

$$\begin{aligned} \min_{x} \quad & f(x) \\ \text{subject to} \quad & g_i(x) \le 0, \\ & h_j(x) = 0, \end{aligned}$$

the KKT conditions are

$$\begin{gathered} \nabla f(x^*) + \sum_{i=1}^{p} \mu_i^* \nabla g_i(x^*) + \sum_{j=1}^{q} \lambda_j^* \nabla h_j(x^*) = 0, \\ g_i(x^*) \le 0, \qquad h_j(x^*) = 0, \\ \mu_i^* \ge 0, \qquad \mu_i^* g_i(x^*) = 0. \end{gathered}$$

The last condition is called complementary slackness. It states that either the i-th inequality is active or its multiplier is zero.

For a convex problem satisfying suitable regularity conditions, the KKT conditions are sufficient as well as necessary for global optimality. For a nonconvex problem, they generally identify only candidate local optima.

3.8 Dynamic Programming

Dynamic programming solves a multi-stage decision problem by decomposing it into smaller subproblems. Its central principle is Bellman's principle of optimality: the remaining decisions of an optimal policy must themselves form an optimal policy for the state produced by the earlier decisions.

Let s_t denote the state at stage t, let a_t be the decision, and let $c_t(s_t, a_t)$ be the stage cost. If the next state is $s_{t+1} = T_t(s_t, a_t)$, the value function satisfies

$$V_t(s) = \min_{a \in \mathcal{A}_t(s)} \left\{ c_t(s, a) + V_{t+1}\big(T_t(s, a)\big) \right\}.$$

Dynamic programming is appropriate when

- the problem has a natural stage structure;
- the current state summarizes all information needed for the future;
- the total objective is separable across stages;
- repeated subproblems occur.

Example 3.8.1. Consider a 0-1 knapsack problem with capacity 5. The item weights and values are

$$(w_1, w_2, w_3) = (2, 3, 4), \qquad (v_1, v_2, v_3) = (3, 4, 5).$$

Let $V(i, c)$ be the maximum value obtainable using the first i items with capacity c. The recurrence is

$$V(i,c) = \begin{cases} V(i-1,c), & w_i > c, \\ \max\{V(i-1,c),\, V(i-1,c-w_i)+v_i\}, & w_i \le c. \end{cases}$$

For capacity 5, selecting items 1 and 2 uses total weight $2 + 3 = 5$ and gives total value $3 + 4 = 7$. No other feasible selection has a larger value, so

$$V(3, 5) = 7.$$

Remark 3.8.2. Dynamic programming can transform an exponential enumeration into a manageable recursion, but its state space may still grow rapidly with the number of state variables. This phenomenon is called the curse of dimensionality.

3.9 Monte Carlo Simulation and Random Search

Monte Carlo methods approximate quantities using repeated random sampling. They are useful when an exact calculation is difficult but random samples can be generated easily. Suppose that $X_1, \ldots, X_N$ are independent samples from a distribution and the desired quantity is

$$\theta = \mathbb{E}[g(X)].$$

The Monte Carlo estimator is

$$\widehat{\theta}_N = \frac{1}{N} \sum_{i=1}^{N} g(X_i).$$

By the law of large numbers, $\widehat{\theta}_N$ converges to θ as $N \to \infty$. Under standard conditions, its standard error decreases at rate $N^{-1/2}$.

Example 3.9.1. The value of π can be estimated by sampling points uniformly from the unit square $[0, 1]^2$. The probability that a random point (X, Y) lies inside the quarter unit circle is

$$\mathbb{P}(X^2 + Y^2 \le 1) = \frac{\pi}{4}.$$

If N_{in} of N sampled points satisfy $X_i^2 + Y_i^2 \le 1$, then

$$\widehat{\pi} = 4\frac{N_{\text{in}}}{N}.$$

For example, if $N = 100000$ and $N_{\text{in}} = 78562$, then

$$\widehat{\pi} = 4\frac{78562}{100000} = 3.14248.$$

Monte Carlo sampling can also be used as a simple random-search method. If the feasible region $\mathcal{F}$ can be sampled, generate feasible points $X_1, \dots, X_N$ and select

$$\widehat{x}_N = \arg\min_{1 \le i \le N} f(X_i).$$

This approach is easy to implement but does not generally guarantee that the global optimum has been found.

Remark 3.9.2. A Monte Carlo estimate should be reported together with the sample size, random-sampling procedure, and an estimate of simulation uncertainty. Repeating the experiment with several random seeds helps determine whether the result is stable.

3.10 Model Verification and Sensitivity Analysis

An optimization solver returning a numerical vector does not by itself prove that the model is correct. The computed solution should be checked using the following steps:

1. substitute the solution into every constraint;
2. verify all integrality and binary restrictions;
3. recompute the objective value independently;
4. identify active and inactive constraints;
5. inspect the solver's exit flag and optimality information;
6. vary important coefficients and resource bounds;
7. compare the solution with simple feasible benchmark decisions;
8. interpret the result in the context of the original problem.

If the parameter vector is θ and the optimal solution is $x^*(\theta)$, sensitivity analysis studies how x^* and the optimal value

$$v(\theta) = f\Big(x^*(\theta); \theta\Big)$$

change when θ is perturbed.

Example 3.10.1. Suppose that an optimal production plan uses all available labor and changes from $(4, 2)$ to $(2, 4)$ when the profit coefficient of the second product increases from 30 to 31. The original solution is highly sensitive to that coefficient. A decision maker should not describe $(4, 2)$ as a robust recommendation unless the profit estimates are sufficiently accurate.

3.11 MATLAB Implementation

The MATLAB function `linprog` uses the convention

$$\begin{aligned} \min_{x} \quad & f^{\mathsf{T}} x \\ \text{subject to} \quad & Ax \le b, \\ & A_{\mathrm{eq}} x = b_{\mathrm{eq}}, \\ & \ell b \le x \le u b. \end{aligned}$$

```
% Maximize 40*x1 + 30*x2
% linprog minimizes, so use the negative coefficients
f = [-40; -30];

A = [1, 1;
     2, 1];
b = [6; 10];

Aeq = [];
beq = [];

lb = [1; 1];
ub = [];

options = optimoptions('linprog', ...
    'Display', 'none');

[x, fval, exitflag, output, lambda] = ...
    linprog(f, A, b, Aeq, beq, ...
    lb, ub, options);

maximumProfit = -fval;

disp('Optimal solution:');
disp(x);

fprintf('Maximum profit = %.4f\n', ...
    maximumProfit);
fprintf('Exit flag = %d\n', exitflag);
```

Listing 3.1: Linear programming with linprog

The function `intlinprog` solves mixed-integer linear programs. The vector `intcon` contains the indices of variables constrained to be integers.

```
% Minimize 4*x1 + 6*x2
f = [4; 6];

% 3*x1 + 5*x2 >= 17 is converted to
% -3*x1 - 5*x2 <= -17
A = [-3, -5];
b = -17;

Aeq = [];
beq = [];

lb = [0; 0];
ub = [];

% Both variables are integers
intcon = [1, 2];

```

```
options = optimoptions('intlinprog', ...
    'Display', 'none');

[x, fval, exitflag] = ...
    intlinprog(f, intcon, A, b, ...
    Aeq, beq, lb, ub, options);

disp('Optimal integer solution:');
disp(x);

fprintf('Minimum cost = %.4f\n', fval);
fprintf('Exit flag = %d\n', exitflag);
```

Listing 3.2: Integer programming with intlinprog

A binary program is implemented by setting the corresponding lower and upper bounds to 0 and 1.

```
% Select the minimum number of centers
f = [1; 1; 1];

% Coverage constraints:
% x1 + x3 >= 1
% x1 + x2 >= 1
% x2 + x3 >= 1
A = [-1,  0, -1;
     -1, -1,  0;
      0, -1, -1];
b = [-1; -1; -1];

intcon = 1:3;
lb = zeros(3,1);
ub = ones(3,1);

[x, fval] = intlinprog( ...
    f, intcon, A, b, ...
    [], [], lb, ub);

disp('Selected centers:');
disp(x);

fprintf('Number of centers = %.0f\n', ...
    fval);
```

Listing 3.3: Binary set-covering model

The function `fmincon` solves constrained nonlinear minimization problems:

$$\begin{aligned} \min_{x} \quad & f(x) \\ \text{subject to} \quad & Ax \le b, \\ & A_{\text{eq}}x = b_{\text{eq}}, \\ & c(x) \le 0, \\ & c_{\text{eq}}(x) = 0, \\ & \ell b \le x \le ub. \end{aligned}$$

```
% Maximize x1*x2 subject to x1+x2=10
% fmincon minimizes, so minimize -x1*x2
objective = @(x) -x(1)*x(2);

x0 = [4; 6];

A = [];
b = [];

Aeq = [1, 1];
beq = 10;

lb = [0; 0];
ub = [];

options = optimoptions('fmincon', ...
    'Algorithm', 'sqp', ...
    'Display', 'none');

[x, fval, exitflag] = ...
    fmincon(objective, x0, ...
    A, b, Aeq, beq, ...
    lb, ub, [], options);

maximumArea = -fval;

disp('Optimal side lengths:');
disp(x);

fprintf('Maximum area = %.4f\n', ...
    maximumArea);
fprintf('Exit flag = %d\n', exitflag);
```

Listing 3.4: Nonlinear programming with fmincon

The following code implements the dynamic-programming recurrence for the 0-1 knapsack example.

```
weights = [2, 3, 4];
values = [3, 4, 5];
capacity = 5;

```

```
n = numel(weights);
V = zeros(n+1, capacity+1);

for i = 1:n
    for c = 0:capacity
        if weights(i) > c
            V(i+1,c+1) = V(i,c+1);
        else
            valueWithout = V(i,c+1);
            valueWith = V(i, ...
                c-weights(i)+1) + values(i);

            V(i+1,c+1) = max( ...
                valueWithout, valueWith);
        end
    end
end

fprintf('Maximum knapsack value = %.0f\n', ...
    V(n+1,capacity+1));
```

Listing 3.5: Dynamic programming for a knapsack problem

The following Monte Carlo program estimates π.

```
rng(2026);

N = 100000;

x = rand(N,1);
y = rand(N,1);

inside = (x.^2 + y.^2 <= 1);
piEstimate = 4*mean(inside);

% Estimated Monte Carlo standard error
pEstimate = mean(inside);
standardError = 4*sqrt( ...
    pEstimate*(1-pEstimate)/N);

fprintf('Estimated pi = %.6f\n', ...
    piEstimate);
fprintf('Estimated standard error = %.6f\n', ...
    standardError);
```

Listing 3.6: Monte Carlo estimation of pi

Remark 3.11.1. MATLAB uses different solvers because linear, integer, and nonlinear optimization problems have fundamentally different mathematical structures. A model should be classified before a solver is selected.

Chapter 4

Graph Theory and Network Optimization Models

4.1 Graphs and Their Mathematical Representation

A graph describes a system of objects and the connections between them. A graph is written as

$$G = (V, E),$$

where $V = \{v_1, \ldots, v_n\}$ is the vertex set and E is the edge set. Vertices may represent cities, stations, users, tasks, or facilities, while edges may represent roads, communication links, relationships, or feasible transitions.

An undirected edge connecting v_i and v_j is written as $\{v_i, v_j\}$, while a directed edge from v_i to v_j is written as (v_i, v_j). A weighted graph associates each edge with a numerical weight w_{ij}, which may represent distance, time, cost, risk, or capacity.

Adjacency matrix. For an unweighted graph, the adjacency matrix is

$$A = (a_{ij})_{n\times n}, \qquad a_{ij} = \begin{cases} 1, & (v_i, v_j) \in E, \\ 0, & (v_i, v_j) \notin E. \end{cases}$$

For an undirected graph, A is symmetric.

Weighted adjacency matrix. For a weighted graph, one may define

$$W = (w_{ij})_{n\times n},$$

where w_{ij} is the weight of edge (v_i, v_j). If there is no edge between v_i and v_j, it is convenient to set $w_{ij} = +\infty$, while $w_{ii} = 0$.

Incidence matrix. For a directed graph with n vertices and m edges, the incidence matrix $B = (b_{ik})_{n\times m}$ is defined by

$$b_{ik} = \begin{cases} 1, & \text{edge } k \text{ leaves vertex } i, \\ -1, & \text{edge } k \text{ enters vertex } i, \\ 0, & \text{otherwise.} \end{cases}$$

This representation is particularly useful in network-flow models.

Example 4.1.1. Consider an undirected weighted graph with edges

$$AB = 2, \qquad AC = 5, \qquad BC = 1, \qquad BD = 4, \qquad CD = 2.$$

Using the vertex order A, B, C, D, its weighted adjacency matrix is

$$W = \begin{pmatrix} 0 & 2 & 5 & \infty \\ 2 & 0 & 1 & 4 \\ 5 & 1 & 0 & 2 \\ \infty & 4 & 2 & 0 \end{pmatrix}.$$

The value $w_{AD} = \infty$ indicates that there is no direct edge between A and D.

A path from v_{i_0} to v_{i_k} is a sequence

$$v_{i_0}, v_{i_1}, \ldots, v_{i_k}$$

such that consecutive vertices are connected by edges. A cycle is a path whose initial and terminal vertices are the same. A graph is connected if every pair of vertices is joined by a path.

4.2 Shortest-Path Models

The shortest-path problem seeks a path of minimum total weight from a source vertex s to a target vertex t. If a path P consists of edges $e_1, \dots, e_k$, its total length is

$$L(P) = \sum_{r=1}^{k} w(e_r).$$

The shortest-path problem can be formulated as a binary linear program. For each directed edge $(i, j) \in E$, define

$$x_{ij} = \begin{cases} 1, & \text{if edge } (i,j) \text{ is selected,} \\ 0, & \text{otherwise.} \end{cases}$$

The model is

$$\begin{aligned} \min_{x} \quad & \sum_{(i,j)\in E} w_{ij} x_{ij} \\ \text{subject to} \quad & \sum_{j:(i,j)\in E} x_{ij} - \sum_{j:(j,i)\in E} x_{ji} = \begin{cases} 1, & i = s, \\ -1, & i = t, \\ 0, & i \neq s, t, \end{cases} \\ & x_{ij} \in \{0, 1\}. \end{aligned}$$

The flow-conservation constraints require one unit of flow to leave the source, enter the target, and be conserved at every intermediate vertex.

Example 4.2.1. Consider the graph

$$AB = 2, \qquad AC = 5, \qquad BC = 1, \qquad BD = 4, \qquad CD = 2.$$

The candidate paths from A to D include

$$A \to B \to D, \qquad A \to C \to D, \qquad A \to B \to C \to D.$$

Their lengths are

$$2 + 4 = 6, \qquad 5 + 2 = 7, \qquad 2 + 1 + 2 = 5.$$

Therefore, the shortest path is

$$A \to B \to C \to D,$$

with total length 5.

4.3 Dijkstra, Bellman–Ford, and Floyd–Warshall Algorithms

Different shortest-path algorithms are appropriate under different assumptions on the edge weights and the number of source vertices.

Dijkstra's algorithm. Dijkstra's algorithm computes the shortest distances from one source vertex when all edge weights are nonnegative. It maintains a tentative distance $d(v)$ for every vertex.

Starting with $d(s) = 0$ and $d(v) = +\infty$ for $v \neq s$, the algorithm repeatedly selects the unvisited vertex with the smallest tentative distance. For every outgoing edge (u, v), it performs the relaxation

$$d(v) \leftarrow \min\{d(v), d(u) + w_{uv}\}.$$

Example 4.3.1. For the graph in the previous section, start at A. Initially,

$$d(A) = 0, \qquad d(B) = d(C) = d(D) = \infty.$$

Relaxing the edges from A gives

$$d(B) = 2, \qquad d(C) = 5.$$

The next selected vertex is B. Relaxing through B gives

$$d(C) = \min\{5, 2+1\} = 3, \qquad d(D) = 2 + 4 = 6.$$

The next selected vertex is C. Relaxing through C gives

$$d(D) = \min\{6, 3+2\} = 5.$$

Hence the final distance from A to D is 5, and the predecessor information reconstructs the path $A \to B \to C \to D$.

Dijkstra's algorithm is not valid in general when negative-weight edges are present. A vertex declared final may later receive a shorter path through a negative edge.

Bellman–Ford algorithm. Bellman–Ford permits negative edge weights and can detect a negative cycle reachable from the source. It relaxes every edge repeatedly. After at most $n-1$ rounds, all shortest distances are determined if no reachable negative cycle exists.

Example 4.3.2. Consider the directed edges

$$s \to a : 4, \qquad s \to b : 5, \qquad a \to b : -2,$$

$$a \to t : 6, \qquad b \to t : 3.$$

The direct route $s \to b \to t$ has length 8, while

$$s \to a \to b \to t$$

has length

$$4 - 2 + 3 = 5.$$

Bellman–Ford correctly finds distance 5, even though the graph contains the negative edge $a \to b$.

If one more relaxation after $n-1$ rounds decreases a distance, then a negative cycle is reachable from the source. In that case, a finite shortest path may not exist.

Floyd–Warshall algorithm. Floyd–Warshall computes the shortest distances between every pair of vertices. Let $d_{ij}^{(k)}$ be the shortest distance from i to j using only vertices $1, \ldots, k$ as intermediate vertices. The recurrence is

$$d_{ij}^{(k)} = \min\left\{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\right\}.$$

Example 4.3.3. Suppose that the initial directed distance matrix is

$$D^{(0)} = \begin{pmatrix} 0 & 3 & 10 \\ \infty & 0 & 4 \\ 2 & \infty & 0 \end{pmatrix}.$$

The route $1 \to 2 \to 3$ has length $3 + 4 = 7$, so it improves the direct distance from vertex 1 to vertex 3. Similarly,

$$2 \to 3 \to 1$$

has length $4 + 2 = 6$, and

$$3 \to 1 \to 2$$

has length $2 + 3 = 5$. The final all-pairs distance matrix is

$$D = \begin{pmatrix} 0 & 3 & 7 \\ 6 & 0 & 4 \\ 2 & 5 & 0 \end{pmatrix}.$$

The standard algorithm choices are

Algorithm	**Appropriate setting**
Breadth-first search	Unweighted graph or equal edge weights.
Dijkstra	One source and nonnegative edge weights.
Bellman–Ford	One source with possible negative edge weights.
Floyd–Warshall	Shortest distances between all pairs of vertices.

4.4 State-Expanded and Multimodal Networks

In some shortest-path problems, the travel cost depends not only on the current location but also on the current transportation mode, time period, or previous decision. A simple graph whose vertices represent only physical locations may then be insufficient.

The state-expansion method replaces each physical location v by several state vertices such as

$$(v, \text{rail}), \qquad (v, \text{road}), \qquad (v, \text{air}).$$

Travel edges connect states using the same mode, while transfer edges connect different modes at the same location. Transfer time and transfer cost are assigned to the latter edges.

Example 4.4.1. Suppose that a traveler moves from city A to city C. The available connections are

Connection	Time
$A \to B$ by rail	2
$B \to C$ by rail	3
$A \to B$ by air	1
$B \to C$ by rail	3
Air-to-rail transfer at B	2

If the transportation mode is ignored, one might incorrectly combine the one-hour air trip with the three-hour rail trip and report total time 4. The state-expanded network includes the two-hour transfer, giving

$$1 + 2 + 3 = 6.$$

The direct rail itinerary has total time $2 + 3 = 5$, so it is actually faster.

This construction allows a standard shortest-path algorithm to solve problems with mode-dependent costs while preserving the relevant state information.

4.5 Minimum Spanning Trees

A spanning tree of a connected undirected graph is a connected subgraph that contains every vertex and has no cycles. Every spanning tree on n vertices contains exactly $n-1$ edges.
A minimum spanning tree, abbreviated as MST, minimizes the total edge weight:

$$\min_{T} \sum_{e \in T} w(e),$$

where T ranges over all spanning trees of the graph.
Minimum spanning trees are used in

- road and communication-network design;
- pipeline and electrical-grid construction;
- hierarchical clustering;
- minimum-cost connection problems.

Kruskal's algorithm. Sort the edges from smallest to largest. Add an edge whenever it does not form a cycle, and stop after $n-1$ edges have been selected.
Prim's algorithm. Start from one vertex and repeatedly add the minimum-weight edge connecting the current tree to a vertex outside the tree.

Example 4.5.1. Consider the graph with edges

$$AB = 1, \qquad AC = 4, \qquad BC = 2, \qquad BD = 5, \qquad CD = 3.$$

Kruskal's algorithm considers the edges in the order

$$AB,\ BC,\ CD,\ AC,\ BD.$$

It first selects AB, then BC, and then CD. These three edges connect all four vertices without forming a cycle. Thus, the minimum spanning tree is

$$T = \{AB, BC, CD\},$$

with total weight

$$1 + 2 + 3 = 6.$$

Remark 4.5.2. A minimum spanning tree minimizes the cost of connecting all vertices, but it does not minimize the path distance between every pair of vertices. A shortest-path tree and a minimum spanning tree solve different problems.

4.6 Maximum Flow and Minimum Cut

A flow network is a directed graph

$$G = (V, E)$$

with a source s, a sink t, and a nonnegative capacity c_{ij} on each edge $(i, j) \in E$. A flow x_{ij} must satisfy the capacity constraints

$$0 \le x_{ij} \le c_{ij}.$$

At every intermediate vertex, flow is conserved:

$$\sum_{j:(j,i)\in E} x_{ji} = \sum_{j:(i,j)\in E} x_{ij}, \qquad i \neq s, t.$$

The value of the flow is

$$v(x) = \sum_{j:(s,j)\in E} x_{sj} - \sum_{j:(j,s)\in E} x_{js}.$$

The maximum-flow problem is

$$\max_x v(x)$$

subject to the capacity and conservation constraints.

Residual network. If the current flow on edge (i, j) is x_{ij}, the forward residual capacity is

$$c_{ij} - x_{ij},$$

while the reverse residual capacity is x_{ij}. A path from s to t in the residual network is called an augmenting path.

The Ford–Fulkerson method repeatedly finds an augmenting path and increases the flow by the minimum residual capacity on that path.

Example 4.6.1. Consider the network

$$s \to a : 3, \qquad s \to b : 2, \qquad a \to b : 1,$$

$$a \to t : 2, \qquad b \to t : 3.$$

Send 2 units along $s \to a \to t$, 2 units along $s \to b \to t$, and 1 unit along $s \to a \to b \to t$. The resulting flow has value

$$2 + 2 + 1 = 5.$$

The total capacity leaving the source is

$$c_{sa} + c_{sb} = 3 + 2 = 5,$$

so no flow can exceed 5. Therefore, the maximum-flow value is 5.

An s-t cut partitions the vertices into S and $V \setminus S$, where $s \in S$ and $t \notin S$. Its capacity is

$$c(S, V \setminus S) = \sum_{\substack{i\in S \\ j\notin S}} c_{ij}.$$

Theorem 4.6.2 (Max-Flow Min-Cut Theorem)**.** The maximum value of an s-t flow equals the minimum capacity of an s-t cut.

In the preceding example, the cut $S = \{s\}$ has capacity $3 + 2 = 5$. Since a flow of value 5 exists, this cut is a minimum cut.

4.7 Minimum-Cost Flow and Transportation Networks

A minimum-cost flow model sends a required amount of flow through a network while minimizing total transportation cost. Let c_{ij} be the unit cost and u_{ij} the capacity of edge (i,j). The model is

$$\begin{aligned} \min_{x} \quad & \sum_{(i,j)\in E} c_{ij}x_{ij} \\ \text{subject to} \quad & \sum_{j:(i,j)\in E} x_{ij} - \sum_{j:(j,i)\in E} x_{ji} = b_i, \qquad i \in V, \\ & 0 \le x_{ij} \le u_{ij}. \end{aligned}$$

Here, $b_i > 0$ represents supply, $b_i < 0$ represents demand, and $b_i = 0$ represents a transshipment vertex. Feasibility requires

$$\sum_{i\in V} b_i = 0.$$

A transportation problem is a special minimum-cost flow problem. If source i has supply s_i, destination j has demand d_j, and the unit transportation cost is c_{ij}, then

$$\begin{aligned} \min_{x} \quad & \sum_{i=1}^{p}\sum_{j=1}^{q} c_{ij}x_{ij} \\ \text{subject to} \quad & \sum_{j=1}^{q} x_{ij} = s_i, \qquad i = 1,\dots,p, \\ & \sum_{i=1}^{p} x_{ij} = d_j, \qquad j = 1,\dots,q, \\ & x_{ij} \ge 0. \end{aligned}$$

Example 4.7.1. Two warehouses have supplies 3 and 2, while two stores have demands 2 and 3. The unit-cost matrix is

$$C = \begin{pmatrix} 1 & 4 \\ 3 & 2 \end{pmatrix}.$$

The transportation model is

$$\begin{aligned} \min \quad & x_{11} + 4x_{12} + 3x_{21} + 2x_{22} \\ \text{subject to} \quad & x_{11} + x_{12} = 3, \\ & x_{21} + x_{22} = 2, \\ & x_{11} + x_{21} = 2, \\ & x_{12} + x_{22} = 3, \\ & x_{ij} \ge 0. \end{aligned}$$

The optimal plan is

$$x_{11} = 2, \qquad x_{12} = 1, \qquad x_{21} = 0, \qquad x_{22} = 2.$$

Its total cost is

$$2(1) + 1(4) + 0(3) + 2(2) = 10.$$

4.8 The Traveling-Salesman Problem

The traveling-salesman problem, abbreviated as TSP, seeks a minimum-length Hamiltonian cycle that visits every city exactly once and returns to the starting city.
Let d_{ij} be the distance from city i to city j, and define

$$x_{ij} = \begin{cases} 1, & \text{if the tour travels directly from } i \text{ to } j, \\ 0, & \text{otherwise.} \end{cases}$$

The basic assignment constraints are

$$\sum_{\substack{j=1 \\ j \neq i}}^{n} x_{ij} = 1, \qquad i = 1, \ldots, n,$$

and

$$\sum_{\substack{i=1 \\ i \neq j}}^{n} x_{ij} = 1, \qquad j = 1, \ldots, n.$$

These constraints require exactly one edge to leave and enter each city, but they do not prevent several disconnected cycles.
The complete integer-programming formulation is

$$\begin{aligned} \min_{x} \quad & \sum_{i=1}^{n} \sum_{\substack{j=1 \\ j \neq i}}^{n} d_{ij} x_{ij} \\ \text{subject to} \quad & \sum_{\substack{j=1 \\ j \neq i}}^{n} x_{ij} = 1, \qquad i = 1, \ldots, n, \\ & \sum_{\substack{i=1 \\ i \neq j}}^{n} x_{ij} = 1, \qquad j = 1, \ldots, n, \\ & \sum_{\substack{i,j \in S \\ i \neq j}} x_{ij} \leq |S| - 1, \qquad \varnothing \neq S \subsetneq V, \\ & x_{ij} \in \{0, 1\}. \end{aligned}$$

The third family of constraints eliminates subtours.

Example 4.8.1. Consider four cities with distance matrix

$$D = \begin{pmatrix} 0 & 2 & 3 & 2 \\ 2 & 0 & 2 & 3 \\ 3 & 2 & 0 & 2 \\ 2 & 3 & 2 & 0 \end{pmatrix}.$$

The tour

$$1 \to 2 \to 3 \to 4 \to 1$$

has total length

$$2 + 2 + 2 + 2 = 8.$$

In comparison, the tour

$$1 \to 2 \to 4 \to 3 \to 1$$

has total length

$$2 + 3 + 2 + 3 = 10.$$

Thus, the first tour is shorter. In fact, its length 8 is optimal.

For large instances, exact integer programming may require substantial computation. Common heuristic methods include

- the nearest-neighbor algorithm;
- the insertion algorithm;
- the 2-opt and 3-opt improvement methods;
- simulated annealing;
- genetic algorithms;
- ant-colony optimization.

2-opt improvement. Suppose that a tour contains the edges $(i, i+1)$ and $(j, j+1)$. If

$$d_{ij} + d_{i+1,j+1} < d_{i,i+1} + d_{j,j+1},$$

replace the old edges by (i, j) and $(i+1, j+1)$, reversing the segment between them. Repeating this operation until no improvement is possible produces a locally optimal tour.

Remark 4.8.2. A heuristic tour may be very good without being provably optimal. The result should be described as an approximate solution unless an optimality bound or exact solver verifies it.

4.9 Selecting and Validating a Network Model

The principal models in this chapter have different objectives:

Model	Question answered
Shortest path	What is the least-cost route between specified vertices?
Minimum spanning tree	How can all vertices be connected at minimum construction cost?
Maximum flow	What is the greatest quantity that can pass through a capacitated network?
Minimum-cost flow	How can a required flow be transported at minimum cost?
Traveling salesman	What is the shortest closed tour visiting every vertex exactly once?

A network model should be checked by verifying

- whether the graph should be directed or undirected;
- whether edge weights are distances, costs, times, or capacities;
- whether negative weights or negative cycles are possible;
- whether switching modes creates additional costs or states;

- whether capacities and conservation equations are satisfied;
- whether the computed route is connected and contains no forbidden subtours;
- whether small changes in edge weights alter the recommended network.

Example 4.9.1. Suppose that two routes have costs 100.0 and 100.2, but the edge costs are estimated with an error of approximately 1. Reporting the first route as decisively better is not justified. Both routes should be retained as competitive alternatives, or the model should be tested under perturbed edge weights.

4.10 MATLAB Implementation

MATLAB represents an undirected graph using `graph` and a directed graph using `digraph`.

```
% Undirected weighted graph
source = [1, 1, 2, 2, 3];
target = [2, 3, 3, 4, 4];
weight = [2, 5, 1, 4, 2];

G = graph(source, target, weight);

% Shortest path from vertex 1 to vertex 4
[path, distance] = shortestpath(G, 1, 4);

disp('Shortest path:');
disp(path);

fprintf('Shortest distance = %.2f\n', ...
    distance);

% All-pairs shortest-distance matrix
D = distances(G);

figure;
plot(G, 'EdgeLabel', G.Edges.Weight);
title('Weighted Graph');
```

Listing 4.1: Graph construction and shortest paths

For a directed graph with negative edges but no negative cycles, MATLAB may select a suitable algorithm using the `'auto'` option.

```
source = [1, 1, 2, 2, 3];
target = [2, 3, 3, 4, 4];
weight = [4, 5, -2, 6, 3];

G = digraph(source, target, weight);

[path, distance] = shortestpath( ...
    G, 1, 4, 'Method', 'mixed');

disp(path);
disp(distance);
```

Listing 4.2: Shortest paths in a directed graph

The function `minspantree` computes a minimum spanning tree.

```
source = [1, 1, 2, 2, 3];
target = [2, 3, 3, 4, 4];
weight = [1, 4, 2, 5, 3];

G = graph(source, target, weight);

T = minspantree(G);
totalWeight = sum(T.Edges.Weight);

fprintf('MST total weight = %.2f\n', ...
    totalWeight);

figure;
plot(T, 'EdgeLabel', T.Edges.Weight);
title('Minimum Spanning Tree');
```

Listing 4.3: Minimum spanning tree

The function `maxflow` calculates both a maximum flow and a corresponding minimum cut.

```
source = [1, 1, 2, 2, 3];
target = [2, 3, 3, 4, 4];
capacity = [3, 2, 1, 2, 3];

G = digraph(source, target, capacity);

[flowValue, flowGraph, ...
    sourceSide, sinkSide] = maxflow(G, 1, 4);

fprintf('Maximum flow = %.2f\n', ...
    flowValue);

disp('Source side of minimum cut:');
disp(sourceSide);

disp('Sink side of minimum cut:');
disp(sinkSide);

figure;
plot(flowGraph, ...
    'EdgeLabel', flowGraph.Edges.Weight);
title('Maximum Flow');
```

Listing 4.4: Maximum flow and minimum cut

A transportation problem can be converted into a linear program.

```
% Variables are ordered as
% x11, x12, x21, x22
```

```
f = [1; 4; 3; 2];

% Supply constraints
AeqSupply = [1, 1, 0, 0;
             0, 0, 1, 1];
beqSupply = [3; 2];

% Demand constraints
AeqDemand = [1, 0, 1, 0;
             0, 1, 0, 1];
beqDemand = [2; 3];

Aeq = [AeqSupply;
       AeqDemand];
beq = [beqSupply;
       beqDemand];

lb = zeros(4,1);

[x, minimumCost] = linprog( ...
    f, [], [], Aeq, beq, lb, []);

flowMatrix = [x(1), x(2);
              x(3), x(4)];

disp('Optimal transportation plan:');
disp(flowMatrix);

fprintf('Minimum cost = %.2f\n', ...
    minimumCost);
```

Listing 4.5: Minimum-cost transportation problem

The following code uses the nearest-neighbor method followed by 2-opt improvement to obtain an approximate TSP tour.

```
D = [0, 2, 3, 2;
     2, 0, 2, 3;
     3, 2, 0, 2;
     2, 3, 2, 0];

n = size(D,1);

% Nearest-neighbor initialization
tour = zeros(1,n);
tour(1) = 1;
unvisited = 2:n;

for k = 2:n
    current = tour(k-1);

    [~, index] = min( ...
        D(current,unvisited));
```

```
18
19      tour(k) = unvisited(index);
20      unvisited(index) = [];
21  end
22
23  % Two-opt improvement
24  improved = true;
25
26  while improved
27      improved = false;
28
29      for i = 1:n-2
30          for j = i+2:n
31              nextI = i+1;
32              nextJ = mod(j,n)+1;
33
34              oldCost = ...
35                  D(tour(i),tour(nextI)) ...
36                  + D(tour(j),tour(nextJ));
37
38              newCost = ...
39                  D(tour(i),tour(j)) ...
40                  + D(tour(nextI),tour(nextJ));
41
42              if newCost < oldCost
43                  tour(nextI:j) = ...
44                      fliplr(tour(nextI:j));
45                  improved = true;
46              end
47          end
48      end
49  end
50
51  closedTour = [tour, tour(1)];
52  tourLength = 0;
53
54  for k = 1:n
55      tourLength = tourLength ...
56          + D(closedTour(k), ...
57              closedTour(k+1));
58  end
59
60  disp('Approximate TSP tour:');
61  disp(closedTour);
62
63  fprintf('Tour length = %.2f\n', ...
64      tourLength);
```

Listing 4.6: Nearest-neighbor and two-opt TSP heuristic

Remark 4.10.1. The shortest-path, spanning-tree, maximum-flow, and TSP functions solve different optimization problems even when they are applied to the same graph. The

intended network structure and objective should be identified before a MATLAB function is selected.

Chapter 5

Interpolation, Curve Fitting, and Regression Models

5.1 Interpolation and Approximation Problems

Interpolation and curve fitting both construct a function from observed data, but they solve different problems.

Suppose that the available data are

$$(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n),$$

where the nodes $x_0, \ldots, x_n$ are distinct.

An interpolating function p satisfies

$$p(x_i) = y_i, \qquad i = 0, \ldots, n.$$

It passes through every data point exactly and is appropriate when the data are regarded as exact.

A fitted function $f(x; \theta)$ generally does not pass through every point. Instead, its parameters are chosen to minimize an overall discrepancy such as

$$\sum_{i=1}^{n} \Big(f(x_i; \theta) - y_i \Big)^2.$$

Fitting is more appropriate when the observations contain measurement error, sampling variation, or random noise.

	Interpolation	**Curve fitting**
Requirement	Passes through every data point	Approximates the data as a whole
Data assumption	Observations are exact	Observations may contain noise
Typical methods	Lagrange, Newton, spline	Least squares, regression
Main risk	Oscillation and extrapolation	Underfitting or overfitting

Example 5.1.1. Suppose that the observations are

$$(0, 1), \qquad (1, 3), \qquad (2, 2).$$

A quadratic interpolation polynomial must pass through all three points. In contrast, a fitted straight line will generally miss some of the points, but it may provide a simpler description of their overall trend.

5.2 Polynomial Interpolation

Given $n + 1$ distinct nodes $x_0, \ldots, x_n$, polynomial interpolation seeks a polynomial of degree at most n,

$$p_n(x) = a_0 + a_1 x + \cdots + a_n x^n,$$

such that $p_n(x_i) = y_i$ for $i = 0, \ldots, n$.

The coefficients satisfy the Vandermonde system

$$\begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^n \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{pmatrix}.$$

The determinant of the Vandermonde matrix is

$$\prod_{0 \le i < j \le n} (x_j - x_i),$$

which is nonzero because the interpolation nodes are distinct.

Theorem 5.2.1. For $n+1$ distinct nodes $x_0, \dots, x_n$, there exists a unique polynomial p_n of degree at most n satisfying

$$p_n(x_i) = y_i, \qquad i = 0, \dots, n.$$

Proof. The Vandermonde matrix is invertible because its determinant is nonzero, so the interpolation equations have a unique solution.
Equivalently, suppose that p_n and q_n are two such interpolating polynomials. Then $r = p_n - q_n$ has degree at most n and has the $n+1$ distinct roots $x_0, \dots, x_n$. Therefore, $r \equiv 0$, so $p_n = q_n$. □

Example 5.2.2. For the points

$$(0, 1), \qquad (1, 3), \qquad (2, 2),$$

write

$$p_2(x) = a_0 + a_1 x + a_2 x^2.$$

The interpolation equations are

$$a_0 = 1, \qquad a_0 + a_1 + a_2 = 3,$$

$$a_0 + 2a_1 + 4a_2 = 2.$$

Solving gives

$$a_0 = 1, \qquad a_1 = \frac{7}{2}, \qquad a_2 = -\frac{3}{2}.$$

Hence,

$$p_2(x) = 1 + \frac{7}{2}x - \frac{3}{2}x^2.$$

Indeed, $p_2(0) = 1$, $p_2(1) = 3$, and $p_2(2) = 2$.

If f is sufficiently differentiable, the interpolation error is

$$f(x) - p_n(x) = \frac{f^{(n+1)}(\xi_x)}{(n+1)!} \prod_{i=0}^{n} (x - x_i)$$

for some ξ_x lying in the smallest interval containing $x, x_0, \dots, x_n$.

5.3 Lagrange and Newton Interpolation

The Vandermonde formulation proves existence and uniqueness but is not the most convenient way to construct the polynomial.
Lagrange interpolation. Define the Lagrange basis polynomials

$$\ell_i(x) = \prod_{\substack{0 \le j \le n \\ j \ne i}} \frac{x - x_j}{x_i - x_j}.$$

They satisfy

$$\ell_i(x_j) = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$$

Therefore, the interpolation polynomial is

$$p_n(x) = \sum_{i=0}^{n} y_i \ell_i(x).$$

Example 5.3.1. For the points

$$(0, 1), \qquad (1, 3), \qquad (2, 2),$$

the Lagrange basis polynomials are

$$\ell_0(x) = \frac{(x-1)(x-2)}{2},$$

$$\ell_1(x) = -x(x-2), \qquad \ell_2(x) = \frac{x(x-1)}{2}.$$

Thus,

$$\begin{aligned} p_2(x) &= \ell_0(x) + 3\ell_1(x) + 2\ell_2(x) \\ &= 1 + \frac{7}{2}x - \frac{3}{2}x^2. \end{aligned}$$

Newton interpolation. Newton interpolation uses divided differences. The first divided difference is

$$f[x_i, x_j] = \frac{f(x_j) - f(x_i)}{x_j - x_i},$$

and higher-order divided differences are defined recursively by

$$f[x_i, \ldots, x_{i+k}] = \frac{f[x_{i+1}, \ldots, x_{i+k}] - f[x_i, \ldots, x_{i+k-1}]}{x_{i+k} - x_i}.$$

The Newton interpolation polynomial is

$$\begin{aligned} p_n(x) = f[x_0] &+ f[x_0, x_1](x - x_0) \\ &+ f[x_0, x_1, x_2](x - x_0)(x - x_1) + \cdots \\ &+ f[x_0, \ldots, x_n] \prod_{j=0}^{n-1} (x - x_j). \end{aligned}$$

Example 5.3.2. For the same three observations, the divided differences are

$$f[x_0] = 1, \qquad f[x_0, x_1] = \frac{3-1}{1-0} = 2,$$

$$f[x_1, x_2] = \frac{2-3}{2-1} = -1,$$

and

$$f[x_0, x_1, x_2] = \frac{-1-2}{2-0} = -\frac{3}{2}.$$

Therefore,

$$\begin{aligned} p_2(x) &= 1 + 2x - \frac{3}{2}x(x-1) \\ &= 1 + \frac{7}{2}x - \frac{3}{2}x^2. \end{aligned}$$

The Lagrange and Newton forms represent the same unique polynomial.

The Newton form is convenient when a new interpolation node is added because the existing terms remain unchanged and only one new term is required.

5.4 Hermite and Piecewise Interpolation

Polynomial interpolation uses function values. Hermite interpolation also uses derivative information.
Suppose that both $f(x_i)$ and $f'(x_i)$ are prescribed at $n+1$ distinct nodes. There exists a unique polynomial of degree at most $2n+1$ satisfying

$$H(x_i) = f(x_i), \qquad H'(x_i) = f'(x_i), \qquad i = 0, \ldots, n.$$

The Hermite interpolation error is

$$f(x) - H(x) = \frac{f^{(2n+2)}(\xi_x)}{(2n+2)!} \prod_{i=0}^{n} (x - x_i)^2.$$

Example 5.4.1. Find a cubic polynomial satisfying

$$H(0) = 1, \qquad H'(0) = 0,$$

$$H(1) = 2, \qquad H'(1) = 3.$$

Write

$$H(x) = a_0 + a_1 x + a_2 x^2 + a_3 x^3.$$

The first two conditions give $a_0 = 1$ and $a_1 = 0$. The remaining conditions give

$$a_2 + a_3 = 1, \qquad 2a_2 + 3a_3 = 3.$$

Thus, $a_2 = 0$ and $a_3 = 1$, so

$$H(x) = 1 + x^3.$$

Piecewise linear interpolation. For $x \in [x_i, x_{i+1}]$, the piecewise linear interpolant is

$$L(x) = y_i + \frac{y_{i+1} - y_i}{x_{i+1} - x_i}(x - x_i).$$

It is continuous but generally not differentiable at the interpolation nodes.

Example 5.4.2. For the data

$$(0, 0), \qquad (1, 1), \qquad (2, 0),$$

the piecewise linear interpolant is

$$L(x) = \begin{cases} x, & 0 \le x \le 1, \\ 2 - x, & 1 \le x \le 2. \end{cases}$$

It passes through all three points but has a corner at $x = 1$.

5.5 Cubic Spline Interpolation

A cubic spline avoids many of the oscillations associated with a single high-degree polynomial. Given nodes

$$a = x_0 < x_1 < \cdots < x_n = b,$$

a cubic spline S satisfies

- S is a cubic polynomial on every interval $[x_i, x_{i+1}]$;
- $S(x_i) = y_i$ for every interpolation node;
- S, S', and S'' are continuous on $[a, b]$.

Additional boundary conditions are needed to determine a unique spline. A natural cubic spline satisfies

$$S''(x_0) = S''(x_n) = 0.$$

Let $h_i = x_{i+1} - x_i$ and $M_i = S''(x_i)$. The interior second derivatives satisfy

$$h_{i-1}M_{i-1} + 2(h_{i-1} + h_i)M_i + h_iM_{i+1} = 6\left(\frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}}\right).$$

Example 5.5.1. Construct the natural cubic spline through

$$(0, 0), \qquad (1, 1), \qquad (2, 0).$$

The natural boundary conditions give $M_0 = M_2 = 0$. Since $h_0 = h_1 = 1$, the interior equation is

$$4M_1 = 6[(0 - 1) - (1 - 0)] = -12.$$

Thus, $M_1 = -3$. The resulting spline is

$$S(x) = \begin{cases} -\frac{1}{2}x^3 + \frac{3}{2}x, & 0 \le x \le 1, \\ 1 - \frac{3}{2}(x-1)^2 + \frac{1}{2}(x-1)^3, & 1 \le x \le 2. \end{cases}$$

The function and its first two derivatives agree at $x = 1$, and $S''(0) = S''(2) = 0$.

Runge phenomenon. Increasing the degree of an interpolation polynomial does not always improve the approximation. When equally spaced nodes are used over a large interval, high-degree interpolation may exhibit severe oscillations near the endpoints. This is known as the Runge phenomenon.
Possible remedies include

- using piecewise polynomial interpolation;
- using cubic splines;
- using Chebyshev nodes rather than equally spaced nodes;
- reducing the degree of the global polynomial.

5.6 Two-Dimensional Interpolation

Two-dimensional interpolation estimates a function $z = f(x, y)$ from values known on a grid or scattered collection of points.

For a rectangular cell

$$[x_0, x_1] \times [y_0, y_1],$$

define

$$s = \frac{x - x_0}{x_1 - x_0}, \qquad t = \frac{y - y_0}{y_1 - y_0}.$$

The bilinear interpolant is

$$\begin{aligned} p(x, y) = {} & (1 - s)(1 - t)z_{00} + s(1 - t)z_{10} \\ & + (1 - s)tz_{01} + stz_{11}, \end{aligned}$$

where $z_{ij} = f(x_i, y_j)$.

Example 5.6.1. Suppose that

$$f(0, 0) = 0, \qquad f(1, 0) = 1,$$

$$f(0, 1) = 2, \qquad f(1, 1) = 3.$$

At $(x, y) = (0.25, 0.5)$, we have $s = 0.25$ and $t = 0.5$. Therefore,

$$\begin{aligned} p(0.25, 0.5) = {} & (0.75)(0.5)(0) + (0.25)(0.5)(1) \\ & + (0.75)(0.5)(2) + (0.25)(0.5)(3) \\ = {} & 1.25. \end{aligned}$$

For regularly spaced gridded data, common interpolation methods include nearest-neighbor, bilinear, bicubic, and spline interpolation. For scattered data, triangulation-based and radial-basis methods are often more appropriate.

5.7 Least-Squares Curve Fitting

Suppose that a function is approximated by

$$f(x) = \sum_{j=1}^{m} \beta_j r_j(x),$$

where the basis functions $r_1, \ldots, r_m$ are specified and the coefficients $\beta_1, \ldots, \beta_m$ are unknown.

The least-squares criterion is

$$J(\beta) = \sum_{i=1}^{n} \left(\sum_{j=1}^{m} \beta_j r_j(x_i) - y_i \right)^2.$$

Define the design matrix

$$R = \begin{pmatrix} r_1(x_1) & \cdots & r_m(x_1) \\ \vdots & \ddots & \vdots \\ r_1(x_n) & \cdots & r_m(x_n) \end{pmatrix},$$

the coefficient vector β, and the observation vector y. Then

$$J(\beta) = \|R\beta - y\|_2^2 .$$

Differentiating gives the normal equations

$$R^{\mathsf{T}} R\widehat{\beta} = R^{\mathsf{T}} y.$$

If the columns of R are linearly independent, then

$$\widehat{\beta} = (R^{\mathsf{T}} R)^{-1} R^{\mathsf{T}} y.$$

For numerical computation, QR decomposition or singular value decomposition is generally more stable than explicitly forming $(R^{\mathsf{T}} R)^{-1}$.

Example 5.7.1. Fit a straight line

$$y = \beta_0 + \beta_1 x$$

to the observations

$$(0,1), \qquad (1,2), \qquad (2,2), \qquad (3,4).$$

The sample means are $\bar{x} = 1.5$ and $\bar{y} = 2.25$. The least-squares slope is

$$\widehat{\beta}_1 = \frac{\sum_{i=1}^{4} (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{4} (x_i - \bar{x})^2} = \frac{4.5}{5} = 0.9.$$

The intercept is

$$\widehat{\beta}_0 = \bar{y} - \widehat{\beta}_1 \bar{x} = 0.9.$$

Hence, the fitted line is

$$\widehat{y} = 0.9 + 0.9x.$$

The fitted values are $0.9, 1.8, 2.7, 3.6$, and the residual sum of squares is

$$\mathrm{SSE} = 0.1^2 + 0.2^2 + (-0.7)^2 + 0.4^2 = 0.70.$$

Common curve families include

Model	Functional form
Linear	$y = a + bx$
Polynomial	$y = a_0 + a_1 x + \cdots + a_p x^p$
Power	$y = a x^b$
Exponential	$y = a e^{bx}$
Logarithmic	$y = a + b \ln x$
Reciprocal	$y = a + b/x$

Some nonlinear-looking models can be linearized. For example, $y = a e^{bx}$ becomes

$$\ln y = \ln a + bx$$

when $y > 0$. However, fitting the transformed model minimizes errors on the logarithmic scale rather than on the original scale.

5.8 Multiple Linear Regression

A multiple linear regression model has the form

$$y_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip} + \varepsilon_i, \qquad i = 1, \ldots, n.$$

In matrix form,

$$y = X\beta + \varepsilon.$$

The standard assumptions are

- the conditional mean is linear: $\mathbb{E}[y \mid X] = X\beta$;
- the errors have mean zero;
- the errors have constant variance;
- the errors are uncorrelated;
- the design matrix has full column rank;
- normality is imposed when exact small-sample inference is required.

The ordinary least-squares estimator is

$$\widehat{\beta} = (X^{\mathsf{T}}X)^{-1}X^{\mathsf{T}}y.$$

The fitted values and residuals are

$$\widehat{y} = X\widehat{\beta}, \qquad e = y - \widehat{y}.$$

The total, regression, and error sums of squares are

$$\mathrm{SST} = \sum_{i=1}^{n}(y_i - \bar{y})^2,$$

$$\mathrm{SSR} = \sum_{i=1}^{n}(\widehat{y}_i - \bar{y})^2, \qquad \mathrm{SSE} = \sum_{i=1}^{n}(y_i - \widehat{y}_i)^2.$$

For a model containing an intercept,

$$\mathrm{SST} = \mathrm{SSR} + \mathrm{SSE}\,.$$

The coefficient of determination is

$$R^2 = \frac{\mathrm{SSR}}{\mathrm{SST}} = 1 - \frac{\mathrm{SSE}}{\mathrm{SST}}.$$

For the straight-line example in the previous section,

$$\mathrm{SST} = 4.75, \qquad \mathrm{SSE} = 0.70,$$

so

$$R^2 = 1 - \frac{0.70}{4.75} \approx 0.8526.$$

Thus, approximately 85.26% of the sample variation in y is explained by the fitted linear relationship.

Because R^2 never decreases when predictors are added, model comparison often uses adjusted R^2:

$$\overline{R}^2 = 1 - \frac{\mathrm{SSE}/(n-p-1)}{\mathrm{SST}/(n-1)}.$$

Overall significance test. To test

$$H_0 : \beta_1 = \cdots = \beta_p = 0,$$

use

$$F = \frac{\mathrm{SSR}/p}{\mathrm{SSE}/(n-p-1)}.$$

Individual coefficient test. For predictor j, test $H_0 : \beta_j = 0$ using

$$t_j = \frac{\widehat{\beta}_j}{\mathrm{SE}(\widehat{\beta}_j)}.$$

Example 5.8.1. Suppose that a fitted housing-price model is

$$\widehat{y} = 50 + 2.4x_1 - 1.1x_2,$$

where y is measured in thousands of dollars, x_1 is floor area, and x_2 is distance from the city center. Holding distance fixed, one additional unit of floor area is associated with an expected price increase of 2.4 thousand dollars. Holding floor area fixed, one additional unit of distance is associated with an expected price decrease of 1.1 thousand dollars. These are conditional associations under the fitted model. They should not automatically be interpreted as causal effects.

5.9 Residual Diagnostics and Multicollinearity

A regression model should be examined through its residuals. Important diagnostic plots include

- residuals against fitted values;
- residuals against individual predictors;
- a normal Q–Q plot;
- scale-location plots;
- leverage and influence plots.

Typical residual patterns have different interpretations:

- systematic curvature suggests a missing nonlinear term;
- a funnel shape suggests nonconstant variance;
- long runs of positive or negative residuals suggest dependence;
- isolated large residuals suggest possible outliers;
- high-leverage points may exert excessive influence.

The standardized residual is

$$r_i = \frac{e_i}{\widehat{\sigma}\sqrt{1-h_{ii}}},$$

where h_{ii} is the i-th diagonal element of the hat matrix

$$H = X(X^{\mathsf{T}}X)^{-1}X^{\mathsf{T}}.$$

Variance inflation factor. To measure multicollinearity, regress predictor x_j on the remaining predictors and let R_j^2 be the resulting coefficient of determination. The variance inflation factor is

$$\mathrm{VIF}_j = \frac{1}{1-R_j^2}.$$

Example 5.9.1. If predictor x_j can be explained by the remaining predictors with $R_j^2 = 0.95$, then

$$\mathrm{VIF}_j = \frac{1}{1-0.95} = 20.$$

This indicates serious multicollinearity. The estimated coefficient of x_j may be unstable even if the model has strong overall predictive performance.

Possible responses to multicollinearity include

- removing a redundant predictor;
- combining related predictors;
- using principal component regression;
- collecting additional data;
- applying Ridge or Lasso regularization.

5.10 Ridge and Lasso Regression

When predictors are strongly correlated or the number of predictors is large, ordinary least squares may produce unstable coefficient estimates.
Before applying regularization, the predictors should normally be standardized so that the penalty treats them on comparable scales. The intercept is usually not penalized.
Ridge regression. Ridge regression minimizes

$$\|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2, \qquad \lambda \geq 0.$$

Its solution is

$$\widehat{\beta}^{\mathrm{Ridge}} = (X^{\mathsf{T}}X + \lambda I)^{-1}X^{\mathsf{T}}y.$$

When $\lambda = 0$, Ridge regression reduces to ordinary least squares. As λ increases, the coefficients are increasingly shrunk toward zero.

Example 5.10.1. Suppose that

$$X^{\mathsf{T}}X = \begin{pmatrix} 100 & 0 \\ 0 & 1 \end{pmatrix}, \qquad X^{\mathsf{T}}y = \begin{pmatrix} 100 \\ 4 \end{pmatrix}.$$

The ordinary least-squares estimate is

$$\widehat{\beta}^{\mathrm{OLS}} = \begin{pmatrix} 1 \\ 4 \end{pmatrix}.$$

With $\lambda = 3$, the Ridge estimate is

$$\widehat{\beta}^{\text{Ridge}} = \begin{pmatrix} 100/103 \\ 4/4 \end{pmatrix} \approx \begin{pmatrix} 0.971 \\ 1 \end{pmatrix}.$$

The weakly determined second coefficient is shrunk much more strongly.

Lasso regression. Lasso minimizes

$$\|y - X\beta\|_2^2 + \lambda \|\beta\|_1 .$$

Because the L^1 penalty is not differentiable at zero, Lasso may set some coefficients exactly equal to zero. It therefore performs estimation and variable selection simultaneously.
For an orthonormal design, the Lasso solution has the soft-thresholding form

$$\widehat{\beta}_j = \operatorname{sgn}(z_j) \max\{|z_j| - \lambda, 0\}.$$

Example 5.10.2. Suppose that the unregularized coefficients in an orthonormal design are

$$z = (3, 0.5),$$

and let $\lambda = 1$. Soft thresholding gives

$$\widehat{\beta}^{\text{Lasso}} = (2, 0).$$

The first predictor is retained with a smaller coefficient, while the second is removed from the model.

The regularization parameter should usually be selected by cross-validation. Training error alone cannot determine λ, because it is minimized at or near $\lambda = 0$.

5.11 Model Selection and Predictive Validation

A fitted model should be evaluated according to the goal of the analysis.
If the goal is explanation, attention should be given to assumptions, coefficient interpretation, uncertainty, and possible confounding. If the goal is prediction, performance on unseen observations is more important than in-sample significance.
Common prediction-error measures are

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^{n} |y_i - \widehat{y}_i| ,$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^{n} (y_i - \widehat{y}_i)^2},$$

and

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^{n} \left| \frac{y_i - \widehat{y}_i}{y_i} \right| .$$

MAPE is undefined when $y_i = 0$ and can be unstable when y_i is close to zero.
In K-fold cross-validation, the observations are divided into K subsets. The model is fitted on $K - 1$ subsets and evaluated on the remaining subset. Repeating this procedure over all folds gives

$$\text{CV} = \frac{1}{K} \sum_{k=1}^{K} \text{Error}_k .$$

Example 5.11.1. Suppose that a degree-8 polynomial has training RMSE 0.10 and cross-validation RMSE 1.80, while a cubic model has training RMSE 0.35 and cross-validation RMSE 0.48. The degree-8 model fits the training sample more closely but generalizes poorly. The cubic model should generally be preferred for prediction.

5.12 MATLAB Implementation

MATLAB provides `interp1` for one-dimensional interpolation.

```
x = [0, 1, 2];
y = [1, 3, 2];

xQuery = linspace(0,2,200);

yLinear = interp1( ...
    x, y, xQuery, 'linear');

ySpline = interp1( ...
    x, y, xQuery, 'spline');

yPchip = interp1( ...
    x, y, xQuery, 'pchip');

figure;
plot(x,y,'ko', ...
    'MarkerFaceColor','k');
hold on;

plot(xQuery,yLinear,'b-', ...
    'LineWidth',1.2);
plot(xQuery,ySpline,'r--', ...
    'LineWidth',1.2);
plot(xQuery,yPchip,'g-.', ...
    'LineWidth',1.2);

legend('Data','Linear', ...
    'Spline','PCHIP', ...
    'Location','best');

xlabel('x');
ylabel('y');
title('Interpolation Methods');
grid on;
```

Listing 5.1: One-dimensional interpolation

The function `polyfit` calculates polynomial least-squares coefficients, while `polyval` evaluates the fitted polynomial.

```
x = [0; 1; 2; 3];
y = [1; 2; 2; 4];

degree = 1;

coefficient = polyfit(x,y,degree);
yFitted = polyval(coefficient,x);

residual = y - yFitted;

```

```
SSE = sum(residual.^2);
SST = sum((y-mean(y)).^2);
R2 = 1 - SSE/SST;

disp('Polynomial coefficients:');
disp(coefficient);

fprintf('SSE = %.4f\n', SSE);
fprintf('R-squared = %.4f\n', R2);

xPlot = linspace(min(x),max(x),200);
yPlot = polyval(coefficient,xPlot);

figure;
scatter(x,y,50,'filled');
hold on;
plot(xPlot,yPlot,'r-', ...
    'LineWidth',1.5);

xlabel('x');
ylabel('y');
title('Least-Squares Line');
grid on;
```

Listing 5.2: Polynomial curve fitting

The function `fitlm` constructs a linear regression model and provides coefficient tests and diagnostic quantities.

```
x1 = [50; 60; 70; 80; 90; 100];
x2 = [10; 8; 7; 5; 4; 2];
y = [105; 130; 151; 180; 201; 230];

T = table(x1,x2,y);

model = fitlm(T, ...
    'y ~ x1 + x2');

disp(model);
disp(model.Coefficients);
disp(model.Rsquared);

% Residual diagnostic plots
figure;
plotResiduals(model,'fitted');

figure;
plotDiagnostics(model,'cookd');

figure;
plotResiduals(model,'probability');
```

Listing 5.3: Multiple linear regression

Variance inflation factors can be calculated by regressing each predictor on the remaining predictors.

```
X = [x1, x2];
p = size(X,2);

VIF = zeros(p,1);

for j = 1:p
    otherIndex = setdiff(1:p,j);

    auxiliaryModel = fitlm( ...
        X(:,otherIndex), X(:,j));

    RjSquared = ...
        auxiliaryModel.Rsquared.Ordinary;

    VIF(j) = 1/(1-RjSquared);
end

disp('Variance inflation factors:');
disp(VIF);
```

Listing 5.4: Variance inflation factors

The function `ridge` computes Ridge-regression paths. The function `lasso` computes Lasso solutions and can select the regularization parameter using cross-validation.

```
X = [x1, x2];
yResponse = y;

% Standardize predictors
XStandard = normalize(X,'zscore');

% Ridge path
lambdaRidge = logspace(-3,3,100);
betaRidge = ridge( ...
    yResponse, XStandard, ...
    lambdaRidge, 0);

figure;
semilogx(lambdaRidge, ...
    betaRidge', 'LineWidth',1.2);

xlabel('Lambda');
ylabel('Coefficient');
title('Ridge Coefficient Paths');
grid on;

% Lasso with ten-fold cross-validation
[betaLasso, fitInfo] = lasso( ...
    XStandard, yResponse, ...
    'CV', 10, ...
```

```
    'Standardize', false);

bestIndex = fitInfo.IndexMinMSE;
bestBeta = betaLasso(:,bestIndex);
bestIntercept = ...
    fitInfo.Intercept(bestIndex);

disp('Selected Lasso coefficients:');
disp(bestBeta);

fprintf('Selected intercept = %.4f\n', ...
    bestIntercept);
fprintf('Selected lambda = %.6f\n', ...
    fitInfo.Lambda(bestIndex));

lassoPlot(betaLasso,fitInfo, ...
    'PlotType','CV');
```

Listing 5.5: Ridge and Lasso regression

Two-dimensional gridded interpolation can be performed using `interp2`.

```
x = [0, 1];
y = [0, 1];

% Rows correspond to y and columns to x
Z = [0, 1;
     2, 3];

xQuery = 0.25;
yQuery = 0.50;

zQuery = interp2( ...
    x, y, Z, ...
    xQuery, yQuery, ...
    'linear');

fprintf('Interpolated value = %.4f\n', ...
    zQuery);
```

Listing 5.6: Two-dimensional interpolation

Remark 5.12.1. Interpolation, curve fitting, and regression may use similar polynomial expressions, but their objectives are different. The numerical method should be chosen according to whether the data are treated as exact, noisy, or stochastic.

Chapter 6

Differential Equations and Dynamical-System Models

6.1 From Dynamic Mechanisms to Differential Equations

A differential-equation model describes how the state of a system changes continuously with time. If

$$x(t) = \Big(x_1(t), x_2(t), \ldots, x_n(t)\Big)^{\mathsf{T}}$$

is the state vector, a first-order dynamical system has the form

$$\frac{dx}{dt} = F(t, x; \theta), \qquad x(t_0) = x_0,$$

where θ is a vector of model parameters and x_0 is the initial state.
A differential-equation model is usually constructed from a balance law:

$$\text{rate of change} = \text{inflow} - \text{outflow} + \text{internal production}.$$

The modeling procedure consists of

1. defining the state variables and their units;
2. identifying the mechanisms that increase or decrease each state;
3. translating the mechanisms into rates;
4. specifying parameter meanings and initial conditions;
5. solving or simulating the equations;
6. comparing the solution with observations.

A higher-order differential equation can be converted into a first-order system. For example, if

$$y'' = f(t, y, y'),$$

define $x_1 = y$ and $x_2 = y'$. Then

$$x_1' = x_2, \qquad x_2' = f(t, x_1, x_2).$$

Example 6.1.1. Consider

$$y'' + 4y' + 29y = 0, \qquad y(0) = 0, \qquad y'(0) = 1.$$

The characteristic equation is

$$r^2 + 4r + 29 = 0,$$

whose roots are $-2 \pm 5i$. Therefore,

$$y(t) = e^{-2t}\Big(C_1 \cos 5t + C_2 \sin 5t\Big).$$

The initial conditions give $C_1 = 0$ and $C_2 = 1/5$, so

$$y(t) = \frac{1}{5} e^{-2t} \sin 5t.$$

The solution oscillates while its amplitude decays exponentially.

6.2 Equilibria and Linear Stability

For an autonomous system

$$x' = F(x),$$

an equilibrium point x^* satisfies

$$F(x^*) = 0.$$

If the system starts at x^*, it remains there for all time.
To study the behavior near x^*, write $x = x^* + u$. The linearized system is

$$u' = J(x^*)u,$$

where

$$J(x^*) = \left(\frac{\partial F_i}{\partial x_j}(x^*)\right)_{n\times n}$$

is the Jacobian matrix.
For a continuous-time system,

- if every eigenvalue of $J(x^*)$ has negative real part, the equilibrium is locally asymptotically stable;
- if at least one eigenvalue has positive real part, the equilibrium is unstable;
- if no eigenvalue has positive real part but some eigenvalue has zero real part, linearization alone may be inconclusive.

Example 6.2.1. Consider

$$x' = x(1 - x).$$

The equilibria are $x^* = 0$ and $x^* = 1$. Since

$$F'(x) = 1 - 2x,$$

we have

$$F'(0) = 1 > 0, \qquad F'(1) = -1 < 0.$$

Therefore, $x^* = 0$ is unstable, while $x^* = 1$ is locally asymptotically stable.

For a two-dimensional system, the phase plane displays trajectories in the (x_1, x_2)-plane. Nullclines, defined by $x_1' = 0$ and $x_2' = 0$, help identify equilibria and the directions of motion.

6.3 Numerical Solution of Differential Equations

Many differential equations do not have an elementary closed-form solution. They must be approximated numerically.
For the initial-value problem

$$y' = f(t, y), \qquad y(t_0) = y_0,$$

Euler's method with step size h is

$$y_{k+1} = y_k + hf(t_k, y_k), \qquad t_{k+1} = t_k + h.$$

Example 6.3.1. Apply Euler's method with $h = 0.1$ to

$$y' = y, \qquad y(0) = 1.$$

The first two approximations are

$$y_1 = 1 + 0.1(1) = 1.1$$

and

$$y_2 = 1.1 + 0.1(1.1) = 1.21.$$

The exact value is $y(0.2) = e^{0.2} \approx 1.2214$, so the error after two steps is approximately 0.0114.

The classical fourth-order Runge–Kutta method is

$$\begin{aligned} k_1 &= f(t_k, y_k), \\ k_2 &= f\left(t_k + \frac{h}{2}, y_k + \frac{h}{2}k_1\right), \\ k_3 &= f\left(t_k + \frac{h}{2}, y_k + \frac{h}{2}k_2\right), \\ k_4 &= f(t_k + h, y_k + hk_3), \\ y_{k+1} &= y_k + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4). \end{aligned}$$

Adaptive solvers such as MATLAB's `ode45` automatically adjust the step size according to estimated local error. A smaller tolerance generally increases accuracy and computational cost.

A numerical solution should be checked by

- repeating the calculation with tighter tolerances;
- comparing several solvers when stiffness is suspected;
- verifying conserved quantities and nonnegativity;
- comparing with an exact solution when one is available.

6.4 Malthus Exponential-Growth Model

The Malthus model assumes that the net population growth rate is proportional to the current population:

$$\frac{dN}{dt} = rN, \qquad N(0) = N_0.$$

The parameter r is the constant per-capita net growth rate. Separating variables gives

$$\frac{dN}{N} = r\,dt,$$

so

$$N(t) = N_0 e^{rt}.$$

If $r > 0$, the population grows exponentially. If $r < 0$, it decays exponentially. For $r > 0$, the doubling time is

$$T_2 = \frac{\ln 2}{r}.$$

Example 6.4.1. Suppose that a population begins at $N_0 = 1000$ and has annual net growth rate $r = 0.03$. After ten years,

$$N(10) = 1000e^{0.3} \approx 1349.86.$$

The doubling time is

$$T_2 = \frac{\ln 2}{0.03} \approx 23.10$$

years.

The Malthus model is appropriate only when resources and environmental constraints can be neglected. Long-term exponential growth is usually unrealistic in a bounded environment.

6.5 Logistic Growth Model

The Logistic model introduces a carrying capacity K by allowing the per-capita growth rate to decrease with population size:

$$\frac{1}{N}\frac{dN}{dt} = r\left(1 - \frac{N}{K}\right).$$

Therefore,

$$\frac{dN}{dt} = rN\left(1 - \frac{N}{K}\right), \qquad N(0) = N_0.$$

The solution is

$$N(t) = \frac{K}{1 + \left(\dfrac{K - N_0}{N_0}\right)e^{-rt}}.$$

The equilibria are $N^* = 0$ and $N^* = K$. If $r > 0$, the zero equilibrium is unstable and the carrying-capacity equilibrium is stable.
The second derivative satisfies

$$N'' = rN'\left(1 - \frac{2N}{K}\right).$$

Thus, the inflection point occurs at $N = K/2$. Growth accelerates below $K/2$ and decelerates above $K/2$.

Example 6.5.1. Suppose that

$$K = 1000, \qquad r = 0.4, \qquad N_0 = 100.$$

Then

$$N(t) = \frac{1000}{1 + 9e^{-0.4t}}.$$

The population reaches the inflection level $N = 500$ when

$$500 = \frac{1000}{1 + 9e^{-0.4t}}.$$

Therefore,

$$9e^{-0.4t} = 1,$$

so

$$t = \frac{\ln 9}{0.4} \approx 5.49.$$

The population grows most rapidly at approximately $t = 5.49$.

Remark 6.5.2. The parameter K is not necessarily a fixed biological constant. It may change when technology, resources, climate, or policy changes.

6.6 Predator–Prey Model

The Lotka–Volterra predator–prey model describes the interaction between a prey population $x(t)$ and a predator population $y(t)$:

$$\frac{dx}{dt} = ax - bxy,$$
$$\frac{dy}{dt} = cxy - dy,$$

where

- $a > 0$ is the natural growth rate of the prey;
- $b > 0$ measures the rate at which predators consume prey;
- $d > 0$ is the natural death rate of the predators;
- $c > 0$ measures the conversion of consumed prey into predator growth.

The prey grows exponentially in the absence of predators, while the predator declines exponentially in the absence of prey.
The equilibria are

$$(0, 0)$$

and

$$\left(\frac{d}{c}, \frac{a}{b}\right).$$

The positive equilibrium represents coexistence.
The system has the conserved quantity

$$H(x, y) = cx - d \ln x + by - a \ln y.$$

Indeed, along every positive solution,

$$\frac{d}{dt} H(x(t), y(t)) = 0.$$

Consequently, the classical Lotka–Volterra model produces closed orbits around the coexistence equilibrium rather than convergence toward it.

Example 6.6.1. Let

$$a = 1, \qquad b = 0.1, \qquad c = 0.075, \qquad d = 1.5.$$

The positive equilibrium is

$$x^* = \frac{1.5}{0.075} = 20, \qquad y^* = \frac{1}{0.1} = 10.$$

The Jacobian at this equilibrium is

$$J(x^*, y^*) = \begin{pmatrix} 0 & -2 \\ 0.75 & 0 \end{pmatrix}.$$

Its eigenvalues are

$$\lambda = \pm i\sqrt{1.5}.$$

Thus, the linearized system has oscillatory behavior. In the classical model, positive trajectories form closed curves around $(20, 10)$.

The classical model is idealized because it omits carrying capacities, saturation of predation, seasonal forcing, and random environmental effects.

6.7 Competition Between Two Populations

Two populations competing for limited resources may be described by

$$\frac{dx}{dt} = r_1 x\left(1 - \frac{x + \alpha_{12} y}{K_1}\right),$$

$$\frac{dy}{dt} = r_2 y\left(1 - \frac{y + \alpha_{21} x}{K_2}\right).$$

Here, K_1, K_2 are the carrying capacities, while α_{12} and α_{21} measure the interspecific competitive effects.

The positive coexistence equilibrium satisfies

$$x + \alpha_{12} y = K_1,$$

$$\alpha_{21} x + y = K_2.$$

Provided $1 - \alpha_{12}\alpha_{21} \neq 0$, it is

$$x^* = \frac{K_1 - \alpha_{12} K_2}{1 - \alpha_{12}\alpha_{21}},$$

$$y^* = \frac{K_2 - \alpha_{21} K_1}{1 - \alpha_{12}\alpha_{21}}.$$

A biologically meaningful coexistence equilibrium requires both coordinates to be positive.

Example 6.7.1. Let

$$K_1 = 100, \qquad K_2 = 80,$$

$$\alpha_{12} = 0.5, \qquad \alpha_{21} = 0.4.$$

Then

$$x^* = \frac{100 - 0.5(80)}{1 - 0.5(0.4)} = 75$$

and

$$y^* = \frac{80 - 0.4(100)}{1 - 0.5(0.4)} = 50.$$

Thus, the model predicts coexistence at

$$(x^*, y^*) = (75, 50).$$

Depending on the competition coefficients, the model may predict stable coexistence, extinction of one species, or dependence on the initial populations.

6.8 SI and SIS Epidemic Models

Compartment models divide a population into epidemiological states. After normalizing the total population to 1, let $s(t)$ and $i(t)$ denote the susceptible and infectious proportions.

SI model. The SI model assumes that infected individuals never recover:

$$s + i = 1,$$

$$\frac{ds}{dt} = -\beta s i, \qquad \frac{di}{dt} = \beta s i.$$

Using $s = 1 - i$, the infectious proportion satisfies

$$\frac{di}{dt} = \beta i(1 - i).$$

This is a Logistic equation with solution

$$i(t) = \frac{1}{1 + \left(\dfrac{1 - i_0}{i_0}\right) e^{-\beta t}}.$$

Example 6.8.1. Let $\beta = 0.2$ and $i_0 = 0.01$. Then

$$i(t) = \frac{1}{1 + 99e^{-0.2t}}.$$

At time $t = 20$,

$$i(20) = \frac{1}{1 + 99e^{-4}} \approx 0.355.$$

Thus, approximately 35.5% of the population is infectious at that time under the assumptions of the SI model.

SIS model. The SIS model assumes that recovered individuals immediately become susceptible again:

$$s + i = 1,$$

$$\frac{di}{dt} = \beta si - \gamma i.$$

Using $s = 1 - i$,

$$\frac{di}{dt} = i\Big[(\beta - \gamma) - \beta i\Big].$$

Define

$$R_0 = \frac{\beta}{\gamma}.$$

If $R_0 \leq 1$, the disease-free equilibrium $i^* = 0$ is stable. If $R_0 > 1$, there is an endemic equilibrium

$$i^* = 1 - \frac{\gamma}{\beta} = 1 - \frac{1}{R_0}.$$

Example 6.8.2. Suppose that $\beta = 0.3$ and $\gamma = 0.1$. Then

$$R_0 = \frac{0.3}{0.1} = 3 > 1.$$

The endemic equilibrium is

$$i^* = 1 - \frac{1}{3} = \frac{2}{3}.$$

The SIS model therefore predicts that the infectious proportion approaches 2/3, provided the initial infectious proportion is positive.

6.9 SIR Epidemic Model

The SIR model divides the population into susceptible, infectious, and removed groups:

$$s(t) + i(t) + r(t) = 1.$$

The governing equations are

$$\frac{ds}{dt} = -\beta si,$$
$$\frac{di}{dt} = \beta si - \gamma i,$$
$$\frac{dr}{dt} = \gamma i.$$

The parameter β is the transmission rate and γ is the removal or recovery rate. The mean infectious period is $1/\gamma$.
The basic reproduction number is

$$R_0 = \frac{\beta}{\gamma}.$$

At time t, the effective reproduction factor is proportional to

$$R_{\text{eff}}(t) = R_0 s(t).$$

Since

$$\frac{di}{dt} = i\Big(\beta s - \gamma\Big),$$

the infectious population initially increases if

$$R_0 s(0) > 1$$

and decreases if $R_0 s(0) < 1$.
The infection reaches a peak when

$$s(t_{\text{peak}}) = \frac{\gamma}{\beta} = \frac{1}{R_0}.$$

Dividing the equations for i and s gives

$$\frac{di}{ds} = -1 + \frac{\gamma}{\beta s}.$$

Therefore,

$$i + s - \frac{\gamma}{\beta} \ln s$$

is constant along a trajectory.

Example 6.9.1. Suppose that

$$\beta = 0.3, \qquad \gamma = 0.1,$$

$$s(0) = 0.99, \qquad i(0) = 0.01, \qquad r(0) = 0.$$

Then $R_0 = 3$, and

$$R_0 s(0) = 2.97 > 1,$$

so the epidemic initially grows.
The infectious proportion reaches its peak when

$$s = \frac{1}{3}.$$

Using the conserved relationship,

$$\begin{aligned} i_{\text{peak}} &= i_0 + s_0 - \frac{1}{3} - \frac{1}{3}\ln\left(\frac{s_0}{1/3}\right) \\ &\approx 0.304. \end{aligned}$$

Thus, the model predicts a peak infectious proportion of approximately 30.4%.

As $t \to \infty$, $i(t) \to 0$, but the limiting susceptible proportion s_∞ is generally positive. When $r(0) = 0$, it satisfies the final-size equation

$$\ln\left(\frac{s_\infty}{s_0}\right) = -R_0\big(1 - s_\infty\big).$$

6.10 SEIR, SIRS, and Extended Compartment Models

The SIR framework can be extended when the biological mechanism requires additional states.
SEIR model. If infection is followed by a latent period during which individuals are not yet infectious, introduce an exposed compartment $e(t)$:

$$\begin{aligned} s + e + i + r &= 1, \\ \frac{ds}{dt} &= -\beta si, \\ \frac{de}{dt} &= \beta si - \sigma e, \\ \frac{di}{dt} &= \sigma e - \gamma i, \\ \frac{dr}{dt} &= \gamma i. \end{aligned}$$

The mean latent period is $1/\sigma$.

Example 6.10.1. Suppose that

$$\beta = 0.25, \qquad \sigma = 0.25, \qquad \gamma = 0.10.$$

The mean latent period is

$$\frac{1}{\sigma} = 4$$

days, and the mean infectious period is

$$\frac{1}{\gamma} = 10$$

days. Under the standard frequency-dependent SEIR assumptions,

$$R_0 = \frac{\beta}{\gamma} = 2.5.$$

The exposed compartment delays the appearance of infectious individuals but does not change this basic threshold formula.

SIRS model. If immunity is temporary, removed individuals return to the susceptible class at rate ω:

$$\frac{ds}{dt} = -\beta si + \omega r,$$
$$\frac{di}{dt} = \beta si - \gamma i,$$
$$\frac{dr}{dt} = \gamma i - \omega r.$$

The mean duration of immunity is $1/\omega$.
Additional compartments may describe vaccination, hospitalization, quarantine, age structure, or disease-related death. Each extension should correspond to a real mechanism and identifiable data rather than being added only to make the model appear more complicated.

6.11 Parameter Estimation and Model Validation

Differential-equation parameters are often estimated by minimizing the difference between observations and numerical model solutions. Suppose that observations y_k are available at times t_k, while the model output is $h(x(t_k;\theta))$. A nonlinear least-squares estimator is

$$\widehat{\theta} = \arg\min_{\theta} \sum_{k=1}^{N} \|y_k - h(x(t_k;\theta))\|_2^2 .$$

For the exponential-growth model, the logarithmic transformation gives

$$\ln N(t) = \ln N_0 + rt.$$

Thus, r can be estimated by linear regression of $\ln N(t)$ on t.

Example 6.11.1. Suppose that a population increases from 100 to 150 over five years and is assumed to follow the Malthus model. Then

$$150 = 100e^{5r},$$

so

$$\widehat{r} = \frac{\ln(150/100)}{5} \approx 0.0811.$$

The estimated annual continuous growth rate is approximately 8.11%.

A fitted dynamical model should be checked using

- residual plots over time;
- out-of-sample prediction;
- sensitivity to initial conditions and parameters;
- confidence intervals or bootstrap uncertainty;
- conservation laws and nonnegativity;
- comparison with simpler alternative models;
- identifiability of the parameters.

A small fitting error does not guarantee that every parameter is identifiable. Different parameter combinations may produce nearly identical trajectories.

6.12 MATLAB Implementation

Symbolic solutions can be obtained using `dsolve` when MATLAB can express the solution analytically.

```
syms y(t)

equation = diff(y,t,2) ...
    + 4*diff(y,t) + 29*y == 0;

condition1 = y(0) == 0;
condition2 = subs(diff(y,t),t,0) == 1;

solution = dsolve( ...
    equation, condition1, condition2);

solution = simplify(solution);

disp(solution);
```

Listing 6.1: Symbolic solution of a differential equation

The Logistic model can be solved numerically with `ode45`.

```
r = 0.4;
K = 1000;
N0 = 100;

logisticModel = @(t,N) ...
    r*N*(1-N/K);

timeInterval = [0,25];

[t,N] = ode45( ...
    logisticModel, timeInterval, N0);

figure;
plot(t,N,'b-', ...
    'LineWidth',1.5);
hold on;

yline(K,'r--', ...
    'Carrying capacity');
yline(K/2,'k:', ...
    'Inflection level');

xlabel('Time');
ylabel('Population');
title('Logistic Growth');
grid on;
```

Listing 6.2: Numerical solution of the Logistic model

The predator–prey model is a two-dimensional system.

```
a = 1.0;
b = 0.1;
c = 0.075;
d = 1.5;

model = @(t,z) [ ...
    a*z(1) - b*z(1)*z(2);
    c*z(1)*z(2) - d*z(2)];

initialState = [30; 5];
timeInterval = [0,50];

[t,z] = ode45( ...
    model, timeInterval, initialState);

prey = z(:,1);
predator = z(:,2);

figure;
plot(t,prey,'b-', ...
    'LineWidth',1.3);
hold on;
plot(t,predator,'r-', ...
    'LineWidth',1.3);

legend('Prey','Predator', ...
    'Location','best');
xlabel('Time');
ylabel('Population');
title('Predator--Prey Dynamics');
grid on;

figure;
plot(prey,predator,'k-', ...
    'LineWidth',1.3);
hold on;

plot(d/c,a/b,'ro', ...
    'MarkerFaceColor','r');

xlabel('Prey');
ylabel('Predator');
title('Phase Plane');
grid on;
```

Listing 6.3: Lotka–Volterra predator–prey model

The following code simulates an SIR epidemic.

```
beta = 0.3;
gamma = 0.1;

sirModel = @(t,z) [ ...
```

```
    -beta*z(1)*z(2);
     beta*z(1)*z(2)-gamma*z(2);
     gamma*z(2)];

initialState = [0.99; 0.01; 0];
timeInterval = [0,160];

[t,z] = ode45( ...
    sirModel, timeInterval, initialState);

susceptible = z(:,1);
infectious = z(:,2);
removed = z(:,3);

[peakInfectious,peakIndex] = ...
    max(infectious);
peakTime = t(peakIndex);

figure;
plot(t,susceptible,'b-', ...
    'LineWidth',1.3);
hold on;
plot(t,infectious,'r-', ...
    'LineWidth',1.3);
plot(t,removed,'g-', ...
    'LineWidth',1.3);

legend('Susceptible', ...
    'Infectious','Removed', ...
    'Location','best');

xlabel('Time');
ylabel('Population proportion');
title('SIR Epidemic Model');
grid on;

fprintf('Peak infectious proportion = %.4f\n', ...
    peakInfectious);
fprintf('Peak time = %.4f\n', peakTime);
```

Listing 6.4: SIR epidemic model

Parameters can be estimated by repeatedly solving the differential equation inside an optimization routine.

```
timeData = [0; 2; 4; 6; 8; 10];
populationData = ...
    [100; 198; 355; 551; 713; 824];

N0 = populationData(1);

objective = @(theta) ...
    logisticResidual( ...
```

```
        theta,timeData,populationData,N0);

initialGuess = [0.3; 1000];
lowerBound = [0; max(populationData)];
upperBound = [2; 10000];

estimatedTheta = lsqnonlin( ...
    objective, initialGuess, ...
    lowerBound, upperBound);

estimatedR = estimatedTheta(1);
estimatedK = estimatedTheta(2);

fprintf('Estimated r = %.6f\n', ...
    estimatedR);
fprintf('Estimated K = %.6f\n', ...
    estimatedK);

function residual = logisticResidual( ...
    theta,timeData,populationData,N0)

    r = theta(1);
    K = theta(2);

    model = @(t,N) r*N*(1-N/K);

    [~,NModel] = ode45( ...
        model,timeData,N0);

    residual = NModel-populationData;
end
```

Listing 6.5: Estimating Logistic-model parameters

Remark 6.12.1. A numerical trajectory is conditional on the equations, parameters, and initial values. Solver accuracy cannot compensate for incorrect mechanisms or poorly estimated parameters.

Chapter 7

Time-Series Analysis and Forecasting Models

7.1 Time-Series Components and Decomposition

A time series is a sequence of observations ordered in time:

$$\{y_t : t = 1, 2, \ldots, T\}.$$

Unlike cross-sectional observations, time-series observations are generally dependent. The order of the data therefore contains essential information.
A time series is commonly described using four components:

- the trend component T_t, describing long-term movement;
- the seasonal component S_t, describing regular variation within a fixed period;
- the cyclical component C_t, describing longer and less regular fluctuations;
- the irregular component R_t, describing unpredictable variation.

Additive decomposition. When the magnitude of the seasonal variation is approximately constant,

$$y_t = T_t + S_t + C_t + R_t.$$

Multiplicative decomposition. When the seasonal variation increases or decreases proportionally with the level of the series,

$$y_t = T_t S_t C_t R_t.$$

For a positive series, logarithms convert the multiplicative model into an additive one:

$$\ln y_t = \ln T_t + \ln S_t + \ln C_t + \ln R_t.$$

Example 7.1.1. Suppose that quarterly sales fluctuate by approximately 20 units above or below the trend regardless of the total sales level. An additive model is appropriate. If seasonal fluctuations are approximately 10% of the current sales level, the seasonal amplitude grows with the trend. A multiplicative model is then more appropriate.

A decomposition is descriptive rather than automatically predictive. Forecasting requires a rule for extending the estimated components into the future.

7.2 Moving-Average Models

A moving average smooths short-term fluctuations by replacing each observation with an average of nearby observations.
For a trailing window of length m, the one-step-ahead forecast is

$$\widehat{y}_{t+1} = \frac{1}{m} \sum_{j=0}^{m-1} y_{t-j}.$$

A larger m produces a smoother sequence but responds more slowly to structural changes. The recursion

$$M_{t+1} = M_t + \frac{1}{m}(y_{t+1} - y_{t-m+1})$$

allows a moving average to be updated without recomputing the entire sum.
A centered moving average is used primarily for trend extraction rather than real-time forecasting. For odd $m = 2q + 1$,

$$\widetilde{M}_t = \frac{1}{m} \sum_{j=-q}^{q} y_{t+j}.$$

Example 7.2.1. Consider

$$y_1 = 10, \quad y_2 = 12, \quad y_3 = 13, \quad y_4 = 15, \quad y_5 = 16.$$

Using a three-period moving average,

$$\widehat{y}_4 = \frac{10 + 12 + 13}{3} \approx 11.67,$$

$$\widehat{y}_5 = \frac{12 + 13 + 15}{3} \approx 13.33.$$

The forecast for the next unobserved value is

$$\widehat{y}_6 = \frac{13 + 15 + 16}{3} \approx 14.67.$$

Remark 7.2.2. A simple moving average does not explicitly model trend or seasonality. Applying it directly to a strongly trending series generally produces lagged forecasts.

7.3 Simple Exponential Smoothing

Simple exponential smoothing is suitable for a series with no pronounced trend or seasonality. Let ℓ_t be the estimated level at time t. The updating equation is

$$\ell_t = \alpha y_t + (1 - \alpha)\ell_{t-1}, \qquad 0 < \alpha < 1.$$

The forecast is

$$\widehat{y}_{t+h|t} = \ell_t, \qquad h \geq 1.$$

Expanding the recursion gives

$$\ell_t = \alpha y_t + \alpha(1 - \alpha)y_{t-1} + \alpha(1 - \alpha)^2 y_{t-2} + \cdots .$$

Thus, recent observations receive larger weights, and the weights decrease geometrically into the past.
A large value of α makes the forecast respond quickly to recent changes. A small value produces stronger smoothing.

Example 7.3.1. Let $\alpha = 0.3$, and initialize the level using $\ell_1 = y_1 = 100$. Suppose that

$$y_2 = 110, \qquad y_3 = 105.$$

Then

$$\ell_2 = 0.3(110) + 0.7(100) = 103,$$

and

$$\ell_3 = 0.3(105) + 0.7(103) = 103.6.$$

Therefore, the forecasts made at time 3 are

$$\widehat{y}_{4|3} = \widehat{y}_{5|3} = 103.6.$$

The smoothing parameter can be selected by minimizing the in-sample one-step-ahead squared errors:

$$\mathrm{SSE}(\alpha) = \sum_{t=2}^{T} (y_t - \ell_{t-1})^2.$$

7.4 Holt Trend and Damped-Trend Models

Simple exponential smoothing cannot extrapolate a trend. Holt's linear-trend model maintains both a level ℓ_t and a trend b_t:

$$\ell_t = \alpha y_t + (1-\alpha)(\ell_{t-1} + b_{t-1}),$$

$$b_t = \beta(\ell_t - \ell_{t-1}) + (1-\beta)b_{t-1}.$$

The h-step-ahead forecast is

$$\widehat{y}_{t+h|t} = \ell_t + hb_t.$$

Example 7.4.1. Suppose that

$$y_t = 100, \qquad \ell_{t-1} = 95, \qquad b_{t-1} = 4,$$

with $\alpha = 0.4$ and $\beta = 0.3$. The updated level is

$$\ell_t = 0.4(100) + 0.6(95+4) = 99.4.$$

The updated trend is

$$b_t = 0.3(99.4 - 95) + 0.7(4) = 4.12.$$

Thus,

$$\widehat{y}_{t+1|t} = 99.4 + 4.12 = 103.52,$$

and

$$\widehat{y}_{t+3|t} = 99.4 + 3(4.12) = 111.76.$$

Extending a constant linear trend far into the future may produce unrealistic forecasts. A damped-trend model introduces a damping parameter $0 < \phi \leq 1$:

$$\ell_t = \alpha y_t + (1-\alpha)(\ell_{t-1} + \phi b_{t-1}),$$

$$b_t = \beta(\ell_t - \ell_{t-1}) + (1-\beta)\phi b_{t-1}.$$

The forecast is

$$\widehat{y}_{t+h|t} = \ell_t + \left(\phi + \phi^2 + \cdots + \phi^h\right) b_t.$$

If $\phi < 1$, the effect of the trend gradually decreases with the forecast horizon. When $\phi = 1$, the model reduces to Holt's undamped trend.

Example 7.4.2. If $\ell_t = 99.4$, $b_t = 4.12$, and $\phi = 0.8$, then the three-step forecast is

$$\begin{aligned}\widehat{y}_{t+3|t} &= 99.4 + (0.8 + 0.8^2 + 0.8^3)(4.12) \\ &\approx 107.44.\end{aligned}$$

This is lower than the undamped forecast 111.76 because the future trend is not extrapolated at full strength.

7.5 Holt–Winters Seasonal Models

Holt–Winters models combine level, trend, and seasonality. Let m be the seasonal period, such as $m = 4$ for quarterly data or $m = 12$ for monthly data.

Additive Holt–Winters model. Additive seasonality is suitable when seasonal fluctuations have approximately constant magnitude:

$$\ell_t = \alpha(y_t - s_{t-m}) + (1-\alpha)(\ell_{t-1} + b_{t-1}),$$

$$b_t = \beta(\ell_t - \ell_{t-1}) + (1-\beta)b_{t-1},$$

$$s_t = \gamma(y_t - \ell_t) + (1-\gamma)s_{t-m}.$$

The forecast is

$$\widehat{y}_{t+h|t} = \ell_t + hb_t + s_{t+h-m(k+1)},$$

where $k = \lfloor (h-1)/m \rfloor$.

Example 7.5.1. Suppose that

$$y_t = 130, \qquad \ell_{t-1} = 100, \qquad b_{t-1} = 5,$$

and the corresponding seasonal component from the preceding cycle is $s_{t-m} = 20$. Let

$$\alpha = 0.4, \qquad \beta = 0.2, \qquad \gamma = 0.3.$$

Then

$$\ell_t = 0.4(130 - 20) + 0.6(100 + 5) = 107,$$

$$b_t = 0.2(107 - 100) + 0.8(5) = 5.4,$$

and

$$s_t = 0.3(130 - 107) + 0.7(20) = 20.9.$$

If the seasonal component corresponding to the next period is 12, then

$$\widehat{y}_{t+1|t} = 107 + 5.4 + 12 = 124.4.$$

Multiplicative Holt–Winters model. Multiplicative seasonality is suitable when seasonal fluctuations are proportional to the level:

$$\ell_t = \alpha\frac{y_t}{s_{t-m}} + (1-\alpha)(\ell_{t-1} + b_{t-1}),$$

$$b_t = \beta(\ell_t - \ell_{t-1}) + (1-\beta)b_{t-1},$$

$$s_t = \gamma\frac{y_t}{\ell_t} + (1-\gamma)s_{t-m}.$$

The forecast is

$$\widehat{y}_{t+h|t} = (\ell_t + hb_t)s_{t+h-m(k+1)}.$$

Remark 7.5.2. A multiplicative seasonal model requires positive data. A logarithmic transformation can often convert multiplicative seasonal variation into an approximately additive pattern.

7.6 Stationarity, Differencing, and Autocorrelation

A time series $\{X_t\}$ is weakly stationary if

$$\mathbb{E}[X_t] = \mu$$

is independent of t,

$$\operatorname{Var}(X_t) = \sigma^2$$

is finite and independent of t, and

$$\operatorname{Cov}(X_t, X_{t-k}) = \gamma_k$$

depends only on the lag k.
The autocorrelation function, abbreviated as ACF, is

$$\rho_k = \frac{\gamma_k}{\gamma_0}.$$

For observations $x_1, \dots, x_n$, the sample autocorrelation at lag k is

$$\widehat{\rho}_k = \frac{\sum_{t=1}^{n-k}(x_t - \bar{x})(x_{t+k} - \bar{x})}{\sum_{t=1}^{n}(x_t - \bar{x})^2}.$$

The partial autocorrelation function, abbreviated as PACF, measures the association between X_t and X_{t-k} after removing the linear effects of the intermediate lags.

Example 7.6.1. For a stationary AR(1) process with coefficient $\phi = 0.7$, the theoretical autocorrelation function is

$$\rho_k = \phi^k = 0.7^k.$$

Thus,

$$\rho_1 = 0.7, \qquad \rho_2 = 0.49, \qquad \rho_3 = 0.343.$$

The ACF decreases gradually rather than becoming exactly zero after a fixed lag.

A trending series is generally nonstationary. The lag operator is defined by

$$LX_t = X_{t-1}.$$

The first-difference operator is

$$\Delta X_t = (1 - L)X_t = X_t - X_{t-1}.$$

The d-th difference is

$$\Delta^d X_t = (1 - L)^d X_t.$$

A seasonal difference with period m is

$$\Delta_m X_t = (1 - L^m)X_t = X_t - X_{t-m}.$$

Example 7.6.2. For the series

$$100, \quad 108, \quad 115, \quad 123,$$

the first differences are

$$8, \quad 7, \quad 8.$$

The original series has an increasing trend, whereas the differenced series fluctuates around approximately 7.67. Differencing may therefore produce a more nearly stationary series.

7.7 Autoregressive Models

An autoregressive model of order p, abbreviated as AR(p), is

$$X_t = c + \phi_1 X_{t-1} + \cdots + \phi_p X_{t-p} + \varepsilon_t,$$

where $\{\varepsilon_t\}$ is white noise with mean 0 and variance σ_ε^2.
Using the lag operator, the model can be written as

$$\phi(L)X_t = c + \varepsilon_t,$$

where

$$\phi(z) = 1 - \phi_1 z - \cdots - \phi_p z^p.$$

The AR(p) process is stationary when all roots of

$$\phi(z) = 0$$

lie outside the unit circle.
For an AR(1) model

$$X_t = c + \phi X_{t-1} + \varepsilon_t,$$

stationarity is equivalent to $|\phi| < 1$. Its stationary mean is

$$\mu = \frac{c}{1-\phi},$$

and its variance is

$$\mathrm{Var}(X_t) = \frac{\sigma_\varepsilon^2}{1-\phi^2}.$$

Example 7.7.1. Consider

$$X_t = 2 + 0.6X_{t-1} + \varepsilon_t.$$

The stationary mean is

$$\mu = \frac{2}{1-0.6} = 5.$$

If the current observation is $X_t = 10$, then the one-step forecast is

$$\widehat{X}_{t+1|t} = 2 + 0.6(10) = 8.$$

The h-step forecast is

$$\widehat{X}_{t+h|t} = 5 + 0.6^h(10-5).$$

Therefore, the forecast gradually returns to the long-run mean 5.

For an AR(p) process, the ACF typically decays gradually, while the PACF becomes zero after lag p in the ideal theoretical model.

7.8 Moving-Average and ARMA Models

A moving-average model of order q, abbreviated as MA(q), is

$$X_t = \mu + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \cdots + \theta_q \varepsilon_{t-q}.$$

The term "moving average" here refers to a stochastic model involving past shocks. It is different from the deterministic smoothing method introduced earlier.
For an MA(1) process,

$$X_t = \mu + \varepsilon_t + \theta \varepsilon_{t-1}.$$

Its autocorrelation function is

$$\rho_1 = \frac{\theta}{1+\theta^2},$$

and $\rho_k = 0$ for $k \geq 2$.

Example 7.8.1. Let

$$X_t = 10 + \varepsilon_t + 0.5\varepsilon_{t-1}.$$

If $\varepsilon_t = 2$ and $\varepsilon_{t-1} = -1$, then

$$X_t = 10 + 2 + 0.5(-1) = 11.5.$$

The one-step forecast is

$$\widehat{X}_{t+1|t} = 10 + 0.5\varepsilon_t = 11,$$

while forecasts at horizons $h \geq 2$ equal the mean 10.

An ARMA(p, q) model combines autoregressive terms and moving-average shock terms:

$$X_t = c + \sum_{i=1}^{p} \phi_i X_{t-i} + \varepsilon_t + \sum_{j=1}^{q} \theta_j \varepsilon_{t-j}.$$

Example 7.8.2. Consider the ARMA(1,1) model

$$X_t = 1 + 0.5X_{t-1} + \varepsilon_t + 0.4\varepsilon_{t-1}.$$

Suppose that $X_t = 8$ and the estimated current innovation is $\varepsilon_t = 1$. The one-step forecast is

$$\widehat{X}_{t+1|t} = 1 + 0.5(8) + 0.4(1) = 5.4.$$

At the second step, the future innovations have conditional mean zero, so

$$\widehat{X}_{t+2|t} = 1 + 0.5(5.4) = 3.7.$$

For a theoretical MA(q) model, the ACF becomes zero after lag q, whereas the PACF generally decays. For an ARMA model, both the ACF and PACF usually decay rather than exhibiting an exact cutoff.

7.9 ARIMA and Seasonal ARIMA Models

An ARIMA(p, d, q) model applies an ARMA(p, q) model to a d-times-differenced series:

$$\phi(L)(1 - L)^d X_t = c + \theta(L)\varepsilon_t.$$

Here,

- p is the autoregressive order;
- d is the number of ordinary differences;
- q is the moving-average order.

The orders should be selected using the time-series structure, ACF and PACF, information criteria, and residual diagnostics. Differencing should not be continued automatically once a series is sufficiently stationary, because over-differencing can introduce unnecessary dependence and increase forecast variance.

Example 7.9.1. An ARIMA(0,1,0) model with drift is

$$X_t - X_{t-1} = c + \varepsilon_t.$$

For the observations

$$100, \quad 108, \quad 115, \quad 123,$$

the mean first difference is

$$\widehat{c} = \frac{8 + 7 + 8}{3} = \frac{23}{3}.$$

The forecasts are

$$\widehat{X}_{5|4} = 123 + \frac{23}{3} \approx 130.67$$

and

$$\widehat{X}_{6|4} = 123 + 2\left(\frac{23}{3}\right) \approx 138.33.$$

For a seasonal period m, the seasonal ARIMA model is denoted by

$$\text{ARIMA}(p, d, q) \times (P, D, Q)_m.$$

Its general form is

$$\Phi(L^m)\phi(L)(1 - L)^d(1 - L^m)^D X_t = \Theta(L^m)\theta(L)\varepsilon_t.$$

Seasonal differencing removes patterns that repeat after m observations.

7.10 Estimation, Diagnostics, and Forecast Evaluation

A time-series model should be estimated using a chronological training sample. Randomly shuffling the observations destroys temporal order and can leak future information into the training data.
After fitting a model, the residuals should behave approximately like white noise:

$$\mathbb{E}[\varepsilon_t] = 0, \qquad \text{Var}(\varepsilon_t) = \sigma^2,$$

$$\text{Cov}(\varepsilon_t, \varepsilon_{t-k}) = 0 \quad \text{for } k \neq 0.$$

The Ljung–Box statistic tests whether several residual autocorrelations are jointly zero:

$$Q = n(n+2)\sum_{k=1}^{h}\frac{\widehat{\rho}_k^2}{n-k}.$$

A small p-value indicates that the residuals retain serial dependence and that the fitted model has not captured all predictable structure.
Models can be compared using

$$\mathrm{AIC} = -2\ell + 2k$$

and

$$\mathrm{BIC} = -2\ell + k\ln n,$$

where ℓ is the maximized log-likelihood and k is the number of estimated parameters. Smaller values are preferred among models fitted to the same response data.
Forecast performance should also be checked outside the fitting sample using MAE, RMSE, or another appropriate error measure.

Example 7.10.1. Suppose that two forecasting models have the following test-set errors:

Model	RMSE	Ljung–Box p-value
A	4.2	0.31
B	3.8	0.002

Model B has lower test RMSE, but its residuals exhibit strong remaining autocorrelation. Model A has slightly larger prediction error but more adequately captures the time dependence. The final choice should depend on whether prediction accuracy or structural adequacy is the primary objective.

For rolling-origin evaluation, the model is repeatedly fitted using data up to time t and tested on later observations. This procedure respects the information available at each forecast origin.

7.11 MATLAB Implementation

The following code calculates a trailing moving-average forecast and simple exponential smoothing.

```
y = [100; 110; 105; 115; 120; 118];
n = numel(y);

% Three-period moving-average forecasts
windowLength = 3;
movingForecast = NaN(n,1);

for t = windowLength+1:n
    movingForecast(t) = mean( ...
        y(t-windowLength:t-1));
end

nextMovingForecast = mean( ...
    y(end-windowLength+1:end));

```

```
% Simple exponential smoothing
alpha = 0.3;
level = zeros(n,1);
level(1) = y(1);

oneStepForecast = NaN(n,1);

for t = 2:n
    oneStepForecast(t) = level(t-1);

    level(t) = alpha*y(t) ...
        + (1-alpha)*level(t-1);
end

nextSESForecast = level(end);

fprintf('Next moving-average forecast = %.4f\n', ...
    nextMovingForecast);
fprintf('Next SES forecast = %.4f\n', ...
    nextSESForecast);

figure;
plot(y,'ko-', ...
    'LineWidth',1.2);
hold on;
plot(movingForecast,'b--', ...
    'LineWidth',1.2);
plot(oneStepForecast,'r-.', ...
    'LineWidth',1.2);

legend('Observed', ...
    'Moving average','SES', ...
    'Location','best');
xlabel('Time');
ylabel('Value');
title('Smoothing Forecasts');
grid on;
```

Listing 7.1: Moving average and exponential smoothing

The following implementation calculates Holt's linear-trend forecasts.

```
y = [100; 110; 105; 115; 120; 128];
n = numel(y);

alpha = 0.4;
beta = 0.3;

level = zeros(n,1);
trend = zeros(n,1);

level(1) = y(1);
trend(1) = y(2)-y(1);
```

```
12
13 for t = 2:n
14     previousLevel = level(t-1);
15
16     level(t) = alpha*y(t) ...
17         + (1-alpha) ...
18         *(level(t-1)+trend(t-1));
19
20     trend(t) = beta ...
21         *(level(t)-previousLevel) ...
22         +(1-beta)*trend(t-1);
23 end
24
25 forecastHorizon = 5;
26 h = (1:forecastHorizon)';
27
28 forecast = level(end) ...
29     + h*trend(end);
30
31 disp('Holt forecasts:');
32 disp(forecast);
```

Listing 7.2: Holt linear-trend model

The following code illustrates additive Holt–Winters smoothing.

```
1  y = [80; 95; 110; 100; ...
2       88; 104; 120; 108; ...
3       96; 112; 130; 116];
4
5  m = 4;
6  n = numel(y);
7
8  alpha = 0.4;
9  beta = 0.2;
10 gamma = 0.3;
11
12 level = zeros(n,1);
13 trend = zeros(n,1);
14 season = zeros(n,1);
15
16 firstSeasonMean = mean(y(1:m));
17 secondSeasonMean = mean(y(m+1:2*m));
18
19 level(m) = firstSeasonMean;
20 trend(m) = ...
21     (secondSeasonMean-firstSeasonMean)/m;
22
23 season(1:m) = ...
24     y(1:m)-firstSeasonMean;
25
26 for t = m+1:n
27     previousLevel = level(t-1);
```

```
28
29      level(t) = alpha ...
30          *(y(t)-season(t-m)) ...
31          +(1-alpha) ...
32          *(level(t-1)+trend(t-1));
33
34      trend(t) = beta ...
35          *(level(t)-previousLevel) ...
36          +(1-beta)*trend(t-1);
37
38      season(t) = gamma ...
39          *(y(t)-level(t)) ...
40          +(1-gamma)*season(t-m);
41  end
42
43  forecastHorizon = 8;
44  forecast = zeros(forecastHorizon,1);
45
46  for h = 1:forecastHorizon
47      seasonalIndex = ...
48          n-m+mod(h-1,m)+1;
49
50      forecast(h) = level(n) ...
51          + h*trend(n) ...
52          + season(seasonalIndex);
53  end
54
55  disp('Holt--Winters forecasts:');
56  disp(forecast);
```

Listing 7.3: Additive Holt–Winters model

The ACF, PACF, and stationarity tests can be used to inspect a series before fitting ARIMA models.

```
1  y = y(:);
2
3  figure;
4  autocorr(y);
5  title('Sample ACF');
6
7  figure;
8  parcorr(y);
9  title('Sample PACF');
10
11 % Augmented Dickey--Fuller test
12 % h = 1 means that the unit-root null is rejected
13 [hADF,pADF] = adftest(y);
14
15 fprintf('ADF decision = %d\n', hADF);
16 fprintf('ADF p-value = %.6f\n', pADF);
17
18 % First difference
```

```
differenceY = diff(y);

figure;
autocorr(differenceY);
title('ACF of First Differences');
```

Listing 7.4: ACF, PACF, and stationarity inspection

MATLAB's `arima`, `estimate`, and `forecast` functions can be used to fit ARIMA models.

```
y = [100; 108; 115; 123; 129; ...
     138; 145; 151; 160; 168];

% ARIMA(1,1,1) specification
model = arima( ...
    'ARLags',1, ...
    'D',1, ...
    'MALags',1);

estimatedModel = estimate( ...
    model,y, ...
    'Display','off');

forecastHorizon = 5;

[yForecast,yMSE] = forecast( ...
    estimatedModel, ...
    forecastHorizon, ...
    'Y0',y);

standardError = sqrt(yMSE);

lowerBound = yForecast ...
    - 1.96*standardError;
upperBound = yForecast ...
    + 1.96*standardError;

disp('Forecasts:');
disp(yForecast);

figure;
timeObserved = 1:numel(y);
timeForecast = ...
    numel(y)+(1:forecastHorizon);

plot(timeObserved,y,'k-o', ...
    'LineWidth',1.2);
hold on;

plot(timeForecast,yForecast,'b-o', ...
    'LineWidth',1.2);
plot(timeForecast,lowerBound,'r--');
plot(timeForecast,upperBound,'r--');
```

```
44
45 legend('Observed','Forecast', ...
46     '95% lower','95% upper', ...
47     'Location','best');
48
49 xlabel('Time');
50 ylabel('Value');
51 title('ARIMA Forecast');
52 grid on;
```

Listing 7.5: ARIMA estimation and forecasting

The fitted residuals should be checked for remaining autocorrelation.

```
1  residual = infer(estimatedModel,y);
2
3  figure;
4  subplot(2,1,1);
5  plot(residual,'k-');
6  yline(0,'r--');
7  title('Estimated Innovations');
8
9  subplot(2,1,2);
10 autocorr(residual);
11 title('Residual ACF');
12
13 % Ljung--Box test at several lags
14 lags = [5, 8];
15
16 [hLB,pLB] = lbqtest( ...
17     residual, 'Lags', lags);
18
19 disp('Ljung--Box decisions:');
20 disp(hLB);
21
22 disp('Ljung--Box p-values:');
23 disp(pLB);
```

Listing 7.6: ARIMA residual diagnostics

Remark 7.11.1. Time-series forecasting is not ordinary regression with time as one more predictor. Serial dependence, temporal ordering, trend, seasonality, and forecast-origin information must all be handled explicitly.

Chapter 8

Clustering and Classification Models

8.1 Unsupervised and Supervised Learning

Clustering and classification both divide observations into groups, but the availability of known labels distinguishes the two problems.

Clustering is an unsupervised-learning problem. Given observations

$$x_1, x_2, \ldots, x_n \in \mathbb{R}^p,$$

the objective is to discover groups whose members are similar to one another and different from members of other groups. The true group labels are not provided in advance.

Classification is a supervised-learning problem. Given labeled training observations

$$(x_1, y_1), \ldots, (x_n, y_n),$$

where y_i is a class label, the objective is to construct a rule that predicts the class of a new observation.

A complete clustering or classification analysis generally includes

1. selecting meaningful features;
2. treating missing values and abnormal observations;
3. encoding categorical variables;
4. standardizing features when scale matters;
5. selecting a distance, similarity, or predictive model;
6. fitting the model;
7. evaluating stability or predictive performance;
8. interpreting the resulting groups.

Example 8.1.1. Suppose that customer data contain annual spending, visit frequency, and average purchase value. If customer categories are not known, clustering may be used to discover customer segments. If the customers have already been labeled as high-risk or low-risk, classification may be used to predict the risk label of a new customer.

Remark 8.1.2. Cluster labels such as $1, 2, 3$ have no intrinsic ordering. Renaming the clusters does not change the clustering result.

8.2 Distances and Similarities

A distance function measures dissimilarity between observations. The choice of distance determines the geometric structure that a clustering or classification method detects.

For $x, y \in \mathbb{R}^p$, the Manhattan distance is

$$d_1(x, y) = \sum_{j=1}^{p} |x_j - y_j| .$$

The Euclidean distance is

$$d_2(x, y) = \sqrt{\sum_{j=1}^{p} (x_j - y_j)^2}.$$

More generally, the Minkowski distance of order $q \geq 1$ is

$$d_q(x, y) = \left(\sum_{j=1}^{p} |x_j - y_j|^q \right)^{1/q}.$$

The Chebyshev distance is

$$d_\infty(x, y) = \max_{1 \leq j \leq p} |x_j - y_j| .$$

Example 8.2.1. Let

$$x = (1, 2, 4), \qquad y = (3, 1, 2).$$

Then

$$d_1(x, y) = 2 + 1 + 2 = 5,$$
$$d_2(x, y) = \sqrt{2^2 + 1^2 + 2^2} = 3,$$

and

$$d_\infty(x, y) = 2.$$

The numerical value of the separation depends on the selected geometry.

Cosine similarity. The cosine similarity is

$$\cos(x, y) = \frac{x^\mathsf{T} y}{\|x\|_2 \, \|y\|_2}.$$

It measures the angle between two vectors and is insensitive to multiplying either vector by a positive constant.
For the preceding example,

$$x^\mathsf{T} y = 1(3) + 2(1) + 4(2) = 13,$$

so

$$\cos(x, y) = \frac{13}{\sqrt{21}\sqrt{14}} \approx 0.758.$$

Correlation distance. If $\rho(x, y)$ is the Pearson correlation between the coordinates of x and y, a correlation-based dissimilarity is

$$d_{\text{corr}}(x, y) = 1 - \rho(x, y).$$

It emphasizes similarity of shape after centering rather than absolute magnitude.
When variables have different units, Euclidean distance should normally be calculated after standardization:

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}.$$

Remark 8.2.2. A feature measured on a large numerical scale can dominate Euclidean distance. Standardization is not merely cosmetic; it can change the entire clustering result.

8.3 Hierarchical Clustering

Hierarchical clustering constructs a nested sequence of partitions. In agglomerative hierarchical clustering, every observation begins as a singleton cluster, and the two nearest clusters are repeatedly merged.
Let G_p and G_q be two clusters.
Single linkage uses the minimum pairwise distance:

$$D_{\text{single}}(G_p, G_q) = \min_{\substack{x_i \in G_p \\ x_j \in G_q}} d(x_i, x_j).$$

Complete linkage uses the maximum pairwise distance:

$$D_{\text{complete}}(G_p, G_q) = \max_{\substack{x_i \in G_p \\ x_j \in G_q}} d(x_i, x_j).$$

Average linkage uses

$$D_{\text{average}}(G_p, G_q) = \frac{1}{|G_p|\,|G_q|} \sum_{x_i \in G_p} \sum_{x_j \in G_q} d(x_i, x_j).$$

Centroid linkage uses the distance between cluster centers:

$$D_{\text{centroid}}(G_p, G_q) = d(\bar{x}_p, \bar{x}_q),$$

where

$$\bar{x}_p = \frac{1}{|G_p|} \sum_{x_i \in G_p} x_i.$$

Ward's method merges the pair of clusters producing the smallest increase in the within-cluster sum of squares.

Example 8.3.1. Consider four one-dimensional observations:

$$1, \quad 2, \quad 8, \quad 10.$$

Using single linkage, the closest pair is $\{1, 2\}$, with distance 1. The next closest pair is $\{8, 10\}$, with distance 2. The distance between the resulting clusters is

$$D_{\text{single}}(\{1, 2\}, \{8, 10\}) = \min\{7, 9, 6, 8\} = 6.$$

Therefore, cutting the dendrogram at any height between 2 and 6 produces the two clusters

$$\{1, 2\} \qquad \text{and} \qquad \{8, 10\}.$$

Single linkage can create long chain-shaped clusters. Complete linkage tends to produce more compact clusters, while Ward's method is closely connected to the squared Euclidean objective used by K-means.

8.4 K-Means Clustering

K-means divides n observations into K clusters by minimizing the within-cluster sum of squares. Let $C_1, \dots, C_K$ be the clusters and μ_k their centers. The objective is

$$J = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|_2^2 .$$

The algorithm alternates between two steps:

1. assign each observation to its nearest center;
2. replace each center by the mean of its assigned observations.

Each iteration does not increase J, so the algorithm eventually converges. However, it may converge to a local rather than global minimum.

Example 8.4.1. Consider

$$x_1 = (1,1), \quad x_2 = (1,2), \quad x_3 = (5,5), \quad x_4 = (6,5),$$

and let $K = 2$. Initialize the centers as

$$\mu_1^{(0)} = (1,1), \qquad \mu_2^{(0)} = (5,5).$$

The assignment step gives

$$C_1 = \{x_1, x_2\}, \qquad C_2 = \{x_3, x_4\}.$$

The updated centers are

$$\mu_1 = \left(1, \frac{3}{2}\right), \qquad \mu_2 = \left(\frac{11}{2}, 5\right).$$

The within-cluster sum of squares is

$$\begin{aligned} J &= \|x_1 - \mu_1\|_2^2 + \|x_2 - \mu_1\|_2^2 \\ &\quad + \|x_3 - \mu_2\|_2^2 + \|x_4 - \mu_2\|_2^2 \\ &= \frac{1}{4} + \frac{1}{4} + \frac{1}{4} + \frac{1}{4} = 1. \end{aligned}$$

The next assignment step leaves the clusters unchanged, so the algorithm has converged.

K-means is most suitable when clusters are approximately compact, spherical, and of comparable scale. It is sensitive to

- initialization;
- extreme observations;
- the selected number of clusters;
- variables measured on different scales;
- clusters with irregular shapes or very different densities.

8.5 K-Means++ and the Number of Clusters

K-means++ improves initialization by spreading the initial centers throughout the data. The procedure is

1. choose the first center uniformly from the observations;
2. for every observation x_i, calculate the squared distance D_i^2 to its nearest selected center;
3. choose the next center with probability
$$\mathbb{P}(x_i \text{ is selected}) = \frac{D_i^2}{\sum_j D_j^2};$$
4. repeat until K centers have been selected;
5. run the ordinary K-means iterations.

Example 8.5.1. Suppose that the one-dimensional observations are
$$0, \quad 1, \quad 4, \quad 5,$$
and the first center selected is 0. The squared distances to this center are
$$0, \quad 1, \quad 16, \quad 25.$$
Therefore, the probabilities of selecting the remaining observations as the second center are
$$\frac{1}{42}, \qquad \frac{16}{42}, \qquad \frac{25}{42}.$$
The point 5 is most likely to be selected because it is farthest from the first center.

Elbow method. Let J_K be the K-means objective for K clusters. Since J_K decreases as K increases, one may select a value near the point at which the rate of improvement changes substantially.

Silhouette coefficient. For observation i, let $a(i)$ be its average distance to observations in the same cluster, and let $b(i)$ be the smallest average distance to observations in another cluster. Its silhouette value is
$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}.$$
A value near 1 indicates a well-separated observation, a value near 0 indicates an observation near a cluster boundary, and a negative value suggests possible misclassification.

Example 8.5.2. If one observation has
$$a(i) = 1, \qquad b(i) = 4,$$
then
$$s(i) = \frac{4 - 1}{4} = 0.75.$$
This observation is substantially closer to its own cluster than to the nearest competing cluster.

8.6 DBSCAN Density-Based Clustering

DBSCAN identifies clusters as connected regions of high point density. It requires two parameters:

- a neighborhood radius $\varepsilon > 0$;
- a minimum number of points MinPts.

The ε-neighborhood of x is

$$N_\varepsilon(x) = \{y : d(x, y) \le \varepsilon\}.$$

A point is a core point if

$$|N_\varepsilon(x)| \ge \text{MinPts}\,.$$

A noncore point within the neighborhood of a core point is a border point. A point that is neither a core point nor reachable from a core point is classified as noise.
DBSCAN can identify irregularly shaped clusters and does not require the number of clusters in advance.

Example 8.6.1. Consider

$$A = (0, 0), \quad B = (0.5, 0), \quad C = (0, 0.5),$$

$$D = (0.5, 0.5), \quad E = (5, 5).$$

Let $\varepsilon = 0.8$ and MinPts $= 3$, counting the point itself. Every one of A, B, C, D has at least three points in its ε-neighborhood, so these four points form one density-connected cluster. The point E has no nearby observations and is classified as noise.

DBSCAN may perform poorly when different clusters have substantially different densities. Its result is also sensitive to the scale of the features and the choice of ε.

8.7 K-Nearest-Neighbor Classification

The K-nearest-neighbor method, abbreviated as KNN, classifies a new observation using the labels of the K closest training observations.
For a query point x, let $N_K(x)$ be the set of its K nearest neighbors. The predicted class is

$$\widehat{y}(x) = \arg\max_c \sum_{i \in N_K(x)} \mathbf{1}_{\{y_i = c\}}.$$

A distance-weighted version gives larger influence to nearby observations:

$$\widehat{y}(x) = \arg\max_c \sum_{i \in N_K(x)} \frac{\mathbf{1}_{\{y_i = c\}}}{d(x, x_i) + \varepsilon}.$$

Example 8.7.1. Suppose that the labeled training observations are

Point	Class
$(1, 1)$	A
$(2, 1)$	A
$(4, 3)$	B
$(5, 4)$	B
$(4, 5)$	B

For the query point $x = (3, 2)$, the three nearest observations are

$$(2, 1), \qquad (4, 3), \qquad (1, 1).$$

Their labels are A, B, A. Therefore, KNN with $K = 3$ predicts class A.

A small K produces a flexible but potentially unstable classifier. A large K produces smoother boundaries but may ignore local structure. The value of K should be selected using validation data or cross-validation.

8.8 Logistic and Softmax Regression

Logistic regression models the probability of a binary outcome $Y \in \{0, 1\}$. For features $x = (x_1, \ldots, x_p)^\mathsf{T}$, define

$$\eta = \beta_0 + \beta^\mathsf{T} x.$$

The class probability is

$$p(x) = \mathbb{P}(Y = 1 \mid x) = \frac{1}{1 + e^{-\eta}}.$$

The log-odds are linear:

$$\log\left(\frac{p(x)}{1 - p(x)}\right) = \beta_0 + \beta^\mathsf{T} x.$$

For a sample (x_i, y_i), the negative log-likelihood is

$$\mathcal{L}(\beta) = -\sum_{i=1}^{n} \left[y_i \log p_i + (1 - y_i) \log(1 - p_i)\right].$$

The parameter estimates minimize $\mathcal{L}$, possibly with an additional regularization penalty. Using threshold q, the classification rule is

$$\widehat{y} = \begin{cases} 1, & p(x) \geq q, \\ 0, & p(x) < q. \end{cases}$$

The threshold $q = 0.5$ is common but is not always appropriate.

Example 8.8.1. Suppose that a one-variable Logistic model is

$$\log\left(\frac{p}{1 - p}\right) = -4 + 0.8x.$$

For $x = 6$,

$$\eta = -4 + 0.8(6) = 0.8.$$

Therefore,

$$p = \frac{1}{1 + e^{-0.8}} \approx 0.690.$$

Using threshold 0.5, the predicted class is 1.

For K classes, Softmax regression uses

$$\mathbb{P}(Y = k \mid x) = \frac{\exp(\beta_k^\mathsf{T} x)}{\sum_{\ell=1}^{K} \exp(\beta_\ell^\mathsf{T} x)}.$$

The probabilities sum to 1, and the class with the largest probability is usually selected.

8.9 Decision Trees

A classification tree recursively divides the feature space into regions. At each node, a split is selected to produce child nodes that are more homogeneous than the parent node. For a node containing class proportions $p_1, \dots, p_K$, the Gini impurity is

$$G = 1 - \sum_{k=1}^{K} p_k^2.$$

The entropy impurity is

$$H = -\sum_{k=1}^{K} p_k \log p_k.$$

A split is chosen to maximize the reduction in weighted impurity.

Example 8.9.1. Suppose that a node contains 10 observations, of which 6 are positive and 4 are negative. Its Gini impurity is

$$G_{\text{parent}} = 1 - \left(\frac{6}{10}\right)^2 - \left(\frac{4}{10}\right)^2 = 0.48.$$

A candidate split produces a left node containing four positive observations and a right node containing two positive and four negative observations. The left-node impurity is 0, while the right-node impurity is

$$G_{\text{right}} = 1 - \left(\frac{2}{6}\right)^2 - \left(\frac{4}{6}\right)^2 = \frac{4}{9}.$$

The weighted impurity after the split is

$$\frac{4}{10}(0) + \frac{6}{10}\left(\frac{4}{9}\right) \approx 0.2667.$$

Thus, the Gini reduction is

$$0.48 - 0.2667 = 0.2133.$$

A fully grown tree may overfit the training data. Common controls include maximum depth, minimum leaf size, pruning, and cross-validation.
Decision trees are interpretable and can model nonlinear interactions, but small changes in the training sample may produce a substantially different tree.

8.10 Support-Vector Machines

A support-vector machine, abbreviated as SVM, constructs a separating hyperplane

$$w^{\mathsf{T}} x + b = 0$$

with a large margin between the classes.
For labels $y_i \in \{-1, 1\}$, the soft-margin SVM solves

$$\begin{aligned} \min_{w,b,\xi} \quad & \frac{1}{2}\|w\|_2^2 + C\sum_{i=1}^{n} \xi_i \\ \text{subject to} \quad & y_i(w^{\mathsf{T}} x_i + b) \geq 1 - \xi_i, \qquad i = 1, \dots, n, \\ & \xi_i \geq 0. \end{aligned}$$

The parameter C controls the tradeoff between a large margin and training errors.

Example 8.10.1. Suppose that the one-dimensional observations $-2, -1$ belong to class -1, while $1, 2$ belong to class 1. The maximum-margin decision boundary is $x = 0$. One canonical representation is

$$w = 1, \qquad b = 0.$$

The two closest observations, -1 and 1, satisfy

$$y_i(wx_i + b) = 1$$

and are the support vectors. The distance between the two margin boundaries is

$$\frac{2}{\|w\|_2} = 2.$$

The kernel method replaces inner products by a kernel

$$K(x, z) = \langle \phi(x), \phi(z) \rangle,$$

allowing nonlinear decision boundaries without explicitly constructing the high-dimensional feature map ϕ. Common kernels include linear, polynomial, and Gaussian radial-basis kernels.

8.11 Classification Performance Measures

For binary classification, let the positive class be 1. The confusion matrix contains

	Predicted positive	Predicted negative
Actual positive	TP	FN
Actual negative	FP	TN

The accuracy is

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}.$$

Precision is

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}},$$

and recall, also called sensitivity, is

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$

Specificity is

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}.$$

The F_1-score is the harmonic mean of precision and recall:

$$F_1 = \frac{2\,\text{TP}}{2\,\text{TP} + \text{FP} + \text{FN}}.$$

Example 8.11.1. Suppose that

$$\text{TP} = 40, \qquad \text{FP} = 10,$$

$$\text{TN} = 45, \qquad \text{FN} = 5.$$

Then

$$\text{Accuracy} = \frac{40 + 45}{100} = 0.85,$$

$$\text{Precision} = \frac{40}{50} = 0.80,$$

$$\text{Recall} = \frac{40}{45} \approx 0.889,$$

$$\text{Specificity} = \frac{45}{55} \approx 0.818,$$

and

$$F_1 = \frac{80}{80 + 10 + 5} \approx 0.842.$$

The receiver operating characteristic curve plots the true-positive rate against the false-positive rate as the classification threshold varies. The area under the curve, abbreviated as AUC, summarizes ranking performance across thresholds.
For imbalanced data, accuracy may be misleading. If 99% of observations belong to one class, always predicting that class gives 99% accuracy but does not identify any minority-class observations.

8.12 Validation and Model Selection

Supervised-learning performance must be evaluated on observations not used to fit the model. A standard procedure divides the data into training, validation, and test sets.
Cross-validation may be used to select

- the number of neighbors in KNN;
- tree depth and minimum leaf size;
- SVM penalty and kernel parameters;
- Logistic-regression regularization;
- classification thresholds.

All preprocessing operations that estimate quantities from data must be fitted inside the training set. These operations include standardization, missing value imputation, variable selection, and dimension reduction. Applying them to the complete data before cross-validation produces information leakage.
For clustering, no true labels may be available. Evaluation can then use

- silhouette coefficients;
- within-cluster dispersion;
- cluster stability under resampling;
- agreement across initialization seeds;
- practical interpretability of the clusters.

Example 8.12.1. Suppose that K-means with $K = 4$ has a slightly smaller within-cluster sum of squares than K-means with $K = 3$, but one of the four clusters contains only two unstable observations and disappears under small perturbations. The three-cluster solution may be preferable because it is more stable and interpretable.

8.13 MATLAB Implementation

The following code calculates several distance matrices.

```
X = [1, 2, 4;
     3, 1, 2;
     5, 4, 6];

euclideanDistance = pdist2( ...
    X, X, 'euclidean');

manhattanDistance = pdist2( ...
    X, X, 'cityblock');

chebyshevDistance = pdist2( ...
    X, X, 'chebychev');

cosineDistance = pdist2( ...
    X, X, 'cosine');

disp('Euclidean distances:');
disp(euclideanDistance);

disp('Cosine similarities:');
disp(1-cosineDistance);
```

Listing 8.1: Distance and similarity calculations

Hierarchical clustering can be performed using `linkage`.

```
X = [1; 2; 8; 10];

pairwiseDistance = pdist(X, ...
    'euclidean');

tree = linkage( ...
    pairwiseDistance, 'single');

figure;
dendrogram(tree);
title('Single-Linkage Dendrogram');

% Divide the observations into two clusters
clusterLabel = cluster( ...
    tree, 'MaxClust', 2);

disp(clusterLabel);
```

Listing 8.2: Hierarchical clustering

MATLAB's `kmeans` function supports K-means++ initialization through the `'plus'` option.

```
X = [1, 1;
     1, 2;
```

```
     5, 5;
     6, 5];

K = 2;

[label,center,sumDistance] = kmeans( ...
    X, K, ...
    'Start','plus', ...
    'Replicates',20);

disp('Cluster labels:');
disp(label);

disp('Cluster centers:');
disp(center);

fprintf('Within-cluster SSE = %.4f\n', ...
    sum(sumDistance));

figure;
gscatter(X(:,1),X(:,2),label);
hold on;
plot(center(:,1),center(:,2), ...
    'kx', ...
    'MarkerSize',12, ...
    'LineWidth',2);

title('K-Means Clustering');
grid on;

figure;
silhouette(X,label);
title('Silhouette Plot');
```

Listing 8.3: K-means and silhouette analysis

The number of clusters can be selected using several internal criteria.

```
evaluation = evalclusters( ...
    X, 'kmeans', 'silhouette', ...
    'KList', 2:4);

bestK = evaluation.OptimalK;

fprintf('Selected number of clusters = %d\n', ...
    bestK);
```

Listing 8.4: Selecting the number of clusters

DBSCAN is implemented by the `dbscan` function.

```
X = [0,   0;
     0.5, 0;
     0,   0.5;
```

```
     0.5, 0.5;
     5,   5];

epsilon = 0.8;
minimumPoints = 3;

label = dbscan( ...
    X, epsilon, minimumPoints);

disp('DBSCAN labels:');
disp(label);

% A label of -1 indicates noise
figure;
gscatter(X(:,1),X(:,2),label);
title('DBSCAN Clustering');
grid on;
```

Listing 8.5: DBSCAN clustering

KNN classification can be fitted using `fitcknn`.

```
XTrain = [1, 1;
          2, 1;
          4, 3;
          5, 4;
          4, 5];

YTrain = categorical( ...
    {'A';'A';'B';'B';'B'});

knnModel = fitcknn( ...
    XTrain, YTrain, ...
    'NumNeighbors',3, ...
    'Standardize',true);

queryPoint = [3, 2];

predictedClass = predict( ...
    knnModel, queryPoint);

disp('Predicted class:');
disp(predictedClass);
```

Listing 8.6: K-nearest-neighbor classification

Binary Logistic regression is implemented using `fitglm`.

```
x = [2; 3; 5; 7; 9; 10];
y = [0; 0; 0; 1; 1; 1];

T = table(x,y);

logisticModel = fitglm( ...
```

```
    T, 'y ~ x', ...
    'Distribution','binomial', ...
    'Link','logit');

newX = table(6, ...
    'VariableNames',{'x'});

predictedProbability = predict( ...
    logisticModel,newX);

predictedClass = ...
    predictedProbability >= 0.5;

fprintf('Predicted probability = %.4f\n', ...
    predictedProbability);
fprintf('Predicted class = %d\n', ...
    predictedClass);
```

Listing 8.7: Binary Logistic regression

Decision trees and support-vector machines can be fitted as follows.

```
X = [1, 1;
     2, 1;
     2, 2;
     4, 3;
     5, 4;
     4, 5];

Y = categorical( ...
    {'A';'A';'A';'B';'B';'B'});

treeModel = fitctree( ...
    X,Y, ...
    'MinLeafSize',1);

svmModel = fitcsvm( ...
    X,Y, ...
    'KernelFunction','linear', ...
    'Standardize',true);

queryPoint = [3,2];

treePrediction = predict( ...
    treeModel,queryPoint);

svmPrediction = predict( ...
    svmModel,queryPoint);

disp(treePrediction);
disp(svmPrediction);

figure;
```

```
view(treeModel,'Mode','graph');
```

Listing 8.8: Decision-tree and SVM classifiers

The following code calculates a confusion matrix and several performance measures.

```
actual = [ones(45,1); zeros(55,1)];

predicted = [ones(40,1);
             zeros(5,1);
             ones(10,1);
             zeros(45,1)];

confusion = confusionmat( ...
    actual,predicted, ...
    'Order',[1,0]);

TP = confusion(1,1);
FN = confusion(1,2);
FP = confusion(2,1);
TN = confusion(2,2);

accuracy = (TP+TN) ...
    /(TP+TN+FP+FN);

precision = TP/(TP+FP);
recall = TP/(TP+FN);
specificity = TN/(TN+FP);

F1 = 2*TP/(2*TP+FP+FN);

fprintf('Accuracy = %.4f\n', accuracy);
fprintf('Precision = %.4f\n', precision);
fprintf('Recall = %.4f\n', recall);
fprintf('Specificity = %.4f\n', specificity);
fprintf('F1 score = %.4f\n', F1);

figure;
confusionchart(actual,predicted);
title('Confusion Matrix');
```

Listing 8.9: Classification performance measures

Remark 8.13.1. Clustering discovers structure without known labels, while classification learns from labeled observations. Their numerical outputs may look similar, but their assumptions, evaluation methods, and interpretations are fundamentally different.

Chapter 9

Dimension Reduction and Multivariate Analysis

9.1 Multivariate Data and Covariance Structure

Suppose that n objects are observed through p numerical variables. The data matrix is

$$X = \begin{pmatrix} x_{11} & \cdots & x_{1p} \\ \vdots & \ddots & \vdots \\ x_{n1} & \cdots & x_{np} \end{pmatrix}.$$

The mean of variable j is

$$\bar{x}_j = \frac{1}{n}\sum_{i=1}^{n} x_{ij}.$$

Let $\bar{x} = (\bar{x}_1, \ldots, \bar{x}_p)^{\mathsf{T}}$. The centered data matrix is

$$X_c = X - \mathbf{1}_n \bar{x}^{\mathsf{T}}.$$

Its sample covariance matrix is

$$S = \frac{1}{n-1} X_c^{\mathsf{T}} X_c.$$

The diagonal entry s_{jj} is the sample variance of variable j, while the off-diagonal entry s_{jk} is the sample covariance between variables j and k.
The sample correlation matrix is

$$R = (r_{jk})_{p\times p}, \qquad r_{jk} = \frac{s_{jk}}{\sqrt{s_{jj}s_{kk}}}.$$

If variables have similar units and comparable scales, dimension reduction may be based on S. If their units or numerical scales differ substantially, the variables should usually be standardized and the analysis based on R.

Example 9.1.1. Suppose that two variables have covariance matrix

$$S = \begin{pmatrix} 100 & 4 \\ 4 & 1 \end{pmatrix}.$$

The first variable has variance 100, while the second has variance 1. A covariance-based analysis will be dominated by the first variable. The correlation coefficient is

$$r_{12} = \frac{4}{\sqrt{100(1)}} = 0.4.$$

Standardization removes the difference in units and gives the correlation matrix

$$R = \begin{pmatrix} 1 & 0.4 \\ 0.4 & 1 \end{pmatrix}.$$

Remark 9.1.2. Standardization is not automatically appropriate in every application. It assigns every variable unit variance and may therefore give a noisy or practically unimportant variable the same initial scale as an important one.

9.2 Principal Component Analysis

Principal component analysis, abbreviated as PCA, constructs uncorrelated linear combinations of the original variables that preserve as much variance as possible.
Assume that X has been centered. For a unit vector $a \in \mathbb{R}^p$, the linear combination

$$Z = X_c a$$

has sample variance

$$\mathrm{Var}(Z) = a^\mathsf{T} S a.$$

The first principal-component direction solves

$$\begin{aligned} \max_{a \in \mathbb{R}^p} \quad & a^\mathsf{T} S a \\ \text{subject to} \quad & a^\mathsf{T} a = 1. \end{aligned}$$

Using the Lagrangian

$$\mathcal{L}(a, \lambda) = a^\mathsf{T} S a - \lambda(a^\mathsf{T} a - 1),$$

the first-order condition is

$$Sa = \lambda a.$$

Therefore, the first principal-component direction is an eigenvector of S corresponding to its largest eigenvalue.
Let

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_p \geq 0$$

be the eigenvalues of S, with orthonormal eigenvectors $a_1, \ldots, a_p$. The j-th principal component is

$$Z_j = X_c a_j.$$

Its variance is

$$\mathrm{Var}(Z_j) = \lambda_j.$$

Different principal components are uncorrelated:

$$\mathrm{Cov}(Z_j, Z_k) = 0, \qquad j \neq k.$$

Theorem 9.2.1. Among all unit directions, a_1 maximizes the projected sample variance. Subject to being orthogonal to $a_1, \ldots, a_{k-1}$, the direction a_k maximizes the remaining projected variance.

Example 9.2.2. Suppose that two standardized variables have correlation matrix

$$R = \begin{pmatrix} 1 & 0.8 \\ 0.8 & 1 \end{pmatrix}.$$

Its eigenvalues are

$$\lambda_1 = 1.8, \qquad \lambda_2 = 0.2,$$

with eigenvectors

$$a_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \qquad a_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}.$$

Therefore,

$$Z_1 = \frac{X_1 + X_2}{\sqrt{2}}$$

measures their common movement, while

$$Z_2 = \frac{X_1 - X_2}{\sqrt{2}}$$

measures their difference. Because the variables are strongly positively correlated, most variation occurs in the common direction.

9.3 Selecting the Number of Principal Components

The total sample variance is

$$\operatorname{tr}(S) = \sum_{j=1}^{p} \lambda_j.$$

The explained-variance proportion of component j is

$$\eta_j = \frac{\lambda_j}{\sum_{\ell=1}^{p} \lambda_\ell}.$$

The cumulative explained-variance proportion of the first k components is

$$\Gamma_k = \frac{\sum_{j=1}^{k} \lambda_j}{\sum_{j=1}^{p} \lambda_j}.$$

Common component-selection methods include

- retaining enough components to reach a specified cumulative explained variance;
- locating an elbow in the scree plot;
- retaining correlation-matrix eigenvalues larger than 1;
- using parallel analysis or cross-validation;
- preserving enough components for practical interpretation.

Example 9.3.1. Suppose that the eigenvalues are

$$4, \quad 2, \quad 1, \quad 0.5, \quad 0.5.$$

Their sum is 8, so the explained-variance proportions are

$$0.50, \quad 0.25, \quad 0.125, \quad 0.0625, \quad 0.0625.$$

The cumulative proportions are

$$0.50, \quad 0.75, \quad 0.875, \quad 0.9375, \quad 1.$$

If at least 85% of the variance is required, the first three components should be retained.

The threshold should not be chosen mechanically. A component explaining a small proportion of total variance may still contain information important for classification or prediction.

9.4 Scores, Loadings, and Reconstruction

Let

$$A = (a_1, \ldots, a_p)$$

be the orthogonal eigenvector matrix. The principal-component score matrix is

$$T = X_c A.$$

The entry t_{ij} is the score of observation i on principal component j.
The loading of original variable j on principal component k is often defined as

$$\ell_{jk} = \mathrm{Corr}(X_j, Z_k) = \sqrt{\lambda_k}\, a_{jk}$$

when PCA is based on standardized variables. Large absolute loadings identify variables strongly associated with a component.
Using the first k components, the centered observations are approximated by

$$\widehat{X_c} = T_k A_k^{\mathsf{T}},$$

where

$$T_k = X_c A_k.$$

The reconstruction on the original scale is

$$\widehat{X} = \mathbf{1}_n \bar{x}^{\mathsf{T}} + T_k A_k^{\mathsf{T}}.$$

Example 9.4.1. For the two-variable PCA example, suppose that a standardized observation is

$$z = (1, 1).$$

Its first principal-component score is

$$t_1 = z^{\mathsf{T}} a_1 = \sqrt{2},$$

and its second score is 0. Reconstruction using only the first component gives

$$\hat{z} = \sqrt{2}\frac{1}{\sqrt{2}}(1, 1) = (1, 1),$$

so no information is lost for this observation.
For $z = (1, -1)$, the first score is 0. Reconstruction using only the first component gives $(0, 0)$, so the difference direction is completely discarded.

The squared reconstruction error from retaining k components is minimized by PCA among all k-dimensional linear subspaces.

9.5 Interpretation and Biplots

A principal component should be interpreted from the signs and magnitudes of its coefficients or loadings.
Suppose that

$$Z_1 = 0.52X_1 + 0.50X_2 + 0.49X_3 + 0.49X_4.$$

Because all coefficients are positive and similar, Z_1 may represent an overall-size or general-performance factor.

If

$$Z_2 = 0.70X_1 + 0.10X_2 - 0.20X_3 - 0.67X_4,$$

then Z_2 contrasts variables X_1, X_2 against X_3, X_4.
The signs of eigenvectors are arbitrary. Replacing a_j by $-a_j$ produces the same principal direction and explained variance. Interpretation should therefore focus on relative signs and magnitudes rather than on the absolute sign of a component.
A biplot displays observation scores and variable loading vectors in the same two-dimensional representation. It can reveal

- groups of nearby observations;
- observations with extreme component scores;
- variables contributing strongly to each component;
- positively or negatively associated variables.

Example 9.5.1. Suppose that income, consumption, and savings all have large positive loadings on the first component. Regions with large positive first-component scores have high overall economic scale. If pollution has a large positive loading and green-space coverage has a large negative loading on the second component, that component may represent an environmental-development contrast.

Remark 9.5.2. Assigning a name to a component is an interpretation, not a mathematical identity. The proposed meaning should be supported by several strong loadings and by knowledge of the application.

9.6 Factor Analysis

Factor analysis explains correlations among observed variables through a smaller number of unobserved latent factors. The common-factor model is

$$X - \mu = \Lambda F + \varepsilon,$$

where

- $F \in \mathbb{R}^k$ is the common-factor vector;
- $\Lambda \in \mathbb{R}^{p\times k}$ is the factor-loading matrix;
- $\varepsilon \in \mathbb{R}^p$ contains variable-specific effects.

In an orthogonal factor model,

$$\mathbb{E}[F] = 0, \qquad \mathrm{Cov}(F) = I,$$
$$\mathbb{E}[\varepsilon] = 0, \qquad \mathrm{Cov}(\varepsilon) = \Psi,$$

where Ψ is diagonal, and F is uncorrelated with ε. Therefore,

$$\mathrm{Cov}(X) = \Lambda\Lambda^{\mathsf{T}} + \Psi.$$

For standardized variable X_i, the communality is

$$h_i^2 = \sum_{j=1}^{k} \lambda_{ij}^2,$$

while the uniqueness is

$$\psi_i = 1 - h_i^2.$$

Example 9.6.1. Suppose that the estimated loading matrix is

$$\Lambda = \begin{pmatrix} 0.90 & 0.10 \\ 0.85 & 0.15 \\ 0.10 & 0.88 \\ 0.20 & 0.82 \end{pmatrix}.$$

The first two variables load strongly on factor 1, while the last two load strongly on factor 2. The communalities are

$$h_1^2 = 0.90^2 + 0.10^2 = 0.82,$$
$$h_2^2 = 0.85^2 + 0.15^2 = 0.745,$$
$$h_3^2 = 0.10^2 + 0.88^2 = 0.7844,$$

and

$$h_4^2 = 0.20^2 + 0.82^2 = 0.7124.$$

Thus, the two-factor model explains more than 70% of the variance of each standardized variable.

9.7 Factor Adequacy and Rotation

Factor analysis is useful only when the variables have sufficient shared correlation.
The Kaiser–Meyer–Olkin statistic compares ordinary correlations with partial correlations:

$$\mathrm{KMO} = \frac{\sum_{i \neq j} r_{ij}^2}{\sum_{i \neq j} r_{ij}^2 + \sum_{i \neq j} p_{ij}^2},$$

where p_{ij} is the partial correlation between variables i and j after controlling for the remaining variables. A larger KMO value indicates that the common-factor structure is more plausible.

Example 9.7.1. Suppose that

$$\sum_{i \neq j} r_{ij}^2 = 0.72$$

and

$$\sum_{i \neq j} p_{ij}^2 = 0.08.$$

Then

$$\mathrm{KMO} = \frac{0.72}{0.72 + 0.08} = 0.90.$$

The correlations are much larger than the corresponding partial correlations, supporting the use of a low-dimensional factor model.

Bartlett's test of sphericity tests whether the population correlation matrix is the identity matrix. Rejection of the identity hypothesis indicates that the variables contain nontrivial correlations, although it does not determine the correct number of factors.
Factor loadings are not unique. If Q is an orthogonal matrix, then

$$\Lambda F = (\Lambda Q)(Q^{\mathsf{T}} F).$$

Rotation uses this nonuniqueness to produce a simpler loading pattern.
Varimax is an orthogonal rotation that encourages each variable to have large loadings on a small number of factors. Oblique rotations allow the factors to be correlated.

Example 9.7.2. Suppose that an unrotated two-factor solution gives

$$\Lambda = \begin{pmatrix} 0.70 & 0.50 \\ 0.68 & 0.48 \\ 0.65 & -0.52 \\ 0.62 & -0.55 \end{pmatrix}.$$

Both factors are difficult to interpret because most variables load on both. After rotation, the loadings may become approximately

$$\Lambda_{\text{rot}} = \begin{pmatrix} 0.90 & 0.10 \\ 0.86 & 0.12 \\ 0.08 & 0.88 \\ 0.15 & 0.84 \end{pmatrix}.$$

The rotated factors are easier to interpret, while the reproduced covariance structure remains essentially unchanged.

9.8 PCA and Factor Analysis

PCA and factor analysis both reduce dimension, but their mathematical goals are different.

	PCA	**Factor analysis**
Main objective	Preserve total variance	Explain common covariance
Mathematical form	Components are linear combinations of observed variables	Variables are generated from latent factors
Error term	No explicit measurement-error model	Includes variable-specific errors
Diagonal covariance	Total variances are analyzed	Communalities and uniquenesses are separated
Typical use	Compression, visualization, prediction	Latent-construct interpretation

PCA is generally preferred when the main objective is data compression or construction of uncorrelated predictors. Factor analysis is preferred when the main objective is to interpret latent variables that generate the observed correlations.

Example 9.8.1. Suppose that mathematics, physics, chemistry, and biology examination scores are strongly correlated. PCA may construct a small number of components that preserve most variation in the four scores. Factor analysis instead attempts to explain the correlations through latent abilities, such as quantitative reasoning and scientific knowledge.

9.9 Principal Components in Evaluation and Clustering

PCA can be combined with comprehensive evaluation. Suppose that the first k principal components explain most of the total variance. A composite score may be constructed as

$$F_i = \sum_{j=1}^{k} \omega_j z_{ij},$$

where z_{ij} is the score of observation i on component j, and

$$\omega_j = \frac{\lambda_j}{\sum_{\ell=1}^{k} \lambda_\ell}.$$

This score is objective in the sense that its coefficients are determined by sample variance. However, a direction with high variance is not necessarily the most important direction for the practical evaluation problem.

Example 9.9.1. Suppose that two retained components have eigenvalues

$$\lambda_1 = 4, \qquad \lambda_2 = 2.$$

Their normalized weights are

$$\omega_1 = \frac{2}{3}, \qquad \omega_2 = \frac{1}{3}.$$

If an observation has component scores $z_{i1} = 1.5$ and $z_{i2} = -0.3$, then its composite score is

$$F_i = \frac{2}{3}(1.5) + \frac{1}{3}(-0.3) = 0.9.$$

PCA may also be performed before K-means or hierarchical clustering. This can

- reduce noise and redundant dimensions;
- decrease computational cost;
- mitigate strong correlations among features;
- make two-dimensional visualization possible.

The number of retained components should be selected before examining the final cluster interpretation to reduce the risk of choosing a representation only because it produces visually attractive clusters.

9.10 Principal Component Regression

Principal component regression, abbreviated as PCR, first applies PCA to the predictors and then regresses the response on a selected number of principal components.
Let the standardized predictor matrix have the decomposition

$$X = TA^{\mathsf{T}}.$$

Using the first k components, PCR fits

$$y = \alpha \mathbf{1} + T_k \gamma + \varepsilon.$$

The coefficient vector on the original standardized predictors is

$$\beta_{\mathrm{PCR}} = A_k \gamma.$$

PCR is useful when predictors are highly correlated or when p is large. However, PCA ignores the response variable when choosing components. A low-variance direction may still be strongly predictive of y.

Example 9.10.1. Suppose that two standardized predictors have correlation 0.99. Ordinary least squares may produce large coefficients with opposite signs because the individual effects are difficult to separate. The first principal component, approximately

$$Z_1 = \frac{X_1 + X_2}{\sqrt{2}},$$

captures their common variation. Regressing y on Z_1 gives a stable model of their combined predictive effect, although it no longer attempts to estimate the separate effects of X_1 and X_2.

The number of PCR components should be selected through cross-validation rather than only through explained predictor variance.

9.11 Validation and Common Pitfalls

A dimension-reduction model should be checked for stability and practical interpretability. Important validation steps include

- checking whether variables require standardization;
- examining missing values and abnormal observations;
- comparing covariance-based and correlation-based PCA;
- inspecting scree plots and cumulative explained variance;
- checking loading stability under resampling;
- evaluating reconstruction or prediction error;
- fitting preprocessing only on the training data;
- verifying that retained components are practically interpretable.

Common errors include

- interpreting explained variance as practical importance;
- using PCA on variables with incompatible meanings without justification;
- treating the sign of an eigenvector as uniquely determined;
- interpreting PCA components as latent causal factors;
- performing PCA on the complete data before cross-validation;
- retaining components only because their eigenvalues exceed 1;
- using two-dimensional plots as proof that clusters truly exist.

Example 9.11.1. Suppose that PCA is performed on the complete dataset before it is divided into training and test samples. The principal directions then depend on the test observations, so information from the test set has entered the fitted procedure. The reported test error is optimistic. PCA must instead be fitted using the training sample and then applied unchanged to the test sample.

9.12 MATLAB Implementation

MATLAB's pca function returns principal directions, scores, eigenvalues, explained-variance percentages, and variable means.

```
X = [70, 72, 68, 75;
     80, 78, 82, 79;
     65, 67, 64, 69;
     90, 88, 91, 87;
     76, 74, 77, 73];

% Standardize the variables
Z = normalize(X,'zscore');

[coefficient,score,latent, ...
    hotellingT2,explained,mu] = pca(Z);

cumulativeExplained = cumsum(explained);

disp('Eigenvalues:');
disp(latent);

disp('Explained variance percentages:');
disp(explained);

disp('Cumulative explained variance:');
disp(cumulativeExplained);

% Select the smallest number reaching 85 percent
numberOfComponents = find( ...
    cumulativeExplained >= 85, ...
    1, 'first');

fprintf('Selected components = %d\n', ...
    numberOfComponents);

figure;
pareto(explained);
xlabel('Principal Component');
ylabel('Explained Variance (%)');
title('PCA Scree Plot');
```

Listing 9.1: Principal component analysis

The original standardized observations can be reconstructed from the retained components.

```
k = numberOfComponents;

scoreReduced = score(:,1:k);
coefficientReduced = coefficient(:,1:k);

ZReconstructed = ...
    scoreReduced*coefficientReduced' + mu;
```

```
8
9  reconstructionError = ...
10     norm(Z-ZReconstructed,'fro')^2;
11
12 fprintf('Squared reconstruction error = %.6f\n', ...
13     reconstructionError);
```

Listing 9.2: PCA reconstruction

A biplot displays the first two component directions and observation scores.

```
1  variableNames = { ...
2      'Variable 1', ...
3      'Variable 2', ...
4      'Variable 3', ...
5      'Variable 4'};
6
7  figure;
8  biplot( ...
9      coefficient(:,1:2), ...
10     'Scores',score(:,1:2), ...
11     'VarLabels',variableNames);
12
13 title('PCA Biplot');
```

Listing 9.3: PCA biplot

The following code calculates the KMO statistic.

```
1  R = corrcoef(Z);
2  inverseR = pinv(R);
3
4  scaleMatrix = sqrt( ...
5      diag(inverseR)*diag(inverseR)');
6
7  partialCorrelation = ...
8      -inverseR./scaleMatrix;
9
10 p = size(R,1);
11
12 offDiagonalMask = ...
13     ~eye(p);
14
15 correlationSum = sum( ...
16     R(offDiagonalMask).^2);
17
18 partialSum = sum( ...
19     partialCorrelation( ...
20         offDiagonalMask).^2);
21
22 KMO = correlationSum ...
23     /(correlationSum+partialSum);
24
25 fprintf('KMO statistic = %.4f\n', KMO);
```

Listing 9.4: Kaiser–Meyer–Olkin statistic

Factor analysis with Varimax rotation can be performed using `factoran`.

```
numberOfFactors = 2;

[loading,uniqueness,rotation, ...
    statistics,factorScore] = factoran( ...
        Z, numberOfFactors, ...
        'rotate','varimax', ...
        'scores','regression');

disp('Rotated factor loadings:');
disp(loading);

disp('Variable uniquenesses:');
disp(uniqueness);

communality = 1-uniqueness;

disp('Variable communalities:');
disp(communality);

figure;
scatter( ...
    factorScore(:,1), ...
    factorScore(:,2), ...
    60,'filled');

xlabel('Factor 1 Score');
ylabel('Factor 2 Score');
title('Factor Scores');
grid on;
```

Listing 9.5: Exploratory factor analysis

PCA scores can be used as inputs to K-means clustering.

```
Z = normalize(X,'zscore');

[coefficient,score,latent, ...
    ~,explained] = pca(Z);

cumulativeExplained = cumsum(explained);

kPCA = find( ...
    cumulativeExplained >= 85, ...
    1, 'first');

reducedData = score(:,1:kPCA);

numberOfClusters = 2;

```

```
clusterLabel = kmeans( ...
    reducedData, ...
    numberOfClusters, ...
    'Start','plus', ...
    'Replicates',50);

if kPCA >= 2
    figure;
    gscatter( ...
        reducedData(:,1), ...
        reducedData(:,2), ...
        clusterLabel);

    xlabel('Principal Component 1');
    ylabel('Principal Component 2');
    title('Clustering in PCA Space');
    grid on;
end
```

Listing 9.6: PCA followed by K-means clustering

The following code implements principal component regression.

```
X = [1.0, 1.1, 3.0;
     2.0, 2.1, 2.0;
     3.0, 3.2, 5.0;
     4.0, 4.1, 4.0;
     5.0, 5.2, 6.0;
     6.0, 6.1, 7.0];

y = [2.1; 3.8; 6.2; 7.9; 10.1; 12.2];

% Standardize predictors
predictorMean = mean(X,1);
predictorStd = std(X,0,1);

Z = (X-predictorMean) ...
    ./predictorStd;

[coefficient,score,~,~,explained] = pca(Z);

% Number of components should ideally
% be selected by cross-validation
k = 2;

responseMean = mean(y);
centeredResponse = y-responseMean;

gamma = score(:,1:k) ...
    \\ centeredResponse;

betaStandardized = ...
    coefficient(:,1:k)*gamma;
```

```
31
32 fittedValue = responseMean ...
33     + Z*betaStandardized;
34
35 RMSE = sqrt(mean( ...
36     (y-fittedValue).^2));
37
38 fprintf('Training RMSE = %.6f\n', RMSE);
39
40 disp('PCR coefficients on standardized predictors:');
41 disp(betaStandardized);
```

Listing 9.7: Principal component regression

Remark 9.12.1. PCA and factor analysis can summarize a high-dimensional dataset, but they do not automatically identify the scientifically correct variables or latent mechanisms. Mathematical compression and practical explanation remain different objectives.

Chapter 10

Model Validation, Selection, and Mathematical Writing

10.1 The Complete Modeling Process

A mathematical model is not complete when an equation has been written or a numerical result has been produced. A complete modeling process consists of the following stages:

1. clarify the practical objective;
2. identify the system boundary;
3. introduce variables, parameters, and units;
4. state and justify simplifying assumptions;
5. construct the mathematical model;
6. estimate unknown parameters;
7. solve or simulate the model;
8. verify the mathematical and numerical implementation;
9. validate the model against data or known behavior;
10. perform sensitivity and uncertainty analysis;
11. compare the model with reasonable alternatives;
12. interpret the conclusions and state the limitations.

These stages are iterative. If validation fails, one may need to revise the assumptions, variables, parameter estimates, or even the entire model structure.
A useful distinction is

Concept	**Question**
Verification	Was the mathematical model implemented and solved correctly?
Validation	Does the model represent the real system accurately enough for its intended purpose?
Calibration	Which parameter values make the model agree with observations?
Sensitivity	How strongly do conclusions change when inputs change?
Uncertainty	How uncertain are the predictions or decisions?

Example 10.1.1. Suppose that an epidemic model is implemented in MATLAB. Comparing the numerical solution with a known analytical special case is verification. Comparing the simulated infection curve with observed case data is validation. Estimating the transmission rate from those observations is calibration. Changing the transmission rate by 10% is sensitivity analysis. Assigning a probability distribution to the transmission rate and computing a prediction interval is uncertainty analysis.

10.2 Assumptions and Model Boundaries

Every model depends on assumptions. An assumption should simplify the problem while preserving the mechanism relevant to the objective.
A useful assumption should be

- stated explicitly;
- connected to a modeling decision;
- approximately compatible with the available evidence;
- testable or discussable when possible;
- accompanied by a limitation if it is strong.

Assumptions may concern

- the population or spatial region under study;
- the time horizon;
- independence or stationarity;
- linearity or smoothness;
- homogeneous mixing;
- constant parameters;
- complete or accurate observations;
- absence of external intervention.

Example 10.2.1. A transportation model may assume that travel time on each road is constant. This assumption is reasonable for planning during a stable low-traffic period, but it may fail during rush hour because travel time depends on congestion. If congestion is central to the problem, travel time should depend on traffic flow rather than being treated as a fixed edge weight.

A strong assumption is not automatically unacceptable. The relevant question is whether it materially affects the conclusions for the intended use of the model.

10.3 Dimensional Consistency and Sanity Checks

Dimensional analysis is one of the fastest ways to identify an incorrect model. Both sides of an equation must have the same physical units.
For the growth model

$$\frac{dN}{dt} = rN,$$

if N is measured in individuals and t in years, then r must have units year^{-1}.
Dimensionless quantities are especially useful. If a population has carrying capacity K, define

$$u = \frac{N}{K}.$$

If time is rescaled by $\tau = rt$, the Logistic equation becomes

$$\frac{du}{d\tau} = u(1-u).$$

The dimensionless form separates the qualitative structure from the original units.
Sanity checks should include

- checking signs and units;

- checking whether probabilities remain in $[0, 1]$;
- checking whether populations and flows remain nonnegative;
- checking limiting cases;
- checking conservation laws;
- comparing with simple upper and lower bounds;
- testing extreme parameter values.

Example 10.3.1. Suppose that a production model recommends $x_1 = -4$ units of a product. Even if the solver reports convergence, the result is physically meaningless. The model is missing the constraint $x_1 \geq 0$.
Similarly, if a probability model returns $p = 1.08$, either the formula, implementation, or interpretation is incorrect.

For a complicated model, a simplified limiting case can be particularly useful. For example, setting the interaction coefficient to zero in a predator–prey model should recover independent single-population equations.

10.4 Parameter Calibration and Identifiability

Let y_i be observations and let

$$\widehat{y}_i(\theta)$$

be model predictions depending on parameters θ. A common calibration method is nonlinear least squares:

$$\widehat{\theta} = \arg\min_{\theta} \sum_{i=1}^{n} \Big(y_i - \widehat{y}_i(\theta)\Big)^2.$$

For observations with different variances, weighted least squares may be more appropriate:

$$\widehat{\theta} = \arg\min_{\theta} \sum_{i=1}^{n} w_i \Big(y_i - \widehat{y}_i(\theta)\Big)^2.$$

A parameter is structurally identifiable if perfect continuous observations would determine it uniquely. It is practically identifiable if the available noisy and finite data determine it with acceptable precision.

Example 10.4.1. Consider

$$y_i = \theta_1 + \theta_2 x_i + \varepsilon_i.$$

If every observation has $x_i = 1$, then

$$y_i = \theta_1 + \theta_2 + \varepsilon_i.$$

Only the sum $\theta_1 + \theta_2$ can be estimated. For any constant c, the parameter pairs

$$(\theta_1, \theta_2)$$

and

$$(\theta_1 + c, \theta_2 - c)$$

produce identical predictions. Thus, the two parameters are not separately identifiable.

Warning signs of poor identifiability include

- very large standard errors;
- strong parameter correlations;
- different initial guesses producing different estimates;
- a nearly flat objective function;
- good fitted curves but unstable parameter values.

More complicated models generally require richer data. Adding parameters without corresponding information may improve in-sample fit while destroying interpretability.

10.5 Residual Analysis and Goodness of Fit

For observations y_i and fitted values $\widehat{y}_i$, define the residuals by

$$e_i = y_i - \widehat{y}_i.$$

A useful model should leave residuals without strong systematic structure.
Common error measures include

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n} |e_i|,$$

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n} e_i^2,$$

and

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n} e_i^2}.$$

For nonzero observations, the mean absolute percentage error is

$$\mathrm{MAPE} = \frac{100\%}{n}\sum_{i=1}^{n} \left|\frac{e_i}{y_i}\right|.$$

MAPE is unstable when an observation is close to zero.

Example 10.5.1. Suppose that

$$y = (10, 12, 15)$$

and

$$\widehat{y} = (11, 12, 14).$$

The residuals are

$$e = (-1, 0, 1).$$

Therefore,

$$\mathrm{MAE} = \frac{2}{3} \approx 0.667,$$

$$\mathrm{RMSE} = \sqrt{\frac{2}{3}} \approx 0.816,$$

and

$$\mathrm{MAPE} = \frac{100\%}{3}\left(\frac{1}{10} + \frac{1}{15}\right) \approx 5.56\%.$$

Residual plots may reveal

- curvature, suggesting missing nonlinear structure;
- increasing spread, suggesting nonconstant variance;
- temporal patterns, suggesting serial dependence;
- isolated large residuals, suggesting unusual observations;
- different error distributions across groups.

A large R^2 does not guarantee a valid model. A regression may have high R^2 while violating its assumptions, extrapolating poorly, or confusing association with causation.

10.6 Training, Validation, and Generalization

In predictive modeling, the observations used to evaluate the final model should not also be used to fit or select it.
A common division is

- a training set for parameter estimation;
- a validation set for model and hyperparameter selection;
- a test set for final performance assessment.

In K-fold cross-validation, the observations are divided into K folds. Each fold is used once for validation while the others are used for training:

$$\mathrm{CV} = \frac{1}{K}\sum_{k=1}^{K} \mathrm{Error}_k \, .$$

For time-series data, random folds are generally inappropriate. Training must precede validation in time. Rolling-origin evaluation repeatedly predicts future observations using only earlier information.

Example 10.6.1. Suppose that monthly sales from 2020 through 2025 are available. Randomly placing December 2025 in the training set and January 2024 in the test set allows future information to influence a prediction of the past. A valid evaluation instead trains on an initial period and predicts later months.

Data leakage also occurs when preprocessing is performed before splitting. The following operations should be fitted using training data only:

- mean and variance used for standardization;
- missing-value imputation;
- feature selection;
- principal component analysis;
- clustering used to construct features;
- classification-threshold selection.

The expected prediction error can be described by the bias–variance decomposition:

$$\mathbb{E}[(Y - \widehat{f}(x))^2] = \sigma^2 + \Big(\text{Bias}[\widehat{f}(x)]\Big)^2 + \text{Var}[\widehat{f}(x)].$$

A model that is too simple may have high bias, while an excessively flexible model may have high variance.

10.7 Sensitivity Analysis

Sensitivity analysis studies how model outputs change when inputs or parameters change. Let $y = f(\theta)$. The local sensitivity to parameter θ_j is

$$\frac{\partial y}{\partial \theta_j}.$$

A dimensionless normalized sensitivity is

$$S_j = \frac{\theta_j}{y}\frac{\partial y}{\partial \theta_j}.$$

It measures the approximate percentage change in y caused by a one-percent change in θ_j.

Example 10.7.1. If

$$y = \theta^2,$$

then

$$\frac{dy}{d\theta} = 2\theta,$$

so

$$S = \frac{\theta}{\theta^2}(2\theta) = 2.$$

Thus, a small one-percent increase in θ produces approximately a two-percent increase in y.
More exactly, increasing θ by 10% changes the output by

$$\frac{(1.1\theta)^2 - \theta^2}{\theta^2} = 0.21,$$

or 21%. Decreasing θ by 10% changes it by

$$\frac{(0.9\theta)^2 - \theta^2}{\theta^2} = -0.19,$$

or -19%.

One-at-a-time sensitivity changes one parameter while holding the others fixed. It is easy to interpret but cannot detect interactions.
A global sensitivity analysis varies all uncertain inputs jointly. Common approaches include

- random Monte Carlo sampling;
- Latin hypercube sampling;

- Morris screening;
- variance-based Sobol indices;
- scenario analysis.

Sensitivity and uncertainty are different. A parameter may be highly influential but known precisely, or weakly influential but highly uncertain.

10.8 Uncertainty Quantification

Model uncertainty may arise from

- measurement error;
- sampling variability;
- uncertain parameters;
- uncertain initial conditions;
- structural disagreement between plausible models;
- unpredictable random variation.

If $\widehat{\theta}$ has covariance matrix Σ and $g(\theta)$ is a smooth model output, the delta method gives

$$\operatorname{Var}[g(\widehat{\theta})] \approx \nabla g(\theta)^{\mathsf{T}} \Sigma \nabla g(\theta).$$

Example 10.8.1. Suppose that

$$y = \frac{a}{b},$$

with $a = 100$, $b = 20$, standard deviations

$$\sigma_a = 5, \qquad \sigma_b = 2,$$

and independent uncertainties. The estimate is $y = 5$, and

$$\nabla y = \left(\frac{1}{b}, -\frac{a}{b^2} \right) = (0.05, -0.25).$$

Therefore,

$$\begin{aligned} \operatorname{Var}(y) &\approx (0.05)^2(5^2) + (-0.25)^2(2^2) \\ &= 0.3125. \end{aligned}$$

The approximate standard deviation is

$$\sqrt{0.3125} \approx 0.559.$$

Monte Carlo uncertainty propagation. Generate parameter samples

$$\theta^{(1)}, \dots, \theta^{(N)}$$

from their uncertainty distributions and calculate

$$y^{(r)} = f(\theta^{(r)}).$$

The empirical distribution of the outputs estimates prediction uncertainty.

Bootstrap. Resample the observations with replacement, refit the model for each resample, and examine the resulting distribution of estimates. The bootstrap is useful when analytical standard-error formulas are difficult to obtain.

A confidence interval for a parameter and a prediction interval for a future observation are not the same. A prediction interval is generally wider because it includes both parameter uncertainty and future random variation.

10.9 Model Comparison and Parsimony

Models should be compared using criteria appropriate to the objective.
For explanatory models, comparison may involve

- interpretability;
- compatibility with scientific mechanisms;
- uncertainty of parameter estimates;
- residual assumptions;
- identifiability.

For predictive models, comparison should emphasize out-of-sample performance, calibration, and robustness.
Likelihood-based model-selection criteria include

$$\mathrm{AIC} = -2\ell + 2k$$

and

$$\mathrm{BIC} = -2\ell + k \ln n,$$

where ℓ is the maximized log-likelihood, k is the number of estimated parameters, and n is the sample size. Smaller values are preferred among models fitted to the same response data.

Example 10.9.1. Suppose that $n = 100$. Model A has $k_A = 3$ parameters and maximized log-likelihood $\ell_A = -120$, while model B has $k_B = 6$ and $\ell_B = -116$. Then

$$\mathrm{AIC}_A = 240 + 6 = 246,$$

$$\mathrm{AIC}_B = 232 + 12 = 244.$$

AIC slightly prefers model B. However,

$$\mathrm{BIC}_A = 240 + 3 \ln 100 \approx 253.82,$$

while

$$\mathrm{BIC}_B = 232 + 6 \ln 100 \approx 259.63.$$

BIC prefers the simpler model A.

A more complicated model should be preferred only when its additional complexity produces a meaningful improvement in explanation, prediction, or decision quality.

Remark 10.9.2. AIC and BIC are relative criteria. Their numerical values do not show that either candidate model is absolutely adequate.

10.10 Robust Decisions and Scenario Analysis

A robust decision performs acceptably across several plausible scenarios. Suppose that decision x produces outcome $f(x, \theta)$, where θ is uncertain.
A worst-case robust model may use

$$\max_{x \in \mathcal{F}} \min_{\theta \in \Theta} f(x, \theta).$$

If probabilities are available, an expected-value model may use

$$\max_{x \in \mathcal{F}} \mathbb{E}_\theta[f(x, \theta)].$$

These criteria can produce different decisions.

Example 10.10.1. Two plans have profits under low, medium, and high demand:

Plan	Low	Medium	High
A	80	120	150
B	100	115	125

The worst-case profits are

$$\min A = 80, \qquad \min B = 100,$$

so the maximin criterion selects plan B.
If all three scenarios are equally likely, the expected profits are

$$\mathbb{E}[A] = \frac{80 + 120 + 150}{3} \approx 116.67,$$

$$\mathbb{E}[B] = \frac{100 + 115 + 125}{3} \approx 113.33.$$

The expected-value criterion selects plan A. Thus, the recommendation depends on the decision maker's attitude toward uncertainty.

When several models are similarly plausible, an ensemble forecast may be used:

$$\widehat{y} = \sum_{k=1}^{M} w_k \widehat{y}^{(k)}, \qquad w_k \geq 0, \qquad \sum_{k=1}^{M} w_k = 1.$$

The weights may be equal or selected using validation performance.

10.11 Structure of a Mathematical Modeling Paper

A mathematical modeling paper should present a transparent chain from the original problem to the final conclusion.
A useful structure is

1. **Abstract:** summarize the problem, models, main results, and conclusions;
2. **Problem restatement:** clarify the objective in precise language;
3. **Assumptions:** state the simplifications and explain why they are reasonable;
4. **Notation:** define the main variables, parameters, sets, and units;
5. **Data preprocessing:** explain missing values, abnormal observations, transformations, and data sources;
6. **Model formulation:** derive the equations from the practical mechanism;
7. **Solution method:** describe the analytical or numerical procedure;
8. **Results:** present the numerical results clearly;

9. **Validation:** evaluate fit, prediction, sensitivity, and robustness;
10. **Discussion:** interpret the practical meaning and compare alternative models;
11. **Strengths and limitations:** state what the model captures and what it omits;
12. **Conclusion:** answer the original question directly.

The derivation should explain why the model is reasonable, not merely list standard formulas.

Example 10.11.1. A weak description is:

> We use TOPSIS to evaluate the alternatives and obtain the following ranking.

A stronger description is:

> Because the alternatives are evaluated by several indicators with different units and no single indicator is decisive, we first orient and normalize the indicators. We then apply weighted TOPSIS to measure each alternative's relative closeness to the ideal and anti-ideal solutions. Entropy weights are used to reflect the observed discriminatory information of the indicators, and the resulting ranking is tested under perturbed weights.

10.12 Figures, Tables, and Reproducibility

A figure should communicate a specific relationship that is difficult to see from a table alone. Every figure should have

- labeled axes;
- physical units;
- a readable legend;
- an informative caption;
- colors or line styles that remain distinguishable;
- a scale that does not distort the conclusion.

A table is preferable when exact numerical values are important. Numerical precision should reflect actual uncertainty. Reporting many decimal places does not make an estimate more accurate.

Example 10.12.1. If a model parameter is estimated as 2.317846 with standard error 0.24, reporting all six decimal places is misleading. A more appropriate report is

$$\widehat{\theta} = 2.32$$

with standard error 0.24, or a corresponding confidence interval.

A reproducible analysis should record

- data sources and access dates;
- variable definitions and units;

- all preprocessing operations;
- model formulas and parameter values;
- software and solver settings;
- random-number seeds;
- convergence tolerances;
- code required to reproduce tables and figures.

The final conclusion should distinguish clearly among

- results directly calculated from data;
- predictions conditional on model assumptions;
- interpretations or practical recommendations;
- limitations and unresolved uncertainty.

10.13 MATLAB Implementation

The following code calculates common fitting and prediction errors.

```
observed = [10; 12; 15];
predicted = [11; 12; 14];

residual = observed-predicted;

MAE = mean(abs(residual));
MSE = mean(residual.^2);
RMSE = sqrt(MSE);

nonzeroMask = observed ~= 0;
MAPE = 100*mean(abs( ...
    residual(nonzeroMask) ...
    ./observed(nonzeroMask)));

fprintf('MAE = %.6f\n', MAE);
fprintf('MSE = %.6f\n', MSE);
fprintf('RMSE = %.6f\n', RMSE);
fprintf('MAPE = %.4f percent\n', MAPE);

figure;
subplot(2,1,1);
plot(observed,'ko-', ...
    'LineWidth',1.2);
hold on;
plot(predicted,'r*-', ...
    'LineWidth',1.2);

legend('Observed','Predicted', ...
    'Location','best');
title('Observed and Predicted Values');
grid on;
```

```
32
33 subplot(2,1,2);
34 stem(residual,'filled');
35 yline(0,'k--');
36 xlabel('Observation');
37 ylabel('Residual');
38 title('Residuals');
39 grid on;
```

Listing 10.1: Residual and prediction-error measures

The following code performs one-at-a-time sensitivity analysis.

```
1  baselineTheta = 10;
2  perturbation = linspace(-0.2,0.2,41);
3
4  thetaValue = baselineTheta ...
5      *(1+perturbation);
6
7  % Example output y = theta^2
8  outputValue = thetaValue.^2;
9  baselineOutput = baselineTheta^2;
10
11 relativeOutputChange = ...
12     outputValue/baselineOutput-1;
13
14 figure;
15 plot(100*perturbation, ...
16     100*relativeOutputChange, ...
17     'b-', 'LineWidth',1.5);
18
19 xlabel('Parameter Change (%)');
20 ylabel('Output Change (%)');
21 title('Sensitivity of y = theta^2');
22 grid on;
23
24 normalizedSensitivity = 2;
25
26 fprintf('Local normalized sensitivity = %.2f\n', ...
27     normalizedSensitivity);
```

Listing 10.2: One-at-a-time sensitivity analysis

Monte Carlo sampling can propagate parameter uncertainty through a model.

```
1 rng(2026);
2
3 sampleSize = 100000;
4
5 % Independent uncertain parameters
6 aSample = 100+5*randn(sampleSize,1);
7 bSample = 20+2*randn(sampleSize,1);
8
9 % Remove physically invalid denominator samples
```

```
validMask = bSample > 0;
aSample = aSample(validMask);
bSample = bSample(validMask);

outputSampleSample = aSample./bSample;

meanOutput = mean(outputSample);
standardDeviation = std(outputSample);

interval = prctile( ...
    outputSample,[2.5,97.5]);

fprintf('Mean output = %.6f\n', meanOutput);
fprintf('Output standard deviation = %.6f\n', ...
    standardDeviation);
fprintf('95 percent interval = [%.6f, %.6f]\n', ...
    interval(1),interval(2));

figure;
histogram(outputSample,60, ...
    'Normalization','pdf');

xlabel('Model Output');
ylabel('Estimated Density');
title('Monte Carlo Output Distribution');
grid on;
```

Listing 10.3: Monte Carlo uncertainty propagation

The following code uses the nonparametric bootstrap to estimate uncertainty in a sample mean.

```
rng(2026);

data = [4; 5; 6; 8; 12];
n = numel(data);

numberOfBootstrapSamples = 10000;
bootstrapMean = zeros( ...
    numberOfBootstrapSamples,1);

for b = 1:numberOfBootstrapSamples
    index = randi(n,n,1);
    bootstrapSample = data(index);

    bootstrapMean(b) = ...
        mean(bootstrapSample);
end

confidenceInterval = prctile( ...
    bootstrapMean,[2.5,97.5]);

fprintf('Sample mean = %.6f\n', mean(data));
```

```
fprintf('Bootstrap interval = [%.6f, %.6f]\n', ...
    confidenceInterval(1), ...
    confidenceInterval(2));

figure;
histogram(bootstrapMean,50, ...
    'Normalization','pdf');

xlabel('Bootstrap Mean');
ylabel('Estimated Density');
title('Bootstrap Distribution');
grid on;
```

Listing 10.4: Bootstrap confidence interval

AIC and BIC can be calculated from the maximized log-likelihood.

```
sampleSize = 100;

logLikelihoodA = -120;
parameterCountA = 3;

logLikelihoodB = -116;
parameterCountB = 6;

AICA = -2*logLikelihoodA ...
    + 2*parameterCountA;

AICB = -2*logLikelihoodB ...
    + 2*parameterCountB;

BICA = -2*logLikelihoodA ...
    + parameterCountA*log(sampleSize);

BICB = -2*logLikelihoodB ...
    + parameterCountB*log(sampleSize);

fprintf('Model A: AIC = %.4f, BIC = %.4f\n', ...
    AICA,BICA);
fprintf('Model B: AIC = %.4f, BIC = %.4f\n', ...
    AICB,BICB);
```

Listing 10.5: AIC and BIC model comparison

The following code compares decisions across several scenarios.

```
% Rows are plans and columns are scenarios
profit = [80, 120, 150;
          100, 115, 125];

scenarioProbability = [1/3; 1/3; 1/3];

expectedProfit = ...
    profit*scenarioProbability;
```

```
worstCaseProfit = ...
    min(profit,[],2);

[bestExpectedProfit,bestExpectedPlan] = ...
    max(expectedProfit);

[bestWorstProfit,bestRobustPlan] = ...
    max(worstCaseProfit);

fprintf('Expected-value plan = %d\n', ...
    bestExpectedPlan);
fprintf('Expected profit = %.4f\n', ...
    bestExpectedProfit);

fprintf('Maximin plan = %d\n', ...
    bestRobustPlan);
fprintf('Worst-case profit = %.4f\n', ...
    bestWorstProfit);
```

Listing 10.6: Scenario and robust-decision analysis

A simple K-fold cross-validation procedure for a regression model is shown below.

```
X = randn(100,4);
trueBeta = [2; -1; 0.5; 0];
y = 3+X*trueBeta+randn(100,1);

K = 5;
partition = cvpartition( ...
    size(X,1), 'KFold', K);

foldRMSE = zeros(K,1);

for k = 1:K
    trainingIndex = training(partition,k);
    validationIndex = test(partition,k);

    XTraining = X(trainingIndex,:);
    yTraining = y(trainingIndex);

    XValidation = X(validationIndex,:);
    yValidation = y(validationIndex);

    % Preprocessing is fitted only
    % on the current training fold
    predictorMean = mean(XTraining,1);
    predictorStd = std(XTraining,0,1);

    XTrainingStandard = ...
        (XTraining-predictorMean) ...
        ./predictorStd;

```

```
    XValidationStandard = ...
        (XValidation-predictorMean) ...
        ./predictorStd;

    model = fitlm( ...
        XTrainingStandard,yTraining);

    prediction = predict( ...
        model,XValidationStandard);

    foldRMSE(k) = sqrt(mean( ...
        (yValidation-prediction).^2));
end

crossValidatedRMSE = mean(foldRMSE);

fprintf('Cross-validated RMSE = %.6f\n', ...
    crossValidatedRMSE);
```

Listing 10.7: K-fold cross-validation

Remark 10.13.1. A model should not be described as successful merely because it fits the available data. A credible modeling conclusion combines mathematical correctness, empirical validation, sensitivity analysis, uncertainty quantification, practical interpretation, and transparent reporting.

Acknowledgments

I would first like to express my sincere gratitude to the Bilibili creator “Dashixiong,” whose online lectures provided the main foundation for my self-study of many of the mathematical modeling methods collected in these notes. His explanations and examples helped me build an initial understanding of a wide range of commonly used models and their computational implementations.

I am also grateful to the official organizers of the China Undergraduate Mathematical Contest in Modeling and the Mathematical Contest in Modeling and Interdisciplinary Contest in Modeling. Their competition problems, official materials, and long-standing efforts to promote mathematical modeling have provided valuable opportunities for me to apply mathematical ideas to complex real-world problems. Many of the additional methods, examples, and modeling perspectives included in these notes were motivated by my preparation for and participation in these competitions, as well as by my reading of previous competition papers.

I would especially like to thank my teammates Tong Wu and Youcheng Fang, with whom I participated in the China Undergraduate Mathematical Contest in Modeling, and Yuhan Rui and Yibin Lou, with whom I participated in the Mathematical Contest in Modeling. Our discussions, division of work, repeated model revisions, numerical experiments, and joint writing allowed me to develop a much more practical understanding of how mathematical models are constructed, tested, improved, and communicated under competition conditions.

Finally, I am grateful for every modeling problem that required us to move beyond familiar formulas and confront the uncertainty, incompleteness, and complexity of real data. These experiences motivated me to organize the present notes as a systematic reference to the principles, applications, limitations, and computational implementation of commonly used mathematical models.